

GraphRAG+多模态RAG+Ragas的项目开发

讲师：肖斌

第一章、GraphRAG的相关概念

1. GraphRAG的定义

GraphRAG (Graph-based Retrieval-Augmented Generation) 是一种结合**知识图谱**与**检索增强生成 (RAG)** 的技术框架，旨在通过图结构数据增强大语言模型 (LLMs) 的推理能力和回答准确性

核心组件与技术

(1) 知识图谱构建

- 节点与边**：节点表示实体（如人物、概念），边表示关系（如“影响”“属于”）。
- 自动化构建**：通过LLM从非结构化文本中抽取实体和关系（如“莫奈→创作→《睡莲》”），并存储于图数据库（Neo4j等）。
- 社区检测**：使用Leiden算法将图谱聚类为主题社区（如“艺术运动”“科技公司”），并为每个社区生成摘要。

(2) 图检索与推理

- 检索机制**：结合图遍历（如广度优先搜索）和向量相似度，定位相关子图。
- 多跳推理**：沿图谱边路径推导答案（如“供应链中断→芯片短缺→汽车减产”）。

(3) 生成优化

- 图增强生成**：LLM结合检索到的子图结构和文本片段生成回答，提升逻辑性和可解释性

2. 与传统RAG的区别：

传统RAG

- 本质**：通过向量化文本片段实现语义检索，辅助大语言模型 (LLM) 生成回答。
- 目标**：快速响应简单问答，如FAQ检索、文档片段匹配。
- 局限性**：无法处理多实体关系、多跳推理，且文本分块易导致上下文断裂。

GraphRAG

- 本质**：将知识图谱（节点+边）融入检索与生成过程，实现结构化推理。
- 目标**：解决复杂查询（如因果分析、跨领域推理），提升可解释性。

- **突破性**：支持动态关系组合（如“治疗+副作用比较”）和多跳路径追溯。

维度	传统RAG	GraphRAG
技术本质	基于文本片段的语义检索与生成	基于知识图谱的结构化推理与生成
知识组织方式	扁平化文档/向量片段	节点（实体）+ 边（关系）的图结构
设计目标	快速问答、简单信息检索	复杂推理、多跳分析与可解释性回答

特性	传统RAG	GraphRAG
检索方式	余弦相似度匹配	图遍历+向量混合检索（如Cypher/SPARQL查询）
推理能力	单跳检索（Shallow Matching）	多跳推理（如疾病→药物→副作用→替代方案）
上下文构建	拼接相似文本片段	整合子图结构（节点+关系路径）

局限性

问题	传统RAG	GraphRAG
实现复杂度	低（开源生态成熟）	高（需专业图谱构建团队）
动态更新	支持实时文档更新	图谱维护成本高（需重新抽取关系）
计算成本	向量检索效率高	图遍历计算密集（需GPU加速）

3. Microsoft GraphRAG相关概念

微软的GraphRAG 是一种结构化的、分层的检索增强生成 (RAG) 方法，而不是使用纯文本片段的简单语义搜索方法。GraphRAG 过程包括从原始文本中提取知识图、构建社区层次结构、生成这些社区的摘要，然后在执行基于 RAG 的任务时利用这些结构。支持文本文件（如TXT、CSV）。

时间	事件	关键变更
2024-04	ArXiv 论文公开	首次提出“From Local to Global”的图-RAG 思想
2024-07-02	GitHub 开源 v1.0	当天冲上 6 k star；提供 CLI + Python SDK
2024-12	v2.0 大版本	重构工作流、引入 Children-Community 结构，支持增量索引
2025-03	v2.2	支持 OpenAI-o1 等推理模型；新增向量存储快照
2025-07-15	v2.4.0（社区俗称 2.5）	可注入自定义 Pipeline；StorageFactory 注册式插件；依赖库 CVE 清理

GraphRAG 知识模型

知识模型是符合我们数据模型定义的数据输出规范。

- Document

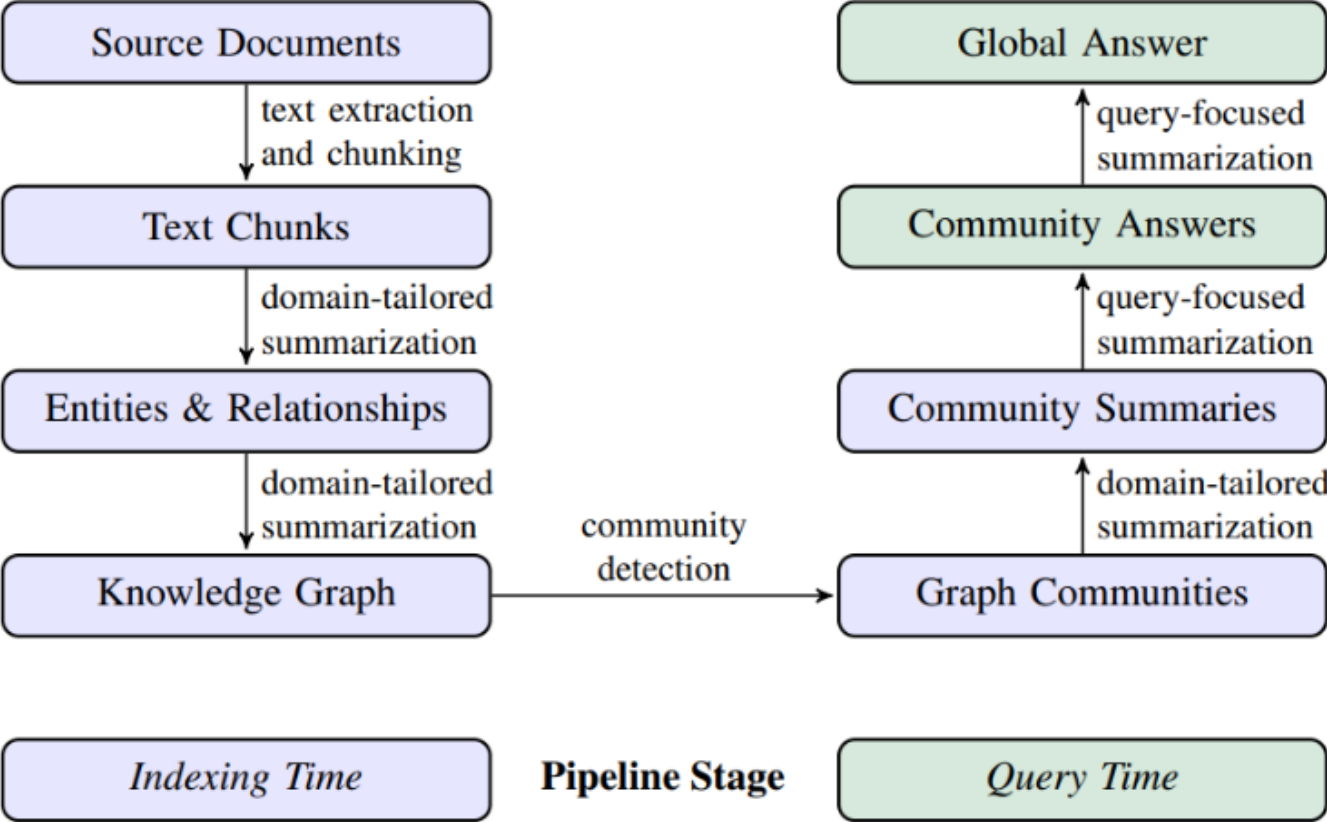
 - 系统中的输入文档。 这些代表 CSV 中的单个行或单个 .txt 文件。

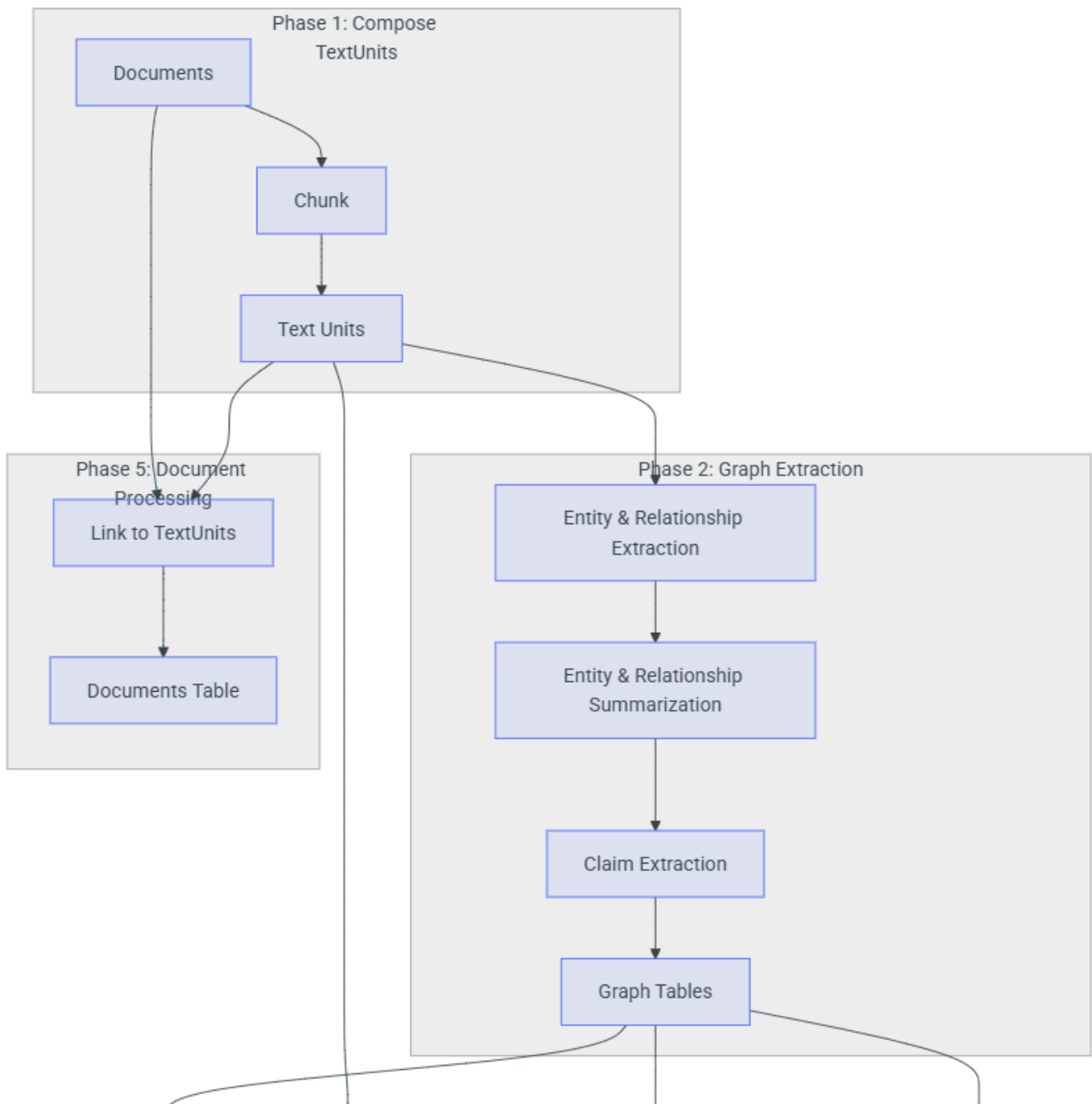
- `TextUnit` - 要分析的文本块。这些块的大小、它们的重叠以及它们是否遵守任何数据边界都可以在下面配置。一个常见的用例是将 `CHUNK_BY_COLUMNS` 设置为 `id`，以便文档和 `TextUnit` 之间存在一对多的关系，而不是多对多的关系。
- `Entity` - 从 `TextUnit` 提取的实体。这些代表人员、地点、事件或您提供的其他实体模型。
- `Relationship` - 两个实体之间的关系。
- `Covariate` - 提取的声明信息，其中包含关于实体且可能受时间限制的声明。
- `Community` - 一旦构建了实体和关系的图，我们就会对它们执行分层社群检测，以创建聚类结构。
- `Community Report` - 每个社群的内容被总结成一个生成的报告，对人工阅读和下游搜索有用。

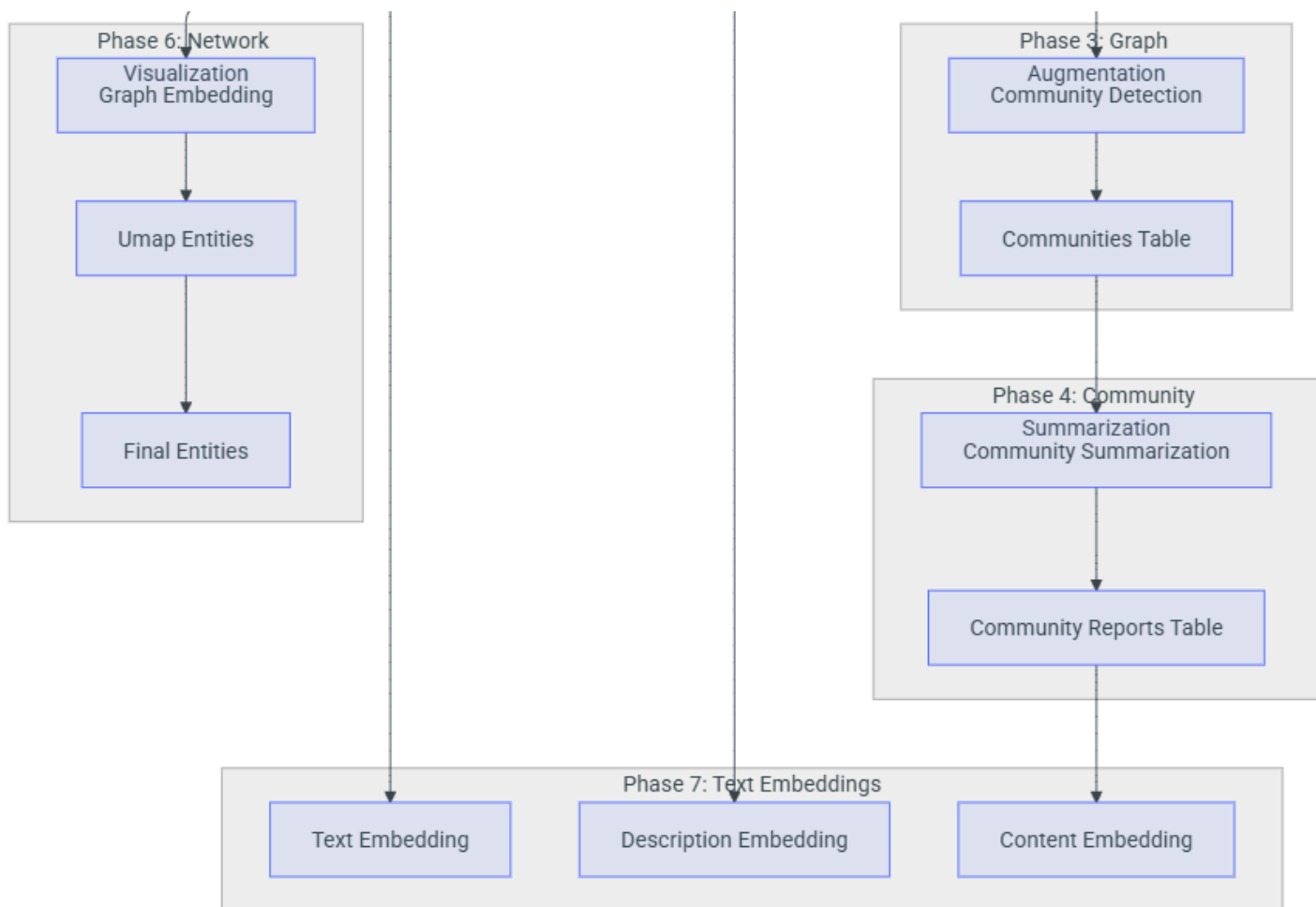
标准方法对所有推理任务都使用语言模型

- 实体提取：提示 LLM 提取命名实体，并从每个文本单元提供描述。
- 关系提取：提示 LLM 描述每个文本单元中每对实体之间的关系。
- 实体摘要：提示 LLM 将在文本单元中找到的每个实体的描述组合成一个摘要。
- 关系摘要：提示 LLM 将在文本单元中找到的每个关系的描述组合成一个摘要。
- 声明提取（可选）：提示 LLM 从每个文本单元中提取和描述声明。
- 社区报告生成：收集每个社区的实体和关系描述（以及可选的声明），并用于提示 LLM 生成摘要报告。

GraphRAG 的工作流程





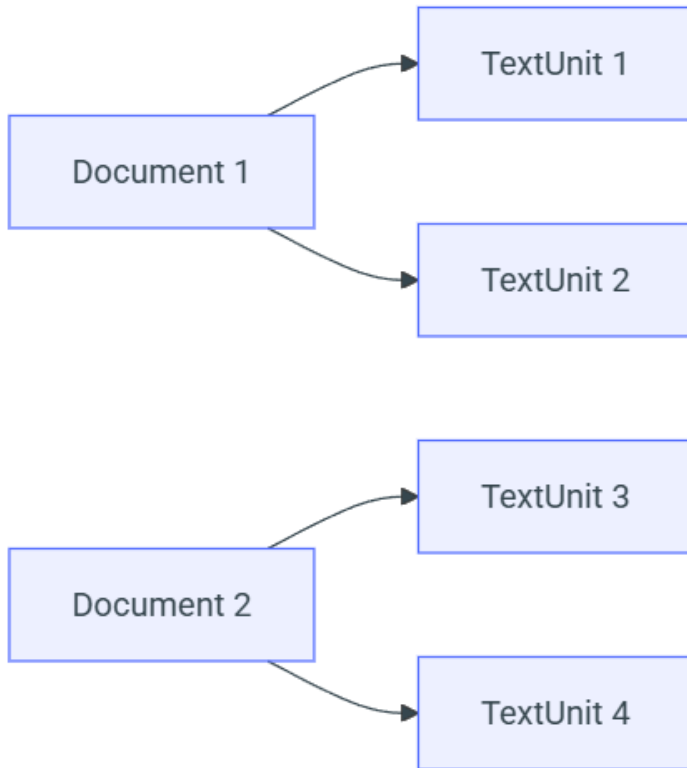


阶段 1：组合 TextUnit

默认配置工作流程的第一阶段是将输入文档转换为 *TextUnit*。*TextUnit* 是一块用于我们的图提取技术的文本。它们也用作提取的知识项的来源参考，以便通过概念将面包屑和出处授权回其原始来源文本。

块大小（以令牌计数）是用户可配置的。默认情况下，这设置为 300 个令牌，尽管我们在使用单个“glean”步骤（“glean”步骤是后续提取）时获得了积极的经验（使用 1200 个令牌块）。较大的块会导致较低保真度的输出和不太有意义的参考文本；但是，使用较大的块可以大大缩短处理时间。

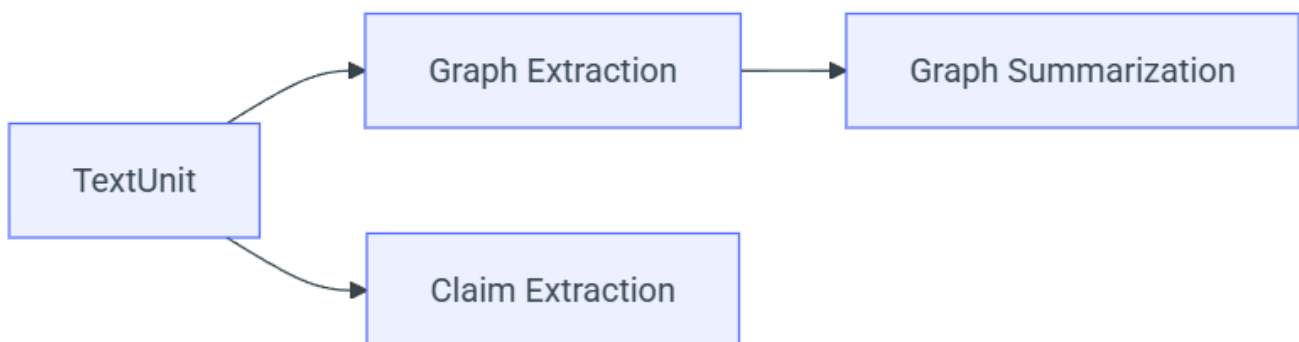
Documents into Text Chunks



阶段 2：图提取

在此阶段，我们分析每个文本单元并提取我们的图原语：**实体**、**关系**和**声明**。实体和关系在我们的 *entity_extract* 动词中一次性提取，声明在我们的 *claim_extract* 动词中提取。然后将结果组合并传递到管道的后续阶段。

Graph Extraction



阶段 3：图聚类

现在我们有了一个可用的实体和关系图，我们希望了解它们的社群结构。这些为我们提供了理解图的拓扑结构的明确方法。在此步骤中，我们使用分层莱顿算法（Leiden）生成实体社群的层次结构。此方法会将递归社群聚类应用于我们的图，直到我们达到社群大小阈值。这将使我们能够了解图的社群结构，并提供一种在不同粒度级别导航和总结图的方法。完成步骤后，最终的**实体**、**关系**和**社群**表将被导出。

Graph Augmentation



阶段 4：社群概括（摘要）

在此步骤中，我们使用 LLM 生成每个社群的摘要。这将使我们能够了解每个社群中包含的不同信息，并提供对图的范围理解，无论是从高级还是低级角度。这些报告包含一份执行概要，并引用社群子结构中的关键实体、关系和声明。

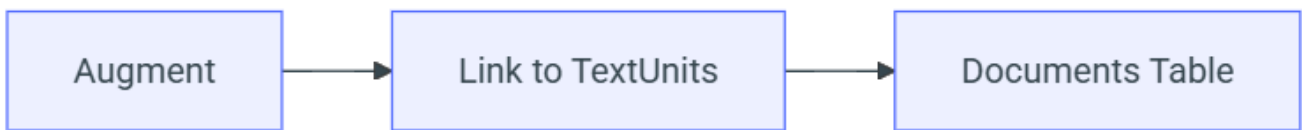
Community Summarization



阶段 5：文档处理

在此工作流程阶段，我们为知识模型创建文档表。在此步骤中，我们将每个文档链接到第一阶段中创建的文本单元。这使我们能够了解哪些文档与哪些文本单元相关，反之亦然。

Document Processing



阶段 6：网络可视化（可选）

在此工作流程阶段，我们执行一些步骤来支持现有图中高维向量空间的网络可视化。

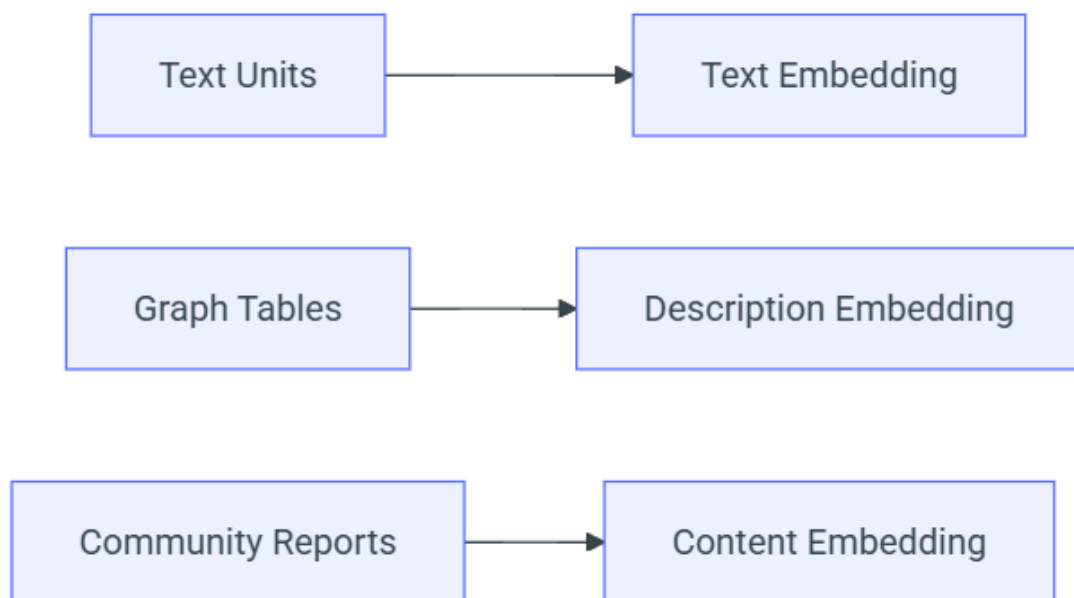
Network Visualization Workflows



阶段 7：文本嵌入

对于所有需要下游向量搜索的工件，我们生成文本嵌入作为最后一步。这些嵌入直接写入配置的向量存储。默认情况下，我们嵌入实体描述、文本单元文本和社群报告文本。

Text Embedding Workflows



构建索引后有输出表：<https://msdocs.cn/graphrag/index/outputs/>

可视化：https://msdocs.cn/graphrag/visualization_guide/#3-open-the-graph-in-gephi

GraphRAG的查询引擎

查询引擎是Graph RAG库的检索模块。它是Graph RAG库的两个主要组成部分之一，另一个是索引流程管道。

- Local Search
- Global Search
- DRIFT Search
- Question Generation

Microsoft GraphRAG 提供local 和global 两种查询方式，分别对应local search 和global search。是源于不同的粒度级别而构建出来用于处理不同类型问题的Pipeline, 其中：

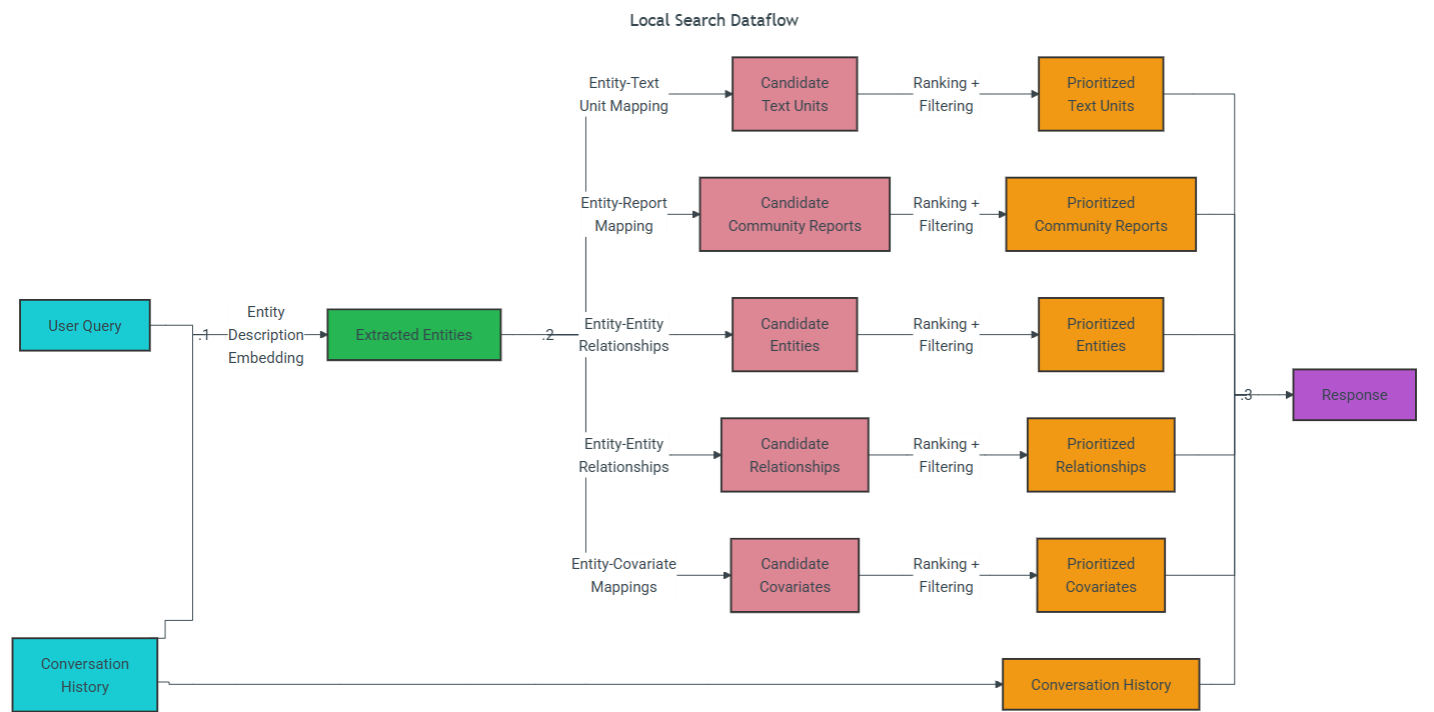
- Local Search 是基于实体的检索。
- Global Search 则是基于社区的检索。

Microsoft GraphRAG 在查询阶段构建的流程，相较于构建索引阶段会更为直观。核心的具体步骤包括：

- 接收用户的查询请求。
- 根据查询所需的详细程度，选择合适的社区级别进行分析。
- 在选定的社区级别进行信息检索。
- 依据社区摘要生成初步的响应。
- 将多个相关社区的初步响应进行整合，形成一个全面的最终答案。

本地搜索

本地搜索方法将知识图谱中的结构化数据与输入文档中的非结构化数据结合起来，在查询时使用相关实体信息来增强 LLM 上下文。它非常适合回答需要理解输入文档中提到的特定实体的问题（例如，“洋甘菊的治疗特性是什么？”）。



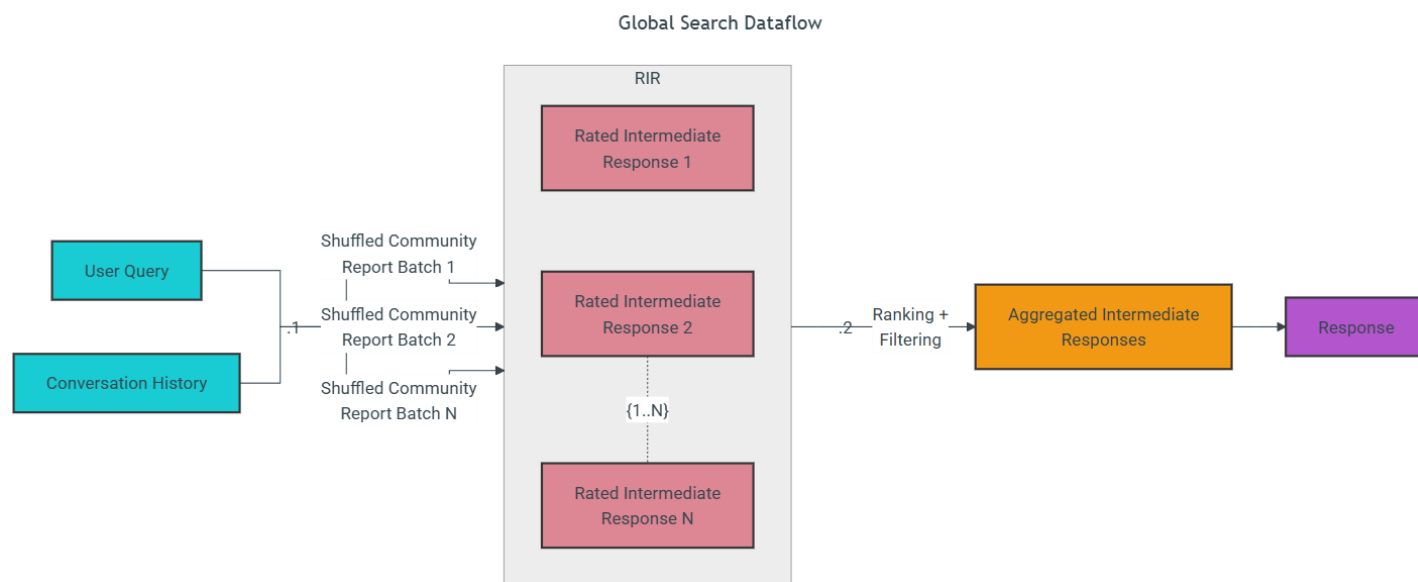
详细过程：给定用户查询，以及可选的对话历史记录，本地搜索方法从知识图谱中识别出一组与用户输入语义相关的实体。这些实体作为访问知识图谱的入口点，可以提取更多相关详细信息，例如连接的实体、关系、实体协变量和社区报告。此外，它还会从与已识别实体相关的原始输入文档中提取相关的文本块。然后对这些候选数据源进行优先级排序和过滤，以适应预定义大小的单个上下文窗口，该窗口用于生成对用户查询的响应。

以下是 `LocalSearch` 类的关键参数

- `llm`：用于生成响应的 OpenAI 模型对象
- `context_builder`：用于准备来自知识模型对象集合的上下文数据的上下文构建器对象
- `system_prompt`：用于生成搜索响应的提示模板。默认模板可以在 `system_prompt` 中找到
- `response_type`：描述所需响应类型和格式的自由形式文本（例如，`多个段落`，`多页报告`）
- `llm_params`：要传递给 LLM 调用的其他参数的字典（例如，温度、`max_tokens`）

- `context_builder_params`：构建搜索提示的上下文时，要传递给 `context_builder` 对象的其他参数的字典
- `callbacks`：可选的回调函数，可用于为 LLM 的完成流事件提供自定义事件处理程序

全局搜索



搜索过程：给定用户查询，以及可选的对话历史，全局搜索方法使用来自图形社区层次结构的指定级别的 LLM 生成的社区报告集合作为上下文数据，以 map-reduce 的方式生成响应。在 `map` 步骤中，社区报告被分割成预定义大小的文本块。然后，每个文本块用于生成包含一系列要点的中间响应，每个要点都附有数值评分，指示该要点的重要性。在 `reduce` 步骤中，聚合从中间响应中筛选出的一组最重要的要点，并将其用作上下文以生成最终响应。

全局搜索响应的质量可能会受到为获取社区报告而选择的社区层次结构级别的严重影响。较低的层次结构级别及其详细的报告往往会产生更彻底的响应，但也可能由于报告数量的增加而增加生成最终响应所需的时间和 LLM 资源。

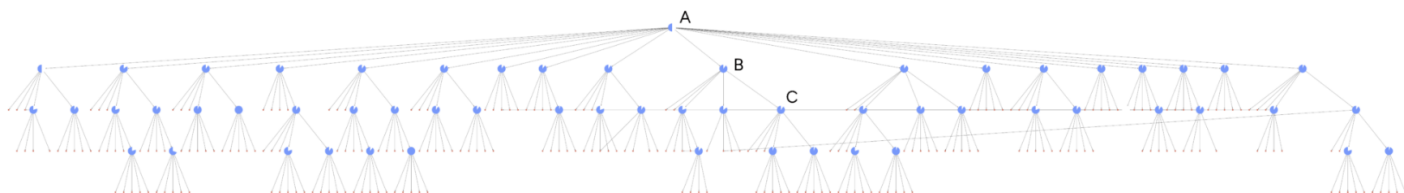
以下是 `GlobalSearch` 类的关键参数

- `llm`：用于生成响应的 OpenAI 模型对象
- `context_builder`：用于从社区报告准备上下文数据的 [context builder](#) 对象
- `map_system_prompt`：`map` 阶段中使用的提示模板。默认模板可以在 [map_system_prompt](#) 中找到
- `reduce_system_prompt`：`reduce` 阶段中使用的提示模板，默认模板可以在 [reduce_system_prompt](#) 中找到
- `response_type`：描述所需响应类型和格式的自由格式文本（例如，`Multiple Paragraphs`、`Multi-Page Report`）

- `allow_general_knowledge`：将此设置为 True 将在 `reduce_system_prompt` 中包含其他说明，以提示 LLM 结合数据集之外的相关真实世界知识。请注意，这可能会增加幻觉，但对于某些场景可能有用。默认为 False * `general_knowledge_inclusion_prompt`：如果启用 `allow_general_knowledge`，则添加到 `reduce_system_prompt` 的指令。默认指令可以在 [general_knowledge_instruction](#) 中找到
- `max_data_tokens`：上下文数据的令牌预算
- `map_llm_params`：要在 `map` 阶段传递给 LLM 调用的附加参数字典（例如，温度、`max_tokens`）
- `reduce_llm_params`：要在 `reduce` 阶段传递给 LLM 调用的附加参数字典（例如，温度、`max_tokens`）
- `context_builder_params`：在为 `map` 阶段构建上下文窗口时，传递给 [context_builder](#) 对象的附加参数字典。
- `concurrent_coroutines`：控制 `map` 阶段的并行度。
- `callbacks`：可选的回调函数，可用于为 LLM 的完成流事件提供自定义事件处理程序。

DRIFT 搜索

DRIFT 搜索（具有灵活遍历的动态推理和推断）建立在微软的 GraphRAG 技术之上，结合了全局搜索和本地搜索的特性，使用我们的 [drift search](#) 方法，以在计算成本和质量结果之间取得平衡的方式生成详细响应。



整个 DRIFT 搜索层级结构，突出显示了 DRIFT 搜索过程的三个核心阶段。

1. A (首先)：DRIFT 将用户的查询与语义最相关的 K 个社区报告进行比较，生成广泛的初始答案和后续问题，以指导进一步的探索。
2. B (后续)：DRIFT 使用本地搜索来优化查询，产生额外的中间答案和后续问题，从而增强特异性，引导引擎找到上下文丰富的信息。图中的每个节点上的字形都显示了算法继续查询扩展步骤的置信度。
3. C (输出层级)：最终输出是一个按相关性排序的问题和答案的层级结构，反映了全局洞察和本地改进的平衡组合，使结果具有适应性和全面性。

DRIFT 搜索通过在搜索过程中包含社区信息，引入了一种新的本地搜索查询方法。这大大扩展了查询起点的广度，并导致在最终答案中检索和使用了更多种类的事实。这种添加通过为本地搜索提供更全面的选项来扩展 GraphRAG 查询引擎，该选项使用社区洞察力将查询细化为详细的后续问题。

以下是 `DRIFTSearch` 类的关键参数

- `llm` : 用于生成响应的 OpenAI 模型对象
- `context_builder` : 用于从社区报告和查询信息准备上下文数据的 [context builder](#) 对象
- `config` : 用于定义 DRIFT 搜索超参数的模型。 [DRIFT Config 模型](#)
- `token_encoder` : 用于跟踪算法预算的令牌编码器。
- `query_state` : 在 [Query State](#) 中定义的状态对象，允许跟踪 DRIFT 搜索实例的执行，以及后续操作和 [DRIFT actions](#)。

Question Generation：基于实体的问题生成

问题生成方法将来自知识图的结构化数据与来自输入文档的非结构化数据相结合，以生成与特定实体相关的候选问题。给定先前用户问题的列表，问题生成方法使用与[本地搜索](#)中使用的相同上下文构建方法来提取和优先处理相关的结构化和非结构化数据，包括实体、关系、协变量、社区报告和原始文本块。然后将这些数据记录拟合到单个 LLM 提示中，以生成候选后续问题，这些问题代表数据中最重要或最紧急的信息内容或主题。

以下是 `Question Generation` 类的关键参数

- `llm` : 用于响应生成的 OpenAI 模型对象
- `context_builder` : [上下文构建器](#)对象，用于准备来自知识模型对象集合的上下文数据，使用与本地搜索中相同的上下文构建器类
- `system_prompt` : 用于生成候选问题的提示模板。默认模板可以在[system_prompt](#)找到
- `llm_params` : 要传递给 LLM 调用的附加参数（例如，温度、max_tokens）的字典
- `context_builder_params` : 在为问题生成提示构建上下文时要传递给 [context_builder](#) 对象的附加参数的字典
- `callbacks` : 可选的回调函数，可用于为 LLM 的完成流事件提供自定义事件处理程序

第二章、Microsoft GraphRAG的安装

1. 黑盒安装和使用

`pip install graphrag` 安装的是“黑盒应用”，拿来即用但改不动；

一、pip 安装 GraphRAG 的硬性要求

表格 复制 导出	
类别	官方 / 社区实测要求
Python	3.10 - 3.12
pip	≥ 23.3
系统	Win10+ / macOS 12+ / Ubuntu 20.04+
磁盘	预留 ≥ 3 GB 空间
网络	能访问 https://pypi.org 、 https://openai.com
LLM	任选其一：① OpenAI GPT-4o/4o-mini② Azure OpenAI③ 本地 vLLM / Ollama④ 其他 OpenAI-API 兼容端点
内存	16 GB RAM 起步；8 GB 也能跑，但速度明显下降

```
Last login: Sat Aug 10 17:15:10 2023 from 175.0.0.1
[root@iZ8vb5acpt0ebqsuf5mtiwZ ~]# python3.12 -m venv graphrag_env
[root@iZ8vb5acpt0ebqsuf5mtiwZ ~]# ls
graphrag_env
[root@iZ8vb5acpt0ebqsuf5mtiwZ ~]# cd graphrag_env/
[root@iZ8vb5acpt0ebqsuf5mtiwZ graphrag_env]# ls
bin include lib lib64 pyvenv.cfg
[root@iZ8vb5acpt0ebqsuf5mtiwZ graphrag_env]# cd bin/
[root@iZ8vb5acpt0ebqsuf5mtiwZ bin]# ls
activate activate.csh activate.fish Activate.ps1 pip pip3 pip3.12 python python3 python3.12
[root@iZ8vb5acpt0ebqsuf5mtiwZ bin]# ll
total 36
-rw-r--r-- 1 root root 2040 Aug 16 18:07 activate
-rw-r--r-- 1 root root 928 Aug 16 18:07 activate.csh
-rw-r--r-- 1 root root 2207 Aug 16 18:07 activate.fish
-rw-r--r-- 1 root root 9033 Aug 16 18:07 Activate.ps1
-rwxr-xr-x 1 root root 238 Aug 16 18:07 pip
-rwxr-xr-x 1 root root 238 Aug 16 18:07 pip3
-rwxr-xr-x 1 root root 238 Aug 16 18:07 pip3.12

1 云服务器
[root@iZ8vb5acpt0ebqsuf5mtiwZ bin]# source ./activate
(graphrag_env) [root@iZ8vb5acpt0ebqsuf5mtiwZ bin]# pip install graphrag
Looking in indexes: http://mirrors.cloud.aliyuncs.com/pypi/simple/
Collecting graphrag
  Downloading http://mirrors.cloud.aliyuncs.com/pypi/packages/49/93/d773256100cdc1b8335928cc4971d97925764a502503rag-2.5.0-py3-none-any.whl (370 kB)
                                     370.4/370.4 kB 2.4 MB/s eta 0:00:00
Collecting environs>=11.0.0 (from graphrag)
  Downloading http://mirrors.cloud.aliyuncs.com/pypi/packages/ad/e3/98d8567eb438c7856a4dcedd97a8a7c6707120a5ada6ons-14.3.0-py3-none-any.whl (16 kB)
Collecting azure-search-documents>=11.5.2 (from graphrag)
  Downloading http://mirrors.cloud.aliyuncs.com/pypi/packages/4b/f5/0f6b52567cbb33f1efba13060514ed7088a86de84d74_search_documents-11.5.3-py3-none-any.whl (298 kB)
                                     298.8/298.8 kB 3.0 MB/s eta 0:00:00
Collecting lancedb>=0.17.0 (from graphrag)
  Downloading http://mirrors.cloud.aliyuncs.com/pypi/packages/9a/b9/3e0e25b7c6dcd4f6b0e977cb886965070ca05d799481db-0.24.3-cp39-abi3-manylinux_2_28_x86_64.whl (35.0 MB)
```

创建工作目录并放入测试数据来构建索引

```
(graphrag_env) [root@iZ8vb5acpt0ebqsuf5mtiwZ ~]# ls
graphrag_env
(graphrag_env) [root@iZ8vb5acpt0ebqsuf5mtiwZ ~]# mkdir -p ./ragtest/input
(graphrag_env) [root@iZ8vb5acpt0ebqsuf5mtiwZ ~]# cp -a /opt/test_rag/input/*.txt ./ragtest/input/
(graphrag_env) [root@iZ8vb5acpt0ebqsuf5mtiwZ ~]# graphrag init --root ./ragtest/
2025-08-16 18:26:46.0057 - INFO - graphrag.cli.initialize - Initializing project at /root/ragtest
(graphrag_env) [root@iZ8vb5acpt0ebqsuf5mtiwZ ~]#
```



```
2025-08-16 18:26:46.0057 - INFO - graphrag.cli.initialize - Initializing project at /root,
(graphrag_env) [root@iZ8vb5acpt0ebqsuf5mtiwZ ~]# cd ./ragtest/
(graphrag_env) [root@iZ8vb5acpt0ebqsuf5mtiwZ ragtest]# ls
input prompts settings.yaml
(graphrag_env) [root@iZ8vb5acpt0ebqsuf5mtiwZ ragtest]# vi settings.yaml
```

发评论本到当前Ychall窗口的全部会话

```
models:
  default_chat_model:
    type: openai_chat # or azure openai_chat
    api_base: https://xiaoai.plus/v1
    # api_version: 2024-05-01-preview
    auth_type: api_key # or azure managed identity
    api_key: sk-KQk1...p0yWo2mnxhe
    # audience: "https://cognitiveservices.azure.com/.default"
    # organization: <organization_id>
    model: gpt-4.1-mini
    # deployment_name: <azure_model_deployment_name>
    # encoding_model: cl100k_base # automatically set by tiktoken if left undefined
    model_supports_json: true # recommended if this is available for your model.
    concurrent_requests: 25 # max number of simultaneous LLM requests allowed
    async_mode: threaded # or asyncio
    retry_strategy: native
    max_retries: 10
    tokens_per_minute: auto # set to null to disable rate limiting
    requests_per_minute: auto # set to null to disable rate limiting
  default_embedding_model:
    type: openai_embedding # or azure openai_embedding
    api_base: https://xiaoai.plus/v1
    # api_version: 2024-05-01-preview
    auth_type: api_key # or azure managed identity
```

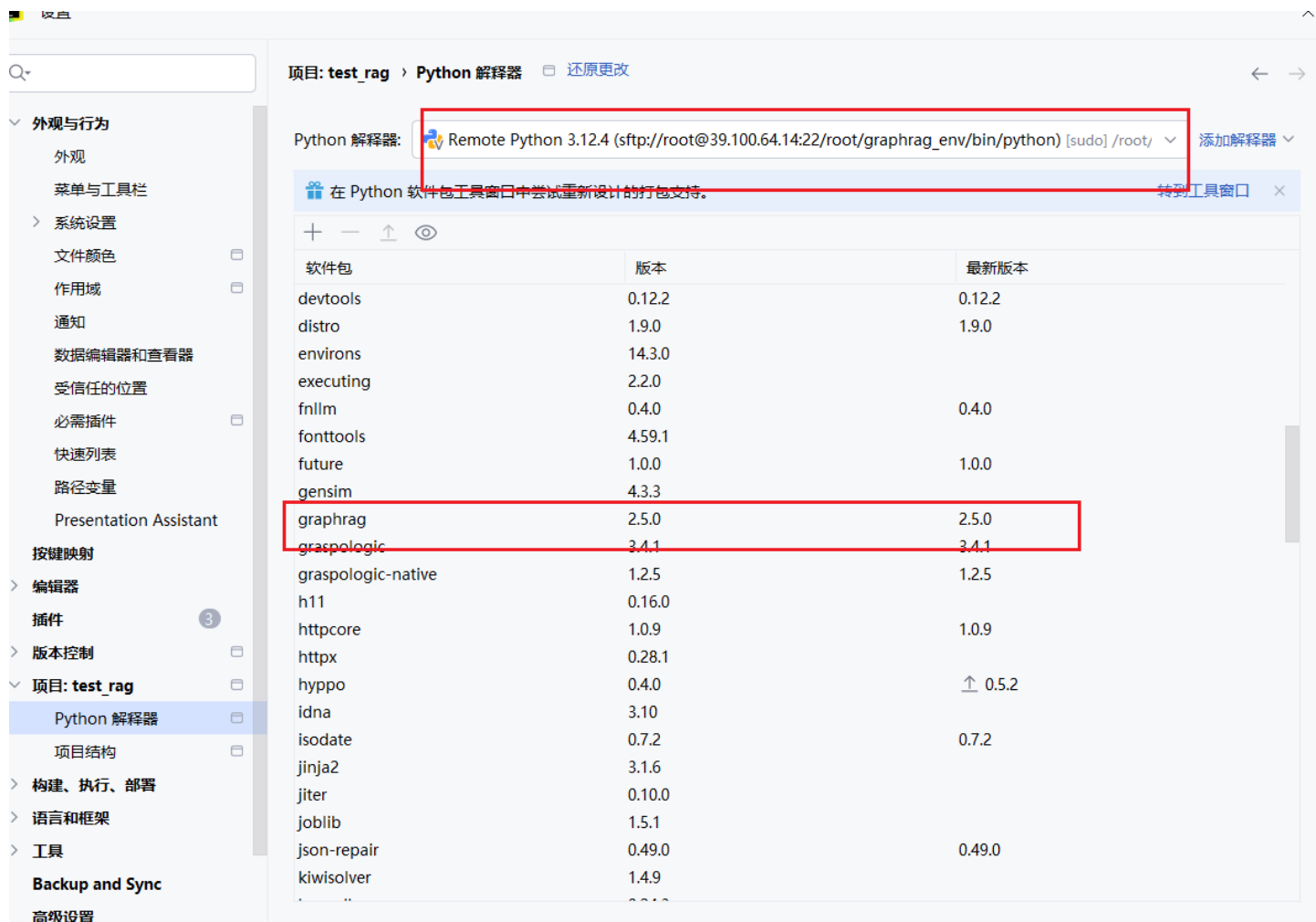
```
graphrag_env) [root@iZ8vb5acpt0ebqsuf5mtiwZ ragtest]# cd ..
graphrag_env) [root@iZ8vb5acpt0ebqsuf5mtiwZ ~]# graphrag index --root ./ragtest/
2025-08-16 18:32:20.0747 - INFO - graphrag.cli.index - Logging enabled at /root/ragtest/logs/logs.txt
2025-08-16 18:32:24.0377 - INFO - graphrag.index.validate_config - LLM Config Params Validated
2025-08-16 18:32:27.0299 - INFO - graphrag.index.validate_config - Embedding LLM Config Params Validated
2025-08-16 18:32:27.0299 - INFO - graphrag.cli.index - Starting pipeline run. False
2025-08-16 18:32:27.0302 - INFO - graphrag.cli.index - Using default configuration: {
  "root_dir": "/root/ragtest",
  "models": {
    "default_chat_model": {
      "api_key": "==== REDACTED ====",
      "auth_type": "api_key",
      "type": "openai_chat",
      "model": "gpt-4.1-mini",
      "encoding_model": "o200k_base",
      "api_base": "https://xiaoai.plus/v1",
      "api_version": null,
```

创建索引

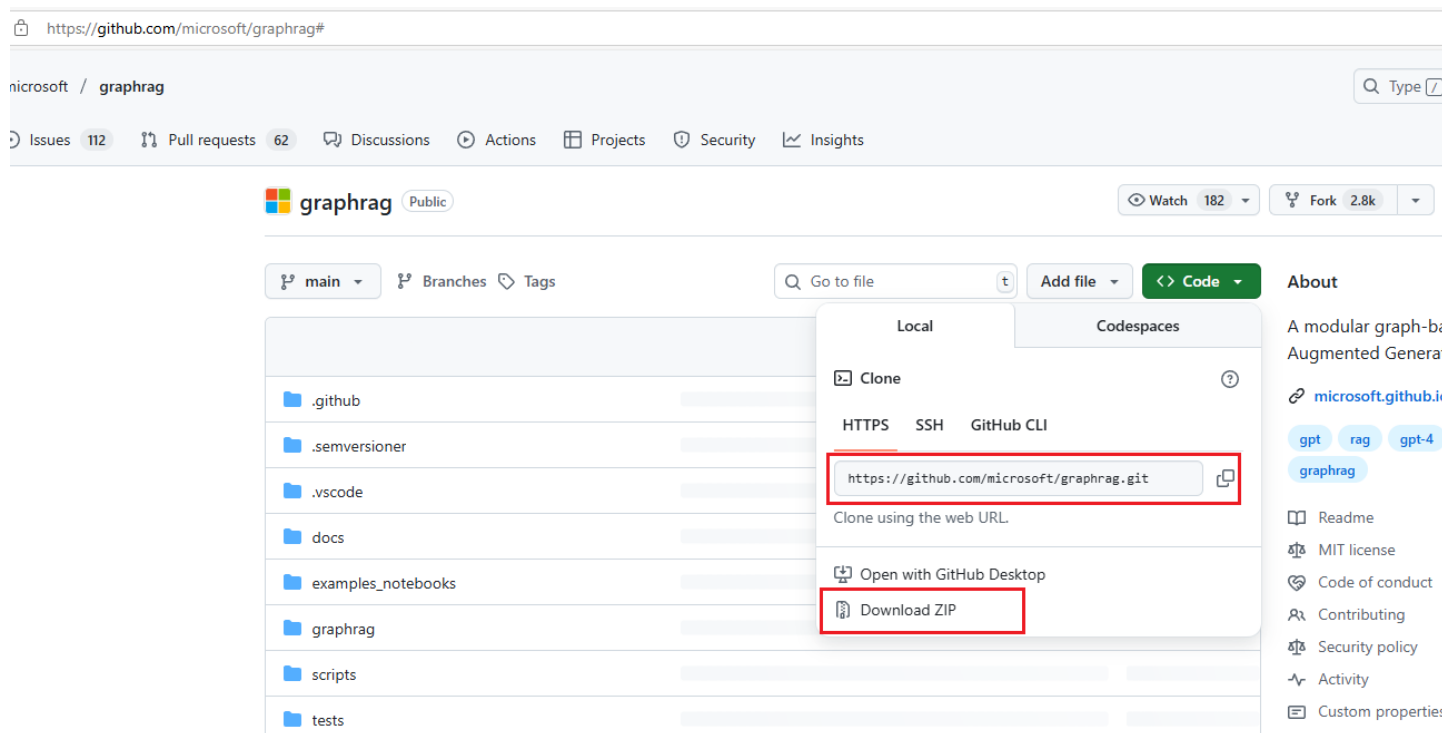
```
(graphrag_env) [root@iZ8vb5acpt0ebqsuf5mtiwZ ragtest]# graphrag query --root /root/ragtest/ --method global --query "马云是谁?"
2025-08-16 18:48:05.0854 - INFO - graphrag.storage.file_pipeline_storage - Creating file storage at /root/ragtest/output
2025-08-16 18:48:05.0855 - INFO - graphrag.utils.storage - reading table from storage: entities.parquet
2025-08-16 18:48:05.0864 - INFO - graphrag.utils.storage - reading table from storage: communities.parquet
2025-08-16 18:48:05.0868 - INFO - graphrag.utils.storage - reading table from storage: community_reports.parquet
2025-08-16 18:48:26.0794 - INFO - graphrag.cli.query - Global Search Response:
# 马云简介
```

马云（Jack Ma）是中国著名企业家，阿里巴巴集团的联合创始人和前董事长。他于1999年创立了阿里巴巴集团，带领公司迅速成长为全球领先的电子商务集团，旗下拥有淘宝、天猫和支付宝等重要子公司。作为中国互联网和电子商务行业的标志性人物，马云在推动中国数字经济发展方面具有举足轻重的地位，对全球电子商务格局也产生了深远影响[Data: Reports (3, 6, 7, 8, 13, +more)]。

创业历程与贡献



2. 源码安装



主页
欢迎
快速入门
开发指南

开发指南

要求

名称	安装	目的
Python 3.10-3.12	下载	该库基于 Python。
Poetry	说明	Poetry 用于 Python 代码库中的包管理和 virtualenv 管理

快速入门

```
inflating: graphrag-main/unified-search-app/pyproject.toml
inflating: graphrag-main/unified-search-app/uv.lock
inflating: graphrag-main/uv.lock
[root@iZ8vb5acpt0ebqsuf5mtiwZ download]# ls
cypher-shell-5.26.8-1.noarch.rpm  graphrag-main.zip  neo4j-5.26.8-1.noarch.rpm  Python-3.12.4.tgz
graphrag-main                    jdk-17.0.16_linux-x64_bin.rpm  Python-3.12.4
[root@iZ8vb5acpt0ebqsuf5mtiwZ download]# cd graphrag-main/
[root@iZ8vb5acpt0ebqsuf5mtiwZ graphrag-main]# ls
breaking-changes.md  CONTRIBUTING.md  docs  mkdocs.yaml  scripts  unified-search-app
CHANGELOG.md         cspell.config.yaml  examples_notebooks  pyproject.toml  SECURITY.md  uv.lock
CODE_OF_CONDUCT.md  DEVELOPING.md      graphrag            RAI_TRANSPARENCY.md  SUPPORT.md
CODEOWNERS          dictionary.txt      LICENSE             README.md          tests
[root@iZ8vb5acpt0ebqsuf5mtiwZ graphrag-main]# pip install poetry
WARNING: Disabling truststore since ssl support is missing
Looking in indexes: http://mirrors.cloud.aliyuncs.com/pypi/simple/
Collecting poetry
  Downloading http://mirrors.cloud.aliyuncs.com/pypi/packages/6d/37/578fe593a07daa5e4417a7965d46093a255ebd7fbb797df6959c0f37y-2.1.4-py3-none-any.whl (278 kB)
Collecting build<2.0.0,=>1.2.1 (from poetry)
...
再安装poetry, 也可以先创建Python虚拟环境
```

代码块

```
1 # 源码安装Python的解释器
2 ./configure --prefix=/usr/python-3.12 --with-openssl=/usr --enable-optimizations
3 make -j$(nproc)
4 make altinstall
5
6 # 安装pip
7 python3.12 -m ensurepip --upgrade
8 wget https://bootstrap.pypa.io/get-pip.py
9 sudo python3.12 get-pip.py
10
11 # 创建软连接 (可以不要)
12 ln -s /usr/python-3.12/bin/python3.12 /usr/python-3.12/bin/python
```

```
standard-0.23.0
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager,
ssibly rendering your system unusable. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv. U
the --root-user-action option if you know what you are doing and want to suppress this warning.
[root@iZ8vb5acpt0ebqsuf5mtiwZ graphrag-main]# ls /usr/python-3.12/bin/
2to3-3.12  dul-upload-pack  get-pip.py  keyring  pip  pkginfo  pyproject-build  python3.12-config  trove-classifiers  virtualenv
doesitcache  dulwich  httpx  normalizer  pip3  poetry  python  python3.12
dul-receive-pack  findpython  idle3.12  pbs-install  pip3.12  pydoc3.12  python3.12
[root@iZ8vb5acpt0ebqsuf5mtiwZ graphrag-main]#
```

```
Running scriptlet: unzip-6.0-47.0.1.al8.x86_64
Verifying      : unzip-6.0-47.0.1.al8.x86_64
```

```
Installed:
unzip-6.0-47.0.1.al8.x86_64
```

Complete!

```
[root@iZ8vb5acpt0ebqsuf5mtiwZ download]# yum install unzip
```

Last metadata expiration check: 0:31:34 ago on Fri 15 Aug 2025 06:17:46 PM CST.

Package unzip-6.0-47.0.1.al8.x86_64 is already installed.

Dependencies resolved.

Nothing to do.

Complete!

```
[root@iZ8vb5acpt0ebqsuf5mtiwZ download]# unzip graphrag-main.zip
```

Archive: graphrag-main.zip

469ee8568f2659e82aa5d3939a88e4f5be5290d9

creating: graphrag-main/

inflating: graphrag-main/.gitattributes

creating: graphrag-main/.github/

creating: graphrag-main/.github/ISSUE_TEMPLATE/

inflating: graphrag-main/.github/ISSUE_TEMPLATE/bug_report.yml

extracting: graphrag-main/.github/ISSUE_TEMPLATE/config.yml

inflating: graphrag-main/.github/ISSUE_TEMPLATE/feature_request.yml

inflating: graphrag-main/.github/ISSUE_TEMPLATE/general_issue.yml

解压源代码

代码块

```
1
2 poetry source add aliyun https://mirrors.aliyun.com/pypi/simple/ --
  priority=primary
3
4 poetry source add --priority=supplemental tsinghua
  https://pypi.tuna.tsinghua.edu.cn/simple
```

```
[root@iZ8vb5acpt0ebqsuf5mtiwZ graphrag-main]# poetry source add --priority=supplemental tsinghua https://pypi.tuna.tsinghua.edu.cn/simple
```

Adding source with name **tsinghua**.

```
[root@iZ8vb5acpt0ebqsuf5mtiwZ graphrag-main]# poetry source list
```

The requested command does not exist in the **source** namespace.

Did you mean one of these perhaps?

source add: Add source configuration for project.

source remove: Remove source configured for the project.

source show: Show information about sources configured for the project.

设置国内的poetry镜像

Documentation: <https://python-poetry.org/docs/cli/#source>

```
[root@iZ8vb5acpt0ebqsuf5mtiwZ graphrag-main]# poetry source show
```

name : **tsinghua**

url : <https://pypi.tuna.tsinghua.edu.cn/simple>

priority : supplemental

查看设置

PyPI is implicitly enabled as a primary source. If you wish to disable it, or alter its priority please refer to <https://python-poetry.org/docs/repositories/#package-sources>.

```
[root@iZ8vb5acpt0ebqsuf5mtiwZ graphrag-main]#
```

```
[root@iZ8vb5acpt0ebqsuf5mtiwZ graphrag-main]# poetry install
```

Updating dependencies

Resolving dependencies... (443.7s)

Package operations: 116 installs, 0 updates, 0 removals

- Installing **typing-extensions** (4.14.1)
- Installing **six** (1.17.0)
- Installing **python-dateutil** (2.9.0.post0)
- Installing **pycparser** (2.22)
- Installing **cffi** (1.17.1)
- Installing **idna** (3.10)
- Installing **markupsafe** (3.0.2)
- Installing **asttokens** (2.4.1)
- Installing **executing** (2.2.0)

进入到源代码目录安装

```
[root@iZ8vb5acpt0ebqsuf5mtiwZ graphrag-main]# poetry run graphrag --help
/root/.cache/pypoetry/virtualenvs/graphrag-rI4eBFKX-py3.12/lib/python3.12/site-packages/azure/search/documents/indexes/_generated/models/_models_py3.py:5644: SyntaxWarning: invalid escape sequence '\W'
  pattern: str = "\W+",
/root/.cache/pypoetry/virtualenvs/graphrag-rI4eBFKX-py3.12/lib/python3.12/site-packages/azure/search/documents/indexes/_generated/models/_models_py3.py:5869: SyntaxWarning: invalid escape sequence '\W'
  pattern: str = "\W+",

Usage: graphrag [OPTIONS] COMMAND [ARGS]...

GraphRAG: A graph-based retrieval-augmented generation (RAG) system.
```

验证，是否安装成功

```
Options
--install-completion  Install completion for the current shell.
--show-completion     Show completion for the current shell, to copy it or customize the installation.
--help               Show this message and exit.

Commands
init                 Generate a default configuration file.
index                Build a knowledge graph index.
update              Update an existing knowledge graph index.
prompt-tune         Generate custom graphrag prompts with your own data (i.e. auto templating).
```

```
Options
--install-completion  Install completion for the current shell.
--show-completion     Show completion for the current shell, to copy it or customize
--help               Show this message and exit.
```

```
Commands
init                 Generate a default configuration file.
index                Build a knowledge graph index.
update              Update an existing knowledge graph index.
prompt-tune         Generate custom graphrag prompts with your own data (i.e. auto templating).
query               Query a knowledge graph index.
```

创建自己的工作目录

```
[root@iZ8vb5acpt0ebqsuf5mtiwZ graphrag-main]# mkdir -p /opt/test_rag/input
[root@iZ8vb5acpt0ebqsuf5mtiwZ graphrag-main]#
```

```
update          Update an existing knowledge graph index.
prompt-tune     Generate custom graphrag prompts with your own data (i.e. auto templating).
query           Query a knowledge graph index.
```

初始化工作目录

```
[root@iZ8vb5acpt0ebqsuf5mtiwZ graphrag-main]# poetry run graphrag init --root /opt/test_rag/
2025-08-15 21:10:24.0397 - INFO - graphrag.cli.initialize - Initializing project at /opt/test_rag
[root@iZ8vb5acpt0ebqsuf5mtiwZ graphrag-main]#
```

发送文本到当前Xshell窗口的全部会话

```
[root@iZ8vb5acpt0ebqsuf5mtiwZ ~]# cd /opt/
[root@iZ8vb5acpt0ebqsuf5mtiwZ opt]# ls
test_rag
[root@iZ8vb5acpt0ebqsuf5mtiwZ opt]# cd test_rag/
[root@iZ8vb5acpt0ebqsuf5mtiwZ test_rag]# ls
input prompts settings.yaml
[root@iZ8vb5acpt0ebqsuf5mtiwZ test_rag]# ll
total 16
drwxr-xr-x 2 root root 4096 Aug 15 21:06 input
drwxr-xr-x 2 root root 4096 Aug 15 21:10 prompts
-rw-r--r-- 1 root root 5129 Aug 15 21:10 settings.yaml
[root@iZ8vb5acpt0ebqsuf5mtiwZ test_rag]#
```

配置文件

```
[root@iZ8vb5acpt0ebqsuf5mtiwZ test_rag]# vi settings.yaml
```

```
### This config file contains required core defaults that must be set, along with a handful of comm
### For a full list of available settings, see https://microsoft.github.io/graphrag/config/yaml/
```

```
### LLM settings ###
```

```
## There are a number of settings to tune the threading and token limits for LLM calls - check the
```

```
models:
```

```
  default_chat_model:
```

```
    type: openai_chat # or azure_openai_chat
```

```
    api_base: https://xiaoai.plus/v1
```

```
    # api_version: 2024-05-01-preview
```

```
    auth_type: api_key # or azure managed identity
```

```
    api_key: sk-KQklfwtIYLA0tpExdKLWrqMhhGm9iW44XQkfqo0yWo2mnxhe
```

```
    # audience: "https://cognitiveservices.azure.com/.default"
```

```
    # organization: <organization_id>
```

```
    model: gpt-4.1-mini
```

```
    # deployment_name: <azure_model_deployment_name>
```

```
    # encoding_model: cl100k_base # automatically set by tiktoken if left undefined
```

```
    model_supports_json: true # recommended if this is available for your model.
```

```
    concurrent_requests: 25 # max number of simultaneous LLM requests allowed
```

```
    async_mode: threaded # or asyncio
```

```
    retry_strategy: native
```

```
    max_retries: 10
```

```
[root@iZ8vb5acpt0ebqsuf5mtiwZ input]# ll
```

```
total 44
```

```
-rw-r--r-- 1 root root 10908 Aug 15 21:26 book1.txt
```

```
-rw-r--r-- 1 root root 5731 Aug 15 21:27 book2.txt
```

```
-rw-r--r-- 1 root root 10805 Aug 15 21:28 book3.txt
```

```
-rw-r--r-- 1 root root 10355 Aug 15 21:28 book4.txt
```

```
[root@iZ8vb5acpt0ebqsuf5mtiwZ input]#
```

测试数据，放入input目录

```
2025-08-15 21:38:10.0920 - INFO - graphrag.index.operations.embed_text.strategies.openai - embedding 1/ inputs via 1/ :
  batches. max_batch_size=16, batch_max_tokens=8191
2025-08-15 21:38:20.0133 - INFO - graphrag.logger.progress - generate embeddings progress: 1/3
2025-08-15 21:38:20.0236 - INFO - graphrag.logger.progress - generate embeddings progress: 2/3
2025-08-15 21:38:20.0320 - INFO - graphrag.logger.progress - generate embeddings progress: 3/3
[2025-08-15T13:38:20Z WARN lance::dataset::write::insert] No existing dataset at /opt/test_rag/output/lancedb/default
lance, it will be created
2025-08-15 21:38:20.0327 - INFO - graphrag.index.workflows.generate_text_embeddings - Workflow completed: generate_text_embeddings
2025-08-15 21:38:20.0327 - INFO - graphrag.api.index - Workflow generate_text_embeddings completed successfully
2025-08-15 21:38:20.0372 - INFO - graphrag.index.run.run_pipeline - Indexing pipeline complete
2025-08-15 21:38:20.0374 - INFO - graphrag.cli.index - All workflows completed successfully.
[root@iZ8vb5acpt0ebqsuf5mtiwZ graphrag-main]# poetry run graphrag index --root /opt/test_rag/
```

构建索引

代码块

- 1 poetry run graphrag query --root /opt/test_rag --method global --query "马云是谁?"
- 2
- 3 poetry run graphrag query --root /opt/test_rag --method global --query "马云和淘宝网是什么关系?"

```
[root@iZ8vb5acpt0ebqsuf5mtiwZ graphrag-main]# poetry run graphrag query --root /opt/test_rag --method global --query "马云是谁?"
2025-08-16 00:29:04.0638 - INFO - graphrag.storage.file_pipeline_storage - Creating file storage at /opt/test_rag/output
2025-08-16 00:29:04.0639 - INFO - graphrag.utils.storage - reading table from storage: entities.parquet
2025-08-16 00:29:04.0647 - INFO - graphrag.utils.storage - reading table from storage: communities.parquet
2025-08-16 00:29:04.0651 - INFO - graphrag.utils.storage - reading table from storage: community_reports.parquet
2025-08-16 00:29:29.0187 - INFO - graphrag.cli.query - Global Search Response:
# 马云简介
```

马云（Jack Ma）是中国著名的企业家和阿里巴巴集团的联合创始人之一。他不仅是阿里巴巴集团的重要领导者，还在中国乃至全球的电子商务和数字经济领域发挥了关键作用。马云持有阿里巴巴集团约8.9%的股份，曾担任集团董事长和首席执行官，推动公司不断创新和扩展业务，巩固了其在互联网行业的领先地位。他的领导力和远见使阿里巴巴成为中国最大的电子商务集团之一，极大地影响了中国的数字经济发展 [Data: Reports (1, 22, 24, 7, +more)]。

早期经历与创业历程

马云于1964年出生于浙江省，毕业于杭州师范学院，获得英语学士学位，曾任教于杭州电子工业学院。他的早期创业经历包括1994年创办海博翻译社和与妻子及合作伙伴共同创办中国黄页网站，这些经历为其后续互联网创业奠定了坚实基础。1999年，马云与18位合伙人共同创立了阿里巴巴网络有限公司，随后推出了淘宝网和支付宝，极大地推动了中国电子商务的发展，体现了他的创新精神和商业眼光 [Data: Reports (22, 24, 7, +more)]。

领导风格与企业治理

在阿里巴巴集团的发展过程中，马云不仅担任董事长和首席执行官，还推动公司治理结构转型为合伙人制度，确保公司管理的持续稳定和发展。这反映了

```
[root@iZ8vb5acpt0ebqsuf5mtiwZ graphrag-main]# poetry run graphrag query --root /opt/test_rag --method global --query "马云和淘宝网是什么关系?"
2025-08-16 00:32:13.0571 - INFO - graphrag.storage.file_pipeline_storage - Creating file storage at /opt/test_rag/output
2025-08-16 00:32:13.0572 - INFO - graphrag.utils.storage - reading table from storage: entities.parquet
2025-08-16 00:32:13.0579 - INFO - graphrag.utils.storage - reading table from storage: communities.parquet
2025-08-16 00:32:13.0584 - INFO - graphrag.utils.storage - reading table from storage: community_reports.parquet
2025-08-16 00:32:35.0123 - INFO - graphrag.cli.query - Global Search Response:
```

马云与淘宝网的关系解析

马云是淘宝网的创始人之一，同时也是阿里巴巴集团的创始人和前董事局主席。淘宝网作为阿里巴巴集团旗下的重要电子商务平台，其发展和成功与马云的领导和战略规划密不可分。

马云的角色与贡献

马云于1999年创立了阿里巴巴集团，随后在2003年领导推出了淘宝网，旨在打造一个面向个人卖家和小型商家的C2C电子商务平台。作为阿里巴巴集团的核心人物，马云不仅推动了淘宝网的诞生，还通过其远见和创新精神，推动淘宝网快速成长为中国最大的在线购物平台之一。

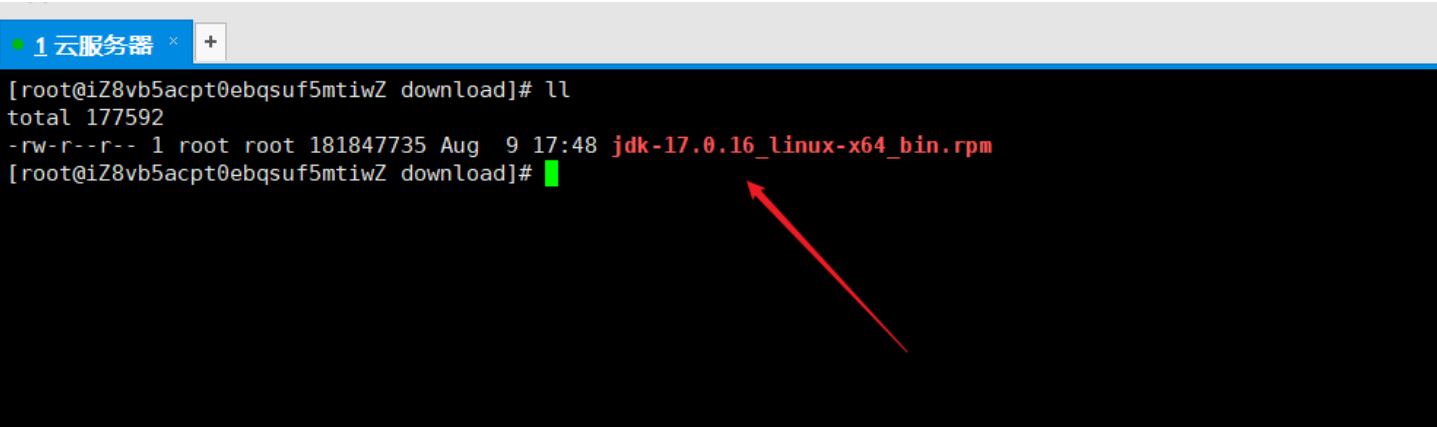
他的创业愿景和领导力为淘宝网的发展奠定了坚实基础，淘宝网成为阿里巴巴集团电商生态系统的核心组成部分。即使在2013年卸任阿里巴巴集团CEO后，马云对淘宝网的发展方向和战略布局仍有深远影响 [Data: Reports (1, 2, 12, 22, +more)]。

淘宝网在阿里巴巴生态系统中的地位

淘宝网是阿里巴巴集团于2003年创立的旗舰电商平台，主要提供消费者对消费者（C2C）的服务，允许个人和企业在线开店销售商品。淘宝网依托阿里巴巴集团的资源、技术和资金支持，成为中国网购市场的主导者，市场份额约占70%-80%。

第三章、Neo4j的安装

Neo4j 5.x 在 Java 17 上运行，从 Neo4j 5.14 开始，它也支持 Java 21。




```
[root@iz8vb5acpt0ebqsuf5mtiwZ download]# ll
total 177592
-rw-r--r-- 1 root root 181847735 Aug  9 17:48 jdk-17.0.16_linux-x64_bin.rpm
[root@iz8vb5acpt0ebqsuf5mtiwZ download]# rpm -ivh jdk-17.0.16_linux-x64_bin.rpm
warning: jdk-17.0.16_linux-x64_bin.rpm: Header V3 RSA/SHA256 Signature, key ID 8d8c756f: NOKEY
Verifying...                               ##### [100%]
Preparing...                               ##### [100%]
Updating / installing...
 1:jdk-17-2000:17.0.16-12                   ##### [100%]
[root@iz8vb5acpt0ebqsuf5mtiwZ download]#
```

安装命令

```
# .bashrc

# User specific aliases and functions

alias rm='rm -i'
alias cp='cp -i'
alias mv='mv -i'

# Source global definitions
if [ -f /etc/bashrc ]; then
    . /etc/bashrc
fi

export JAVA_HOME=/usr/java/jdk-17
export PATH=$PATH:$JAVA_HOME/bin
```

配置环境变量

让环境变量配置生效

1、下载Neo4j的图数据库

<https://neo4j.ac.cn/deployment-center/>

产品

用例

开发者

生成式 AI

学习

定价

联系我们

免费开始

图数据库 自主管理

企业级可用性和安全性，具有横向扩展和纵向扩展选项。可在您的私有云或公有云基础设施中运行。

企业版下载包含 APOC 过程、Bloom 和图数据科学库。可能需要额外的许可证密钥。

旧版企业版可在登录后通过支持门户获取。

企业版

社区版

Neo4j 5.26.8 2025 年 6 月 9 日发布

Red Hat Linux 包 Neo4j 5.26.8 (rpm)

Debian / Ubuntu 包 Neo4j 5.26.8 (deb)

Windows 可执行文件 Neo4j 5.26.8 (zip)

Linux / Mac 可执行文件 Neo4j 5.26.8 (tar)

Neo4j (Debian / Ubuntu) Apt 仓库

访问

Neo4j 仓库

通过使用适用于 RHEL 和基于 Ubuntu/Debian 系统的官方 yum 和 apt 仓库，确保满足操作系统依赖项并简化 Neo4j 的安装和更新。

Cypher Shell

Cypher Shell CLI 用于对 Neo4j 实例运行查询和执行管理任务。

默认情况下，shell 是交互式的，但您也可以通过在命令行直接传递 Cypher 或通过管道传输包含 Cypher 语句的文件（Windows 上需要 PowerShell）来用于脚本编写。它通过 Bolt 协议进行通信。

Neo4j Cypher Shell 5.26.8

Linux cypher-shell-5.26.8-1.noarch.rpm

下载

[SHA-256](#)

2、安装Neo4j图数据库

代码块

```
1 rpm --install cypher-shell-5.26.8-1.noarch.rpm neo4j-5.26.8-1.noarch.rpm
```

```
error: Failed dependencies:
    cypher-shell >= 5.0 is needed by neo4j-5.26.8-1.noarch
    cypher-shell < 6.0 is needed by neo4j-5.26.8-1.noarch
[root@iZ8vb5acpt0ebqsuf5mtiwZ download]# rm -rf cypher-shell-5.26.8-1.noarch.rpm
[root@iZ8vb5acpt0ebqsuf5mtiwZ download]# rpm --install cypher-shell-5.26.8-1.noarch.rpm neo4j-5.26.8-1.noarch.rpm
[root@iZ8vb5acpt0ebqsuf5mtiwZ download]# cd /usr/local/
aegis/ bin/ cloudmonitor/ etc/ games/ include/ lib/ lib64/ libexec/ s
[root@iZ8vb5acpt0ebqsuf5mtiwZ download]# rpm -qa neo4j-5.26.8-1
[root@iZ8vb5acpt0ebqsuf5mtiwZ download]# rpm -qk neo4j-5.26.8-1
rpm: -qk: unknown option
[root@iZ8vb5acpt0ebqsuf5mtiwZ download]# rpm -ql neo4j-5.26.8-1
/etc/bash_completion.d/neo4j-admin_completion
/etc/bash_completion.d/neo4j_completion
/etc/default/neo4j
/etc/init.d/neo4j
/etc/neo4j
/etc/neo4j/neo4j-admin.conf
/etc/neo4j/neo4j.conf
/etc/neo4j/server-logs.xml
/etc/neo4j/user-logs.xml
/lib/systemd/system/neo4j.service
/usr/bin/neo4j
/usr/bin/neo4j-admin
/usr/share/doc/neo4j/LICENSE.txt
/usr/share/doc/neo4j/LICENSES.txt
```

查看安装后的路径

安装

配置存储在 `/etc/neo4j/neo4j.conf` 中。在首次启动数据库之前，建议使用 `neo4j-admin` 的 `set-initial-password` 命令定义本机用户 `neo4j` 的密码。

如果未使用此方法显式设置密码，则它将设置为默认密码 `neo4j`。在这种情况下，您将在首次登录时被提示更改默认密码。

代码块

```
1 neo4j-admin dbms set-initial-password <password> [--require-password-change]
```

```
/var/log/neo4j
[root@iZ8vb5acpt0ebqsuf5mtiwZ download]# neo4j-admin dbms set-initial-password lqaz3edc
Changed password for user 'neo4j'. IMPORTANT: this change will only take effect if performed before the database is started for the first time.
[root@iZ8vb5acpt0ebqsuf5mtiwZ download]# ^C
[root@iZ8vb5acpt0ebqsuf5mtiwZ download]#
```

系统服务使用 `systemctl` 命令进行控制。它接受许多命令

代码块

```
1 systemctl {start|stop|restart|status|edit} neo4j
```

```
[root@iZ8vb5acpt0ebqsuf5mtiwZ download]# systemctl start neo4j
[root@iZ8vb5acpt0ebqsuf5mtiwZ download]# netstat -ntpl
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:111             0.0.0.0:*               LISTEN      1/systemd
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      1989/sshd
tcp6       0      0 :::111                  :::*                    LISTEN      1/systemd
tcp6       0      0 127.0.0.1:7474          :::*                    LISTEN      15007/java
tcp6       0      0 :::22                   :::*                    LISTEN      1989/sshd
tcp6       0      0 127.0.0.1:7687          :::*                    LISTEN      15007/java
[root@iZ8vb5acpt0ebqsuf5mtiwZ download]#
```



```
[root@iZ8vb5acpt0ebqsuf5mtiwZ download]# systemctl status neo4j
● neo4j.service - Neo4j Graph Database
   Loaded: loaded (/usr/lib/systemd/system/neo4j.service; disabled; vendor preset: disabled)
   Active: active (running) since Sat 2025-08-09 19:23:12 CST; 1min 43s ago
     Main PID: 14976 (java)
       Tasks: 78 (limit: 95926)
      Memory: 582.3M
        CGroup: /system.slice/neo4j.service
                └─14976 /usr/bin/java -Xmx128m -classpath /usr/share/neo4j/lib/*:/usr/share/neo4j/etc:/usr/share/neo4j/repo/* -Dapp.name=
                  15007 /usr/lib/jvm/jdk-17.0.16-oracle-x64/bin/java -cp /var/lib/neo4j/plugins/*:/etc/neo4j/*:/usr/share/neo4j/lib/* -XX

Aug 09 19:23:16 iZ8vb5acpt0ebqsuf5mtiwZ neo4j[15007]: 2025-08-09 11:23:16.739+0000 INFO This instance is ServerId{79a2ab97} (79a2ab97)
Aug 09 19:23:18 iZ8vb5acpt0ebqsuf5mtiwZ neo4j[15007]: 2025-08-09 11:23:18.008+0000 INFO ===== Neo4j 5.26.8 =====
Aug 09 19:23:20 iZ8vb5acpt0ebqsuf5mtiwZ neo4j[15007]: 2025-08-09 11:23:20.279+0000 INFO Anonymous Usage Data is being sent to Neo4j
Aug 09 19:23:20 iZ8vb5acpt0ebqsuf5mtiwZ neo4j[15007]: 2025-08-09 11:23:20.325+0000 INFO Bolt enabled on localhost:7687.
Aug 09 19:23:21 iZ8vb5acpt0ebqsuf5mtiwZ neo4j[15007]: 2025-08-09 11:23:21.099+0000 INFO HTTP enabled on localhost:7474.
Aug 09 19:23:21 iZ8vb5acpt0ebqsuf5mtiwZ neo4j[15007]: 2025-08-09 11:23:21.100+0000 INFO Remote interface available at http://localhost:7474
Aug 09 19:23:21 iZ8vb5acpt0ebqsuf5mtiwZ neo4j[15007]: 2025-08-09 11:23:21.103+0000 INFO id: 592505C0F85909673FA89AFF6D42FA32E1C1132
Aug 09 19:23:21 iZ8vb5acpt0ebqsuf5mtiwZ neo4j[15007]: 2025-08-09 11:23:21.103+0000 INFO name: system
Aug 09 19:23:21 iZ8vb5acpt0ebqsuf5mtiwZ neo4j[15007]: 2025-08-09 11:23:21.104+0000 INFO creationDate: 2025-08-09T11:23:19.297Z
Aug 09 19:23:21 iZ8vb5acpt0ebqsuf5mtiwZ neo4j[15007]: 2025-08-09 11:23:21.104+0000 INFO Started.
lines 1-20/20 (END)
```

要使 Neo4j 在系统启动时自动启动，请运行以下命令

代码块

1 systemctl enable neo4j

← 39.100.64.14:7474/browser/

Database access not available. Please use :server connect to establish connection. There's a graph waiting for you.

\$:server connect

Connect to Neo4j

Database access might require an authenticated connection

Connect URL

neo4j:// 39.100.64.14:7687

Database - leave empty for default

Authentication type

Username / Password

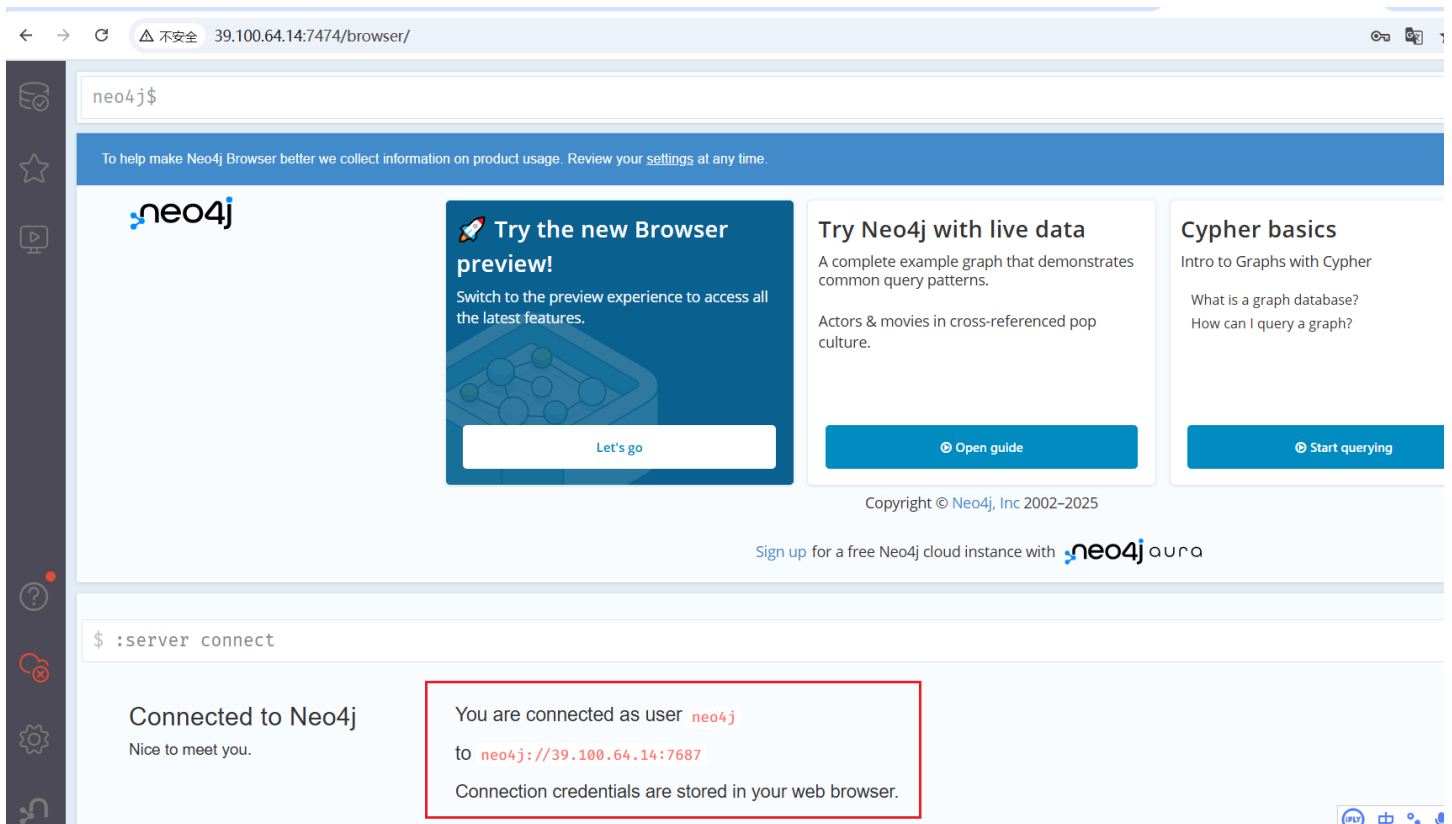
Username

Password

Connect

默认neo4j

之前修改的密码



Neo4j 浏览器是一个面向开发人员的工具，它允许您执行 Cypher 查询并可视化结果。它是 Neo4j 企业版和社区版的默认开发人员界面。

Neo4j 浏览器是开发人员与图交互的工具，主要侧重于：

- 使用 Cypher 编写和运行图查询。
- 任何查询结果均可导出为表格形式。
- 用于对包含节点和关系的查询结果进行图可视化。

官网地址：<https://neo4j.ac.cn/docs/browser-manual/current/operations/dbms-connection/>

3、关于关于 Cypher Shell CLI

Cypher Shell 是一个命令行工具，用于针对 Neo4j 实例运行查询和执行管理任务。默认情况下，该 shell 是交互式的，但您也可以通过直接在命令行上传递 Cypher 或通过管道传输包含 Cypher 语句的文件（在 Windows 上需要 PowerShell）来将其用于脚本编写。它通过 Bolt 协议进行通信。

如果作为产品的一部分安装，Cypher Shell 位于 `bin` 目录中。

运行 Cypher Shell 的语法是

```
cypher-shell [-h] [-a ADDRESS] [-u USERNAME] [--impersonate IMPERSONATE] [-p PASSWORD]
              [--encryption {true,false,default}] [-d DATABASE] [--access-mode {read,write}]
              [--enable-autocompletions] [--format {auto,verbose,plain}] [-P PARAM]
              [--non-interactive] [--sample-rows SAMPLE-ROWS] [--wrap {true,false}] [-v]
              [--driver-version] [-f FILE] [--change-password] [--log [LOG-FILE]]
```

[--history HISTORY-BEHAVIOUR] [--notifications] [--idle-timeout IDLE-TIMEOUT]

[--error-format {gql,legacy,stacktrace}] [--fail-fast | --fail-at-end] [cypher]

连接参数

选项	描述
-a ADDRESS, --address ADDRESS, --uri ADDRESS	要连接的地址和端口。默认为 neo4j://localhost:7687。也可以使用环境变量 NEO4J_ADDRESS 或 NEO4J_URI 指定。
-u USERNAME, --username USERNAME	连接时使用的用户名。也可以使用环境变量 NEO4J_USERNAME 指定。
--impersonate IMPERSONATE	要模拟的用户。
-p PASSWORD, --password PASSWORD	连接密码。也可以使用环境变量 NEO4J_PASSWORD 指定。
--encryption {true,false,default}	是否应加密与 Neo4j 的连接。这必须与 Neo4j 的配置一致。如果选择 'default'，加密设置将从指定的地址推断。例如，'neo4j+ssc' 协议使用加密。
-d DATABASE, --database DATABASE	要连接的数据库。也可以使用环境变量 NEO4J_DATABASE 指定。
--access-mode {read,write}	访问模式。默认为 WRITE。

```
Aug 09 19:31:10 iZ8vb5acpt0ebqsuf5mtiwZ neo4j[15226]: 2025-08-09 11:31:10.189+0000 INFO Started.
[root@iZ8vb5acpt0ebqsuf5mtiwZ download]# cypher-shell -u neo4j -p lqaz3edc
Connected to Neo4j using Bolt protocol version 5.8 at neo4j://localhost:7687 as user neo4j.
Type :help for a list of available commands or :exit to exit the shell.
Note that Cypher queries must end with a semicolon.
neo4j@neo4j> █
```

默认数据库：Neo4j社区版安装后会创建两个数据库：`system`（系统数据库）和 `neo4j`（默认用户数据库）。社区版**不支持**通过Cypher命令（如 `CREATE DATABASE`）直接创建新数据库，这是企业版的功能。社区版一次只能激活一个用户数据库（默认为 `neo4j`），无法同时运行多个用户数据库

```
[root@iZ8vb5acpt0ebqsuf5mtiwZ download]# cypher-shell -u neo4j -p lqaz3edc
Connected to Neo4j using Bolt protocol version 5.8 at neo4j://localhost:7687 as user neo4j.
Type :help for a list of available commands or :exit to exit the shell.
Note that Cypher queries must end with a semicolon.
neo4j@neo4j> show databases;
+-----+
| name      | type      | aliases | access      | address      | role      | writer | requestedStatus | currentStat
sage | default | home   | constituents |              |           |        |                  | 
+-----+
| "neo4j"   | "standard" | []      | "read-write" | "localhost:7687" | "primary" | TRUE   | "online"        | "online"
| TRUE     | TRUE     | []      |              |              |           |        |                  | 
| "system" | "system"   | []      | "read-write" | "localhost:7687" | "primary" | TRUE   | "online"        | "online"
| FALSE    | FALSE    | []      |              |              |           |        |                  | 
+-----+
2 rows
ready to start consuming query after 12 ms, results consumed after another 2 ms
neo4j@neo4j> create database customers;
Unsupported administration command: create database customers
neo4j@neo4j> █
```

:exit 或 :quit

直接输入这些命令可立即退出 cypher-shell 会话

代码块

```
1 MATCH (p:Person {id:'马云'})
2 RETURN p;
3 # 查询所有的Person标签实体(节点), 并且该节点的id值为: 马云
4
5 # 还可以查询图中多个节点。此查询匹配所有带有 Person 标签的节点, 并将结果限制为仅包含五行。
6 MATCH (people:Person)
7 RETURN people
8 LIMIT 5
9
10 # 使用 DETACH DELETE 删除所有节点及其关系
11 MATCH (n)
12 DETACH DELETE n;
13
```

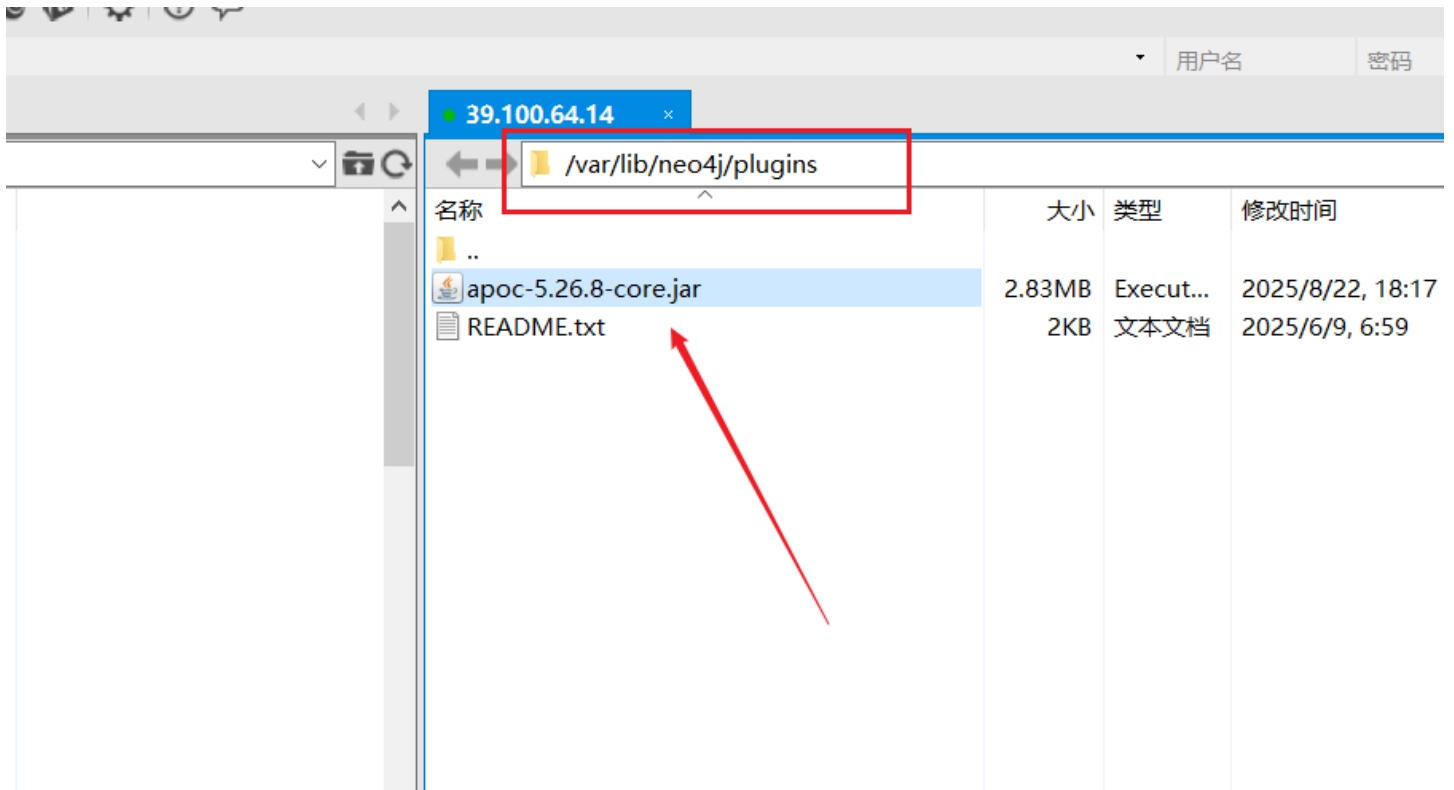
4、安装插件APOC插件

因为 LangChain 使用的 `Neo4jGraph` 类依赖于 **APOC (Awesome Procedures On Cypher)** 插件中的一些增强功能, 比如 `apoc.meta.data()`。若 Neo4j 中未安装或启用这些功能, 就会报错。

为什么要安装 APOC 插件?

- **APOC 插件提供丰富的扩展功能**, 包括元数据查询、函数、图投影、条件执行等, 对构建 GraphRAG 的工作流非常关键。
- LangChain 在连接图数据库时会依赖 APOC 提供的增强查询能力, 例如快速获取模式信息 (schema), 否则不能执行图查询

一、上传插件的jar包到plugins目录



二、修改Neo4j的配置文件：vi /etc/neo4j/neo4j.conf

```
# A comma separated list of procedures and user defined functions that are allowed
# full access to the database through unsupported/insecure internal APIs.
dbms.security.procedures.unrestricted=apoc.*

# A comma separated list of procedures to be loaded by default.
# Leaving this unconfigured will load all procedures found.
dbms.security.procedures.allowlist=apoc.*

#*****
```

修改配置文件

一定要重启Neo4j的数据库服务器：systemctl restart neo4j

三、然后通过执行Cypher的命令来验证安装插件是否成功：

```
438 rows
ready to start consuming query after 106 ms, results consumed after another 526 ms
neo4j@neo4j> RETURN apoc.version();
+-----+
| apoc.version() |
+-----+
| "5.26.8"       |
+-----+

1 row
ready to start consuming query after 52 ms, results consumed after another 1 ms
neo4j@neo4j> :quit;
```

返回APOC的版本 命令

第四章、Neo4j+Langchain开发GraphRAG

```
1 pip install neo4j langchain-neo4j langchain-experimental
```

LangChain-Neo4j 是 LangChain 与 Neo4j 图数据库的深度集成工具，旨在通过大语言模型（LLM）增强图数据的查询、生成和分析能力。

1. Neo4jGraph 操作封装

- 提供构造函数参数如 `url`, `username`, `password`, `database`, `timeout`, `sanitize`, `refresh_schema`, `enhanced_schema`, `driver_config` 等选项，支持更灵活配置。
- 方法包括 `query()`, `add_graph_documents()`, `refresh_schema()`, `close()`，可用于执行 Cypher 查询、构建图结构、刷新 Schema 信息等。

2. Neo4jVector 向量索引管理

- 支持 `from_documents()`, `from_texts()`, `from_existing_graph()` 等多种方式构建向量索引。
- 检索支持 `similarity_search`, `similarity_search_with_score`, `max_marginal_relevance_search` 等，同时支持元数据过滤、Hybrid Search（语义+关键词）。
- 内部包含 `verify_version()` 方法，用于检测 Neo4j 是否支持向量索引（**必须版本 $\geq$ 5.11.0**）。

3. GraphCypherQAChain 与 查询纠正

- 支持将用户自然语言问题转化为 Cypher 查询、执行后将结果传回 LLM 生成自然语言回答，形成问答链（QA）。
- 结合 `CypherQueryCorrector` 帮助纠正查询方向与结构，提升生成 Cypher 的准确性。

4. GraphDocument 与知识图构建

- 使用 `GraphDocument`, `Node`, `Relationship` 等类，可自定义构建图结构，并通过 `add_graph_documents()` 插入 Neo4j，同时支持关联来源（如 `include_source`）。

5. Memory: Neo4jChatMessageHistory 保存对话历史

- 支持以节点与关系形式，在 Neo4j 中保存、查询对话内容，用于上下文回溯、分析等场景

功能模块	描述
Neo4jGraph	图数据库驱动封装、执行查询、Schema 管理
Neo4jVector	向量索引创建与检索，支持语义搜索、Hybrid 搜索、过滤
GraphCypherQAChain	自然语言 → Cypher → 执行 → 自然语言回答
GraphDocument 等类	构建知识图结构，支持文档关联插入
CypherQueryCorrector	修正生成的 Cypher 查询结构与方向
Neo4jChatMessageHistory	在 Neo4j 中存储会话历史

1、连接Neo4j图数据库

代码块

```
1 graph = Neo4jGraph(  
2     url="bolt://39.100.64.14/:7687",  
3     username="neo4j",  
4     password="1qaz3edc",  
5     database="neo4j", # 默认数据库, 无需更改  
6     # enhanced_schema=True,  
7 )
```

enhanced_schema 参数详解

- enhanced_schema 是 Neo4jGraph 的一个布尔类型参数，默认为 False ；
- 若将其设置为 True ，LangChain 将在初始化或刷新 schema 时，主动扫描数据库中的属性值，并生成更丰富的 schema 信息，包括：
 - 各属性的示例值（string 属性）；
 - 若属性值类型为数字或日期，给出 最小值（Min）和最大值（Max）；
 - 若某属性值的可选种类小于约 10 个，会列出所有可能值，反之只保留一个示例值。

这种增强型 schema 能显著帮助 LLM 更准确地生成 Cypher 查询，因为它能利用具体数据样本进行构造。

2、建立唯一约束(幂等)

这一约束确保所有带 Entity 标签的节点，其 id 属性在图中保持唯一性。写入与某已存在节点同 id 的新节点时，会因冲突抛出错误，不会自动覆盖旧节点，也不会无声失败。如果用的是 MERGE 操作，将在遇到已存在节点时匹配该节点，进而允许更新该节点的关系或属性（通常结合 ON MATCH SET 使用）。在这种情况下，并不会创建新节点。


```

1  # 为基础标签 __Entity__ 的 id 建唯一约束, 便于全局合并 (注意反引号)
2  graph.query("""
3  CREATE CONSTRAINT entity_id IF NOT EXISTS
4  FOR (e: `__Entity__`) REQUIRE e.id IS UNIQUE
5  """)

```

3、保证ID的唯一性和稳定性

代码块

```

1  def stable_doc_id(meta: dict) -> str:
2      """
3      选用 WikipediaLoader 的 metadata['source'] (URL) 作为稳定标识;
4      若没有, 则回退到 (title + summary) 的 hash。
5      """
6      base = meta.get("source") or (meta.get("title", "") + "|" +
7      meta.get("summary", ""))
8      return hashlib.md5(base.encode("utf-8")).hexdigest()
9
10 def attach_doc_ids(docs: List[Document]) -> List[Document]:
11     for d in docs:
12         d.metadata = dict(d.metadata) # copy, 避免 in-place 副作用
13         d.metadata["id"] = stable_doc_id(d.metadata)
14         # 也统一一下元数据, 便于后续查询
15         d.metadata.setdefault("source_type", "wikipedia")
16         d.metadata.setdefault("title", d.metadata.get("title", ""))
17         d.metadata.setdefault("summary", d.metadata.get("summary", ""))
18     return docs
19
20 def normalize_id(s: str) -> str:
21     """
22     统一节点 ID: 全角半角、空白、大小写等做轻度归一;
23     中文不做 lower() 强制, 但保留 NFKC 规整和空白规整。
24     """
25     if not isinstance(s, str):
26         return s
27     # 兼容性组合形式 (Normalization Form KC), 分解字符并替换兼容字符到其规范形式
28     # "Hello World!" ---> "Hello World!"
29     s = unicodedata.normalize("NFKC", s)
30     # 将所有连续的空白字符替换为单个空格 例如 "Hello \nWorld" -> "Hello World"
31     s = re.sub(r"\s+", " ", s).strip()
32     return s

```

4、LLM图结构提取

注意：如果需要用大语言模型来提取图结构，必须支持： `with_structured_output` 。

当输入的文本数据中包含超出 `allowed_nodes` 列表的实体类型时，这些实体不会被提取到最终的知识图谱中。

这种设计确保了不同轮次抽取的结果在节点类型上保持一致，避免噪声干扰

代码块

```
1  transformer = LLMGraphTransformer(  
2      llm=llm,  
3      allowed_nodes=allowed_nodes,  
4      allowed_relationships=allowed_relationships,  
5      node_properties=node_properties,  
6  )  
7  
8  async def extract_graph_documents(docs: List[Document]):  
9      # 同步接口在部分版本可用；为保险使用异步  
10     gdocs = await transformer.convert_to_graph_documents(docs)  
11     # 统一节点 ID (避免“阿里 巴巴” vs “阿里巴巴集团”等微小差异导致拆点)  
12     for gd in gdocs:  
13         for n in gd.nodes:  
14             n.id = normalize_id(n.id)  
15         for r in gd.relationships:  
16             r.source.id = normalize_id(r.source.id)  
17             r.target.id = normalize_id(r.target.id)  
18     return gdocs
```

参数总结表

参数名	类型	默认值	作用
llm	LLM实例	必填	指定用于抽取的模型
allowed_nodes	List[str]	None	约束节点类型
allowed_relationships	List[str/tuple]	None	定义合法关系
node_properties	bool/List[str]	FALSE	控制节点属性抽取
relationship_properties	bool/List[str]	FALSE	控制关系属性抽取
ignore_tools_usage	bool	FALSE	是否绕过语言模型的“结构化输出 binding)” 机制。
additional_instructions	str	None	全局抽取规则。把一些额外的自然语言提示输入 LLM 行为（例如“所有人名都姓张”等），适合小范围调整而不替换原有规则。
strict_mode	bool	TRUE	是否采用严格模式

prompt: ChatPromptTemplate | None

- 自定义给 LLM 的完整提示模板（LangChain 的 `ChatPromptTemplate`）。
- 用途：你可以替换或扩展默认 prompt（例如给示例、规则、领域词典），对输出格式、命名规范、属性命名风格等精细控制。若不提供，Transformer 会使用内置默认 prompt。

代码块

```
1  from langchain_experimental.graph_transformers.llm import LLMGraphTransformer
2
3  transformer = LLMGraphTransformer(
4      llm=llm,
5      allowed_nodes=["Person", "Company"],
6      allowed_relationships=[("Person", "WORKS_AT", "Company")],
7      strict_mode=True,
8      node_properties=["name", "title", "email"],
9      relationship_properties=False,
10 )
11 graph_docs = transformer.convert_to_graph_documents(documents)
12 # 解释: 只允许 Person 与 Company 两类节点、且只允许 Person-WORKS_AT->Company 这种关
    系; 只抽 name/title/email 属性。
13
14
15 transformer = LLMGraphTransformer(
16     llm=llm,
17     node_properties=True,
18     relationship_properties=True,
19 )
20 # 解释: 允许任意节点、抽所有节点属性 (更开放)
```

5、幂等增量写入写入Neo4j

graph.add_graph_documents(graph_documents)

该方法用于批量将已抽取的 `GraphDocument`（包含节点和关系结构）导入到 Neo4j 图数据库中，构建实际可查询的图结构。

baseEntityLabel=True 参数

- 当设置为 `True` 时，所有新创建的节点会额外获得一个标签： `Entity`。
- 该标签可以用于创建统一索引，对查询和插入性能有显著优化，尤其在节点类型多样或标签未知的场景中非常实用。
- 实现机制：在导入节点时，首先会 `MERGE` 一个拥有 `Entity` 标签、`id` 属性的节点作为统一入口，然后再通过 APOC 方法添加实际类型标签（`source.type`）。这种组合方式避免了大量标签碎片化问题，并保持高效导入。

`include_source=True` 参数（可选未启用）

- 若启用，该方法会引入每个源 Document（即原始文本块）作为一个独立节点，并通过 `MENTIONS` 关系将其与被抽取的实体节点连接。
- 优点包括：
 - 让你能追踪每个实体来源自哪个文档，有利于源追溯、结果可解释性，以及实现“文档级 RAG”检索策略。

代码块

```
1  def upsert_to_neo4j(graph_documents):
2      """
3      include_source=True: 会把来源 Document 导入为 (:Document {id}),
4      并用 (:Document)-[:MENTIONS]->(实体) 连接;
5      baseEntityLabel=True: 给实体加二级标签 __Entity__, 结合唯一约束, 提升合并与查询性
        能。
6      """
7      graph.add_graph_documents(
8          graph_documents,
9          baseEntityLabel=True,    # 添加 __Entity__ 次级标签, 用于索引优化
10         # include_source=True    # 包含来源 Document 节点, 并连接关系
11     )
```

6、使用LangChain-Neo4j查询图数据库

在 LangChain 0.5.0 中，`GraphCypherQAChain.from_llm()` 是构建图数据库问答链的核心方法。它将语言模型（LLM）与图数据库（如 Neo4j）结合，实现自然语言查询转化为 Cypher 查询的功能。

代码块

```
1  chain = GraphCypherQAChain.from_llm(
2      llm=llm,
3      graph=graph,
4      allow_dangerous_requests=True,    # 显式启用危险操作
5      cypher_prompt=cypher_prompt,
6      qa_prompt=qa_prompt,
7      validate_cypher=True,
8      return_intermediate_steps=True,
9      verbose=True,
10 )
11
12 resp = chain.invoke({"query": "马云是谁?"})
```

一、核心参数解析

1. 基础配置参数

参数名	类型	默认值	作用描述
llm	LLM实例	必填	用于生成Cypher查询和自然语言回答的模型（若未单独指定cypher_llm和qa_llm）
graph	Neo4jGraph实例	必填	连接的Neo4j图数据库对象
verbose	bool	FALSE	是否打印详细执行日志（如生成的Cypher语句和查询结果）

2. Cypher查询控制

参数名	类型	默认值	作用描述
cypher_prompt	PromptTemplate	None	自定义Cypher生成提示模板（覆盖默认模板）
validate_cypher	bool	FALSE	是否自动验证和修正生成的Cypher语句（减少无效查询）
allow_dangerous_requests	bool	FALSE	允许执行潜在危险的Cypher操作（如删除数据），需显式启用

3. 问答生成控制

参数名	类型	默认值	作用描述
qa_prompt	PromptTemplate	None	自定义问答生成提示模板（支持逻辑，如空结果处理）
return_intermediate_steps	bool	FALSE	是否返回中间步骤（如生成的Cypher语句和查询结果）

4. 高级配置

参数名	类型	默认值	作用描述
cypher_llm	LLM实例	None	单独指定生成Cypher查询的模型（与qa_llm解耦）
qa_llm	LLM实例	None	单独指定生成自然语言回答的模型（与cypher_llm解耦）
exclude_types	List[str]	None	忽略特定节点或关系类型（如["Movie"]
top_k	int	None	限制查询返回的结果数量（优化性能）

注意：

多模型分工（cypher_llm + qa_llm）；

use_function_response (bool): 默认为 False。

- True : 将数据库上下文封装为工具调用响应，适用于与 Agent 等系统的集成。
- False : 直接返回查询结果，适用于传统的问答场景。

使用场景

- 与 Agent 集成: 当需要将数据库查询作为工具调用的一部分时，启用 use_function_response。
- 传统问答系统: 如果仅需要直接的查询结果，保持 use_function_response=False。

代码块

```
1 chain = GraphCypherQAChain.from_llm(  
2     llm=llm,  
3     graph=graph,  
4     cypher_prompt=cypher_prompt,  
5     verbose=True,  
6     use_function_response=True,  
7     allow_dangerous_requests=True,  
8 )  
9  
10 # resp = chain.invoke({"query": "马云是谁?详细介绍一下"})  
11 resp = chain.invoke({"query": "马云和马化腾的关系?"})
```

第五章：安装和部署Dots.OCR模型

1、安装Python虚拟环境和第三方库

代码块

```
1  conda config --add envs_dirs /root/autodl-tmp/envs
2
3  conda config --show envs_dirs
4
5
6  conda create --name ocr_env python=3.12 --no-deps
7
8  conda activate ocr_env
9
10 conda install pip
11
12 conda list
13
14 python -m pip install vllm==0.9.1
15
16 python -m pip install -r requirements.txt
17
18 # requirements.txt内容如下:
19 PyMuPDF
20 openai
21 qwen_vl_utils
22 transformers==4.51.3
23 huggingface_hub
24 modelscope
25 flash-attn==2.8.0.post2
26 accelerate
```

2、配置vllm的启动命令

1、拿到模型权重：把 DotsOCR 的模型文件下载到本地某个目录（建议 `./weights/DotsOCR`）。这样你可以离线启动，且路径可被 vLLM 直接指向加载。

2、让 Python 能“看到”模型内的自定义代码：

- 由于 DotsOCR 提供了 vLLM 的自定义适配器 `modeling_dots_ocr_vllm.py`，需要作为模块被导入。
- 把权重的父目录加进 `PYTHONPATH`，并保证目录名可作为合法包名（因此**不要有点**）。

3、在 vLLM 启动前“注册”模型：

这句 `sed` 会在可执行脚本 `vllm`（入口）里插入一行 `from DotsOCR import modeling_dots_ocr_vllm`。

原因：DotsOCR 提供了一个 `modeling_dots_ocr_vllm.py`，里头实现了 vLLM 需要的自定义逻辑（模型与处理器的注册），必须在 vLLM 启动时先被导入一次，否则 vLLM 不知道如何加载该模型。

- 通过 `sed` 给 vLLM 的入口脚本插入一行导入代码，确保进程一启动就执行 DotsOCR 的注册逻辑。
- 这属于临时工作流，方便在 Transformers 尚未完全整合前，vLLM 能识别并加载该模型。

4、启动服务并设置关键开关：

- `--trust-remote-code` 让 vLLM 运行模型仓库中的自定义组件。
- `--chat-template-content-format string` 让消息模板以字符串渲染，匹配 DotsOCR 的处理方式。
- `--served-model-name` 定义对外 API 的模型名；
- `--tensor-parallel-size` / `--gpu-memory-utilization` 控制多卡并行与显存预留策略。
- 如在国内或使用阿里云镜像，`VLLM_USE_MODELSCOPE=true` 用 ModelScope 拉取/解析资源。

代码块

```
1  # 设置模型路径环境变量
2  export hf_model_path=/root/autodl-tmp/models/rednote-hilab/DotsOCR
3
4  # vLLM 启动时通过环境变量找模型：以便 from DotsOCR import modeling_dots_ocr_vllm
  之类的动态 import
5  export PYTHONPATH=$(dirname "$hf_model_path"):$PYTHONPATH
6
7  # 动态修改 vLLM 入口以注册模型。 它直接修改了 vLLM 的安装文件，目的是在 vLLM 启动时预先
  加载（注册）你的自定义模型。
8  sed -i '/^from vllm\.entrypoints\.cli\.main import main$/a\
9  from DotsOCR import modeling_dots_ocr_vllm' `which vllm`
10
11 #
12 # VLLM_USE_MODELSCOPE=true: 一个环境变量，提示 vLLM 可能从 ModelScope（而非常见的
  Hugging Face）加载模型或配置
13 # CUDA_VISIBLE_DEVICES=0: 指定只在第一块 GPU（索引0）** 上运行模型
14 # tensor-parallel-size 1: 设置张量并行**的 GPU 数量为 1，表示不进行模型并行，仅在单卡
  上运行
15 # 执行命令
16 vllm serve ${hf_model_path} \
17 --tensor-parallel-size 1 \
18 --gpu-memory-utilization 0.95 \
19 --chat-template-content-format string \
20 --served-model-name dots_ocr \
21 --port 6006 \
22 --trust-remote-code
23
24 #
25 # python -m vllm.entrypoints.openai.api_server --model --host --port deng
```

3、解析代码开发

代码块

```
1
2  def do_parse(
3      input_path: str,
4      output: str = "./output",
5      prompt: str = "prompt_layout_all_en",
6      bbox: Optional[Tuple[int, int, int, int]] = None,
7      ip: str = "localhost",
8      port: int = 6006,
9      model_name: str = "dots_ocr",
10     temperature: float = 0.1,
11     top_p: float = 1.0,
12     dpi: int = 200,
13     max_completion_tokens: int = 16384,
14     num_thread: int = 16,
15     no_fitz_preprocess: bool = False,
16     min_pixels: Optional[int] = None,
17     max_pixels: Optional[int] = None,
18     use_hf: bool = False
19 ):
20     """
21     dots.ocr 多语言文档布局解析器
22
23     参数:
24         input_path (str): 输入PDF/图像文件路径
25         output (str): 输出目录 (默认: ./output)
26         prompt (str): 用于查询模型的提示词, 不同任务使用不同的提示词
27         bbox (Optional[Tuple[int, int, int, int]]): 边界框坐标 (x1, y1, x2, y2)
28         ip (str): 服务器IP地址 (默认: localhost)
29         port (int): 服务器端口 (默认: 8000)
30         model_name (str): 模型名称 (默认: model)
31         temperature (float): 温度参数 (默认: 0.1)
32         top_p (float): 核采样参数 (默认: 1.0)
33         dpi (int): DPI设置 (默认: 200)
34         max_completion_tokens (int): 最大完成标记数 (默认: 16384)
35         num_thread (int): 线程数 (默认: 16)
36         no_fitz_preprocess (bool): 是否禁用Fitz预处理 (默认: False)指的是选择是否使
37         用PyMuPDF (fitz) 库对图像输入进行特定的预处理操作
38         min_pixels (Optional[int]): 最小像素数
39         max_pixels (Optional[int]): 最大像素数
40         use_hf (bool): 是否使用HuggingFace (默认: False)
41     """
```

```

41     # 获取所有可用的提示模式
42     prompts = list(dict_promptmode_to_prompt.keys())
43
44     # 验证prompt参数是否有效
45     if prompt not in prompts:
46         raise ValueError(f"无效的prompt参数: {prompt}。可选值: {prompts}")
47
48     # 创建DotsOCR解析器实例
49     dots_ocr_parser = DotsOCRParser(
50         ip=ip,
51         port=port,
52         model_name=model_name,
53         temperature=temperature,
54         top_p=top_p,
55         max_completion_tokens=max_completion_tokens,
56         num_thread=num_thread,
57         dpi=dpi,
58         output_dir=output,
59         min_pixels=min_pixels,
60         max_pixels=max_pixels,
61         use_hf=use_hf,
62     )
63
64     # 设置Fitz预处理标志
65     fitz_preprocess = not no_fitz_preprocess
66     if fitz_preprocess:
67         print(f"对图像输入使用Fitz预处理，请检查图像像素的变化")
68
69     # 解析文件
70     result = dots_ocr_parser.parse_file(
71         input_path,
72         prompt_mode=prompt,
73         bbox=bbox,
74         fitz_preprocess=fitz_preprocess,
75     )
76
77     return result
78
79
80 if __name__ == "__main__":
81     # main()
82     # do_parse(input_path='../demo_image1.jpg')
83     # do_parse(input_path='../demo_pdf1.pdf', num_thread=32)
84     do_parse(input_path='../第一章 Apache Flink 概述.pdf', num_thread=32)

```

第六章：安装Milvus数据库

1、Milvus 概述

向量是神经网络模型的输出数据格式，可以有效地对信息进行编码，在知识库、语义搜索、检索增强生成（RAG）等人工智能应用中发挥着举足轻重的作用。

Milvus 是一个开源的向量数据库，适合各种规模的人工智能应用。

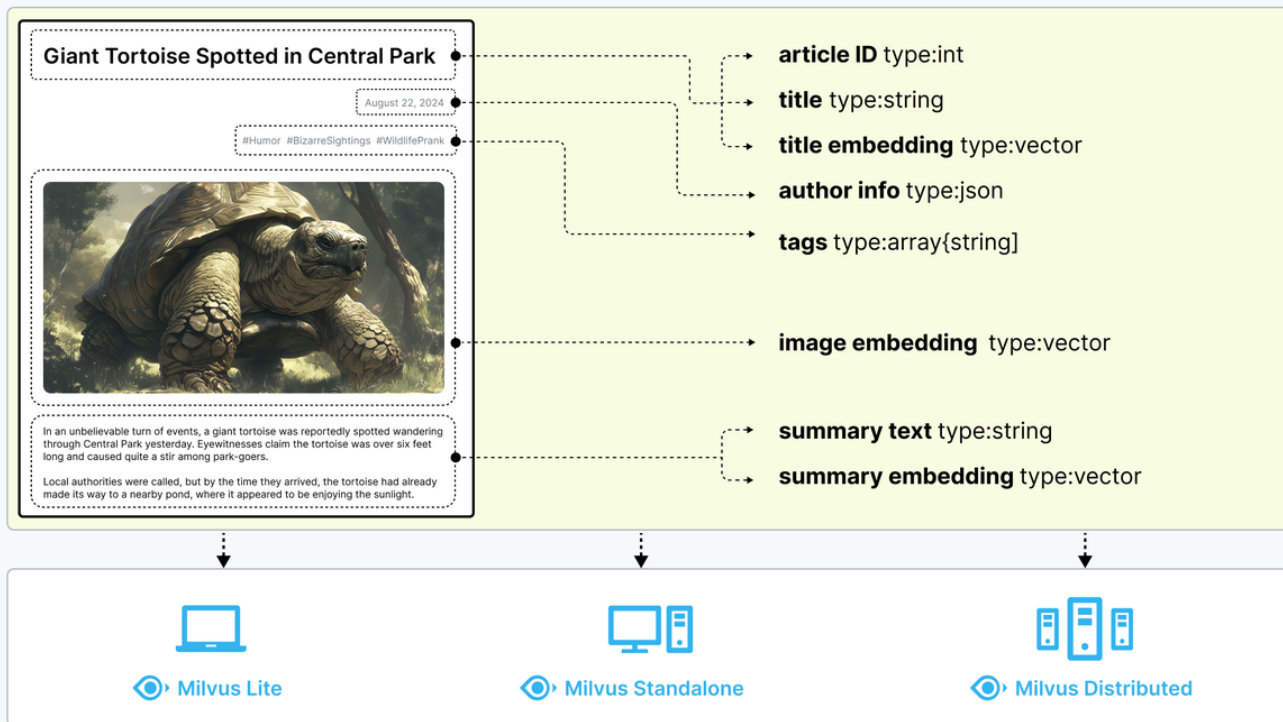
Milvus 是一种高性能、高扩展性的向量数据库，可在从笔记本电脑到大规模分布式系统等各种环境中高效运行。它既可以开源软件的形式提供，也可以云服务的形式提供。

Milvus 是一个开源项目，以 Apache 2.0 许可发布。大多数贡献者都是高性能计算（HPC）领域的专家，擅长构建大型系统和优化硬件感知代码。核心贡献者包括来自 Zilliz、ARM、NVIDIA、AMD、英特尔、Meta、IBM、Salesforce、阿里巴巴和微软的专业人士。

Embeddings 和 Milvus

文本、图像和音频等非结构化数据格式各异，并带有丰富的底层语义，因此分析起来极具挑战性。为了处理这种复杂性，Embeddings 被用来将非结构化数据转换成能够捕捉其基本特征的数字向量。然后将这些向量存储在向量数据库中，从而实现快速、可扩展的搜索和分析。

Milvus 提供强大的数据建模功能，使您能够将非结构化或多模式数据组织成结构化的 Collections。它支持多种数据类型，适用于不同的属性模型，包括常见的数字和字符类型、各种向量类型、数组、集合和 JSON，为您节省了维护多个数据库系统的精力。



Milvus 提供三种部署模式，涵盖各种数据规模--从本地原型到管理数百亿向量的大规模 Kubernetes 集群：

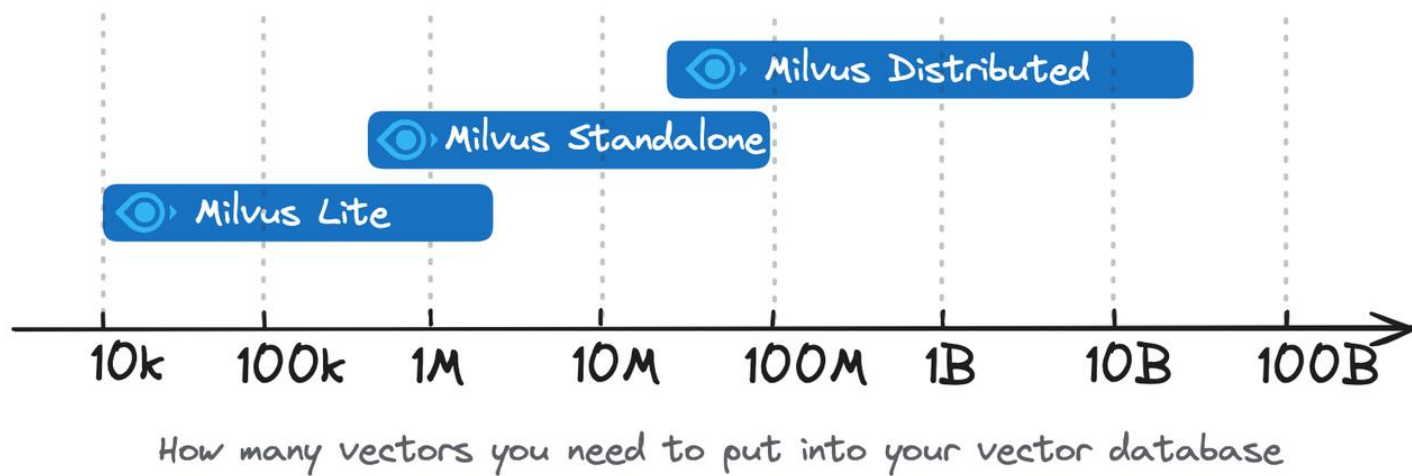
- Milvus Lite 是一个 Python 库，可以轻松集成到您的应用程序中。作为 Milvus 的轻量级版本，它非常适合在 开发环境 中进行快速原型开发，或在资源有限的边缘设备上运行。
- Milvus Standalone 是单机服务器部署，所有组件都捆绑在一个 Docker 镜像中，方便部署。
- Milvus Distributed 可部署在 Kubernetes 集群上，采用云原生架构，专为十亿规模甚至更大的场景而设计。该架构可确保关键组件的冗余。

功能比较

功能	Milvus Lite	Milvus 单机版	分布式 Milvus
SDK / 客户端软件	Python gRPC	Python Go Java Node.js C# RESTful	Python Java Go Node.js C# RESTful
数据类型	密集向量 稀疏向量 二进制向量 布尔值 整数 浮点 VarChar 数组 JSON	密集向量 稀疏向量 二进制向量 布尔型 整数 浮点型 VarChar 数组 JSON	密集向量 稀疏向量 二进制向量 布尔值 整数 浮点 VarChar 数组 JSON
搜索功能	向量搜索 (ANN 搜索) 元数据过滤 范围搜索 标量查询 通过主键获取实体 混合搜索	向量搜索 (ANN 搜索) 元数据过滤 范围搜索 标量查询 通过主键获取实体 混合搜索	向量搜索 (ANN 搜索) 元数据过滤 范围搜索 标量查询 通过主键获取实体 混合搜索
CRUD 操作符	✓	✓	✓
高级数据管理	不适用	访问控制 分区 分区密钥	访问控制 分区 分区密钥 物理资源分组
一致性级别	强	强 有界停滞 会话 最终	强 有界稳定性 会话 最终

Milvus 部署模式的选择取决于项目的阶段和规模。Milvus 为从快速原型开发到大规模企业部署的各种需求提供了灵活而强大的解决方案。

- Milvus Lite建议用于较小的数据集，多达几百万个向量。
- Milvus Standalone适用于中型数据集，可扩展至 1 亿向量。
- Milvus Distributed 专为大规模部署而设计，能够处理从一亿到数百亿向量的数据集。



Milvus 为何如此快速？

Milvus 从设计之初就是一个高效的向量数据库系统。在大多数情况下，Milvus 的性能是其他向量数据库的 2-5 倍。这种高性能是几个关键设计决策的结果：

硬件感知优化：为了让 Milvus 适应各种硬件环境，我们专门针对多种硬件架构和平台优化了其性能，包括 AVX512、SIMD、GPU 和 NVMe SSD。

高级搜索算法：Milvus 支持多种内存和磁盘索引/搜索算法，包括 IVF、HNSW、DiskANN 等，所有这些算法都经过了深度优化。与 FAISS 和 HNSWLib 等流行实现相比，Milvus 的性能提高了 30%-70%。

C++ 搜索引擎向量数据库性能的 80% 以上取决于其搜索引擎。由于 C++ 语言的高性能、底层优化和高效资源管理，Milvus 将 C++ 用于这一关键组件。最重要的是，Milvus 集成了大量硬件感知代码优化，从汇编级向量到多线程并行化和调度，以充分利用硬件能力。

面向列：Milvus 是面向列的向量数据库系统。其主要优势来自数据访问模式。在执行查询时，面向列的数据库只读取查询中涉及的特定字段，而不是整行，这大大减少了访问的数据量。此外，对基于列的数据的操作可以很容易地进行向量化，从而可以一次性在整个列中应用操作，进一步提高性能。

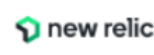
Milvus 支持的搜索类型

Milvus 支持各种类型的搜索功能，以满足不同用例的需求：

- **ANN 搜索：**查找最接近查询向量的前 K 个向量。
- **过滤搜索：**在指定的过滤条件下执行 ANN 搜索。
- **范围搜索：**查找查询向量指定半径范围内的向量。
- **混合搜索：**基于多个向量场进行 ANN 搜索。
- **全文搜索：**基于 BM25 的全文搜索。
- **Rerankers：**根据附加标准或辅助算法调整搜索结果顺序，完善初始 ANN 搜索结果。
- **获取：**根据主键检索数据。
- **查询**使用特定表达式检索数据。

人工智能集成

- **Embeddings 模型集成** Embedding 模型将非结构化数据转换为其在高维数据空间中的数字表示，以便您能将其存储在 Milvus 中。目前，PyMilvus（Python SDK）集成了多个嵌入模型，以便您能快速将数据准备成向量嵌入。
- **Reranker 模型集成** 在信息检索和生成式人工智能领域，Reranker 是优化初始搜索结果顺序的重要工具。PyMilvus 也集成了几种 Rerankers 模型，以优化初始搜索返回结果的顺序。
- **LangChain 和其他人工智能工具集成** 在 GenAI 时代，LangChain 等工具受到了应用程序开发人员的广泛关注。作为核心组件，Milvus 通常在此类工具中充当向量存储。



2、安装Milvus

Milvus Lite 是 Milvus 的轻量级版本，Milvus Lite 可导入您的 Python 应用程序，提供 Milvus 的核心向量搜索功能。Milvus Lite 已包含在 Milvus 的 Python SDK 中。它可以通过 `pip install pymilvus` 简单地部署。在 `pymilvus` 中，指定一个本地文件名作为 MilvusClient 的 uri 参数将使用 Milvus Lite。

https://milvus.io/docs/zh/milvus_lite.md

一、安装和配置docker(Centos9或者RedHat 8~9)

代码块

```
1 # 1、添加阿里云 Docker 仓库
2 sudo dnf config-manager --add-repo https://mirrors.aliyun.com/docker-
  ce/linux/centos/docker-ce.repo
3
4
5 # ubuntu apt 1、添加阿里云 Docker 仓库
```

```

6  curl -fsSL http://mirrors.aliyun.com/docker-ce/linux/ubuntu/gpg | sudo apt-key
    add -
7  curl -fsSL https://mirrors.aliyun.com/docker-ce/linux/ubuntu/gpg | sudo gpg --
    dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
8  echo "deb [arch=$(dpkg --print-architecture) signed-
    by=/usr/share/keyrings/docker-archive-keyring.gpg]
    https://mirrors.aliyun.com/docker-ce/linux/ubuntu $(lsb_release -cs) stable" |
    sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
9
10
11
12  # 2、安装 Docker 引擎
13  sudo dnf install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
    docker-compose-plugin
14
15  # apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
    docker-compose-plugin
16
17  # 3、启动和开机启动
18  sudo systemctl start docker
19  sudo systemctl enable docker
20
21
22  # 4、配置镜像加速器:为了进一步提升拉取 Docker 镜像的速度, 建议配置云镜像加速器
23  sudo mkdir -p /etc/docker
24  sudo vim /etc/docker/daemon.json
25  把下面的内容复制到: daemon.json里面, 保存退出。
26  {
27  "registry-mirrors":
28      [
29          "http://hub-mirror.c.163.com",
30          "https://mirrors.tuna.tsinghua.edu.cn",
31          "http://mirrors.sohu.com",
32          "https://ustc-edu-cn.mirror.aliyuncs.com",
33          "https://ccr.ccs.tencentyun.com",
34          "https://docker.m.daocloud.io",
35          "https://docker.aws19527.cn"
36      ]
37  }
38  sudo systemctl daemon-reload # 重新加载
39  sudo systemctl restart docker

```

二、安装和启动Milvus数据服务

```
1 # 1、下载官网指定的脚本文件
2 # Milvus 在 Milvus 资源库中提供了 Docker Compose 配置文件。要使用 Docker Compose 安
  装 Milvus，只需运行
3 wget https://github.com/milvus-io/milvus/releases/download/v2.6.0/milvus-
  standalone-docker-compose.yml -O docker-compose.yml
4
5 # 2、启动、停止 和删除Milvus数据库的docker容器
6 sudo docker compose up -d
7 docker restart milvus-standalone
8 # Stop Milvus
9 sudo docker compose down
10
11 # Delete service data
12 sudo rm -rf volumes
13
14 # 安装可视化客户端attu
15 docker run -d -p 8080:3000 -e MILVUS_URL=localhost:19530 zilliz/attu:v2.6
```

下载Docker Compose 配置文件

```
[root@iz8vb5acpt0ebqsuf5mtiwZ home]# wget https://github.com/milvus-io/milvus/releases/download/v2.6.0/milvus-standalone-docker-compose.yml -O docker-compose.yml
--2025-09-06 15:57:04-- https://github.com/milvus-io/milvus/releases/download/v2.6.0/milvus-standalone-docker-compose.yml
Resolving github.com (github.com)... 20.205.243.166
Connecting to github.com (github.com)|20.205.243.166|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://release-assets.githubusercontent.com/github-production-release-asset/208728772/fb20162d-c9b3-43c0-ae2b-a9704265272f?sp=r&sv=2018-11-09&sr=b&spr=https&se=2025-09-06T08%3A37%3A32Z&rscd=attachment%3B+filename%3Dmilvus-standalone-docker-compose.yml&rscst=application%2Foctet-stream&skoid=96c2d410-5711-43a1-aedd-ab1947aa7ab0&sktid=398a6654-997b-47e9-b12b-9515b896b4de&skt=2025-09-06T07%3A37%3A13Z&ske=2025-09-06T08%3A37%3A32Z&skvs=b&skv=2018-11-09&sig=peDyAgjsiKLA45az%2BKpqzgPr5YLHGsdUqIuh5V4M3M%3D&jwt=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJnaXRodWludY29tIiwiaXYXkxjoiCMVsZWZfc2Z1hc3NldHMucmVubnRlbncuY29tIiwia2V5Ijoia2V5MSIsImV4cCI6MTc1NzE0NTcyNSwibmJmIjozNzU0MTQ1NDI1LlCjwYYRoIoicmVsZWZfc2ZWFzc2V0cHJvZHViZGlubi5ibG9iIlNmcmUud2luZG93cy5uZXQifQ.WnGNgaC2NWDMFs_v-Wur1NmDigX3uXRHDkZLHLtnM&response-content-disposition=attachment%3B+filename%3Dmilvus-standalone-docker-compose.yml&response-content-type=application
```

第一次启动：

```
docker-compose.yml          100%[=====>]    1.75K  2.91KB/s   in 0.6s
2025-09-06 15:57:23 (2.91 KB/s) - 'docker-compose.yml' saved [1788/1788]

[root@iZ8vb5acpt0ebqsuf5mtiwZ home]# ls
docker-compose.yml  download  embedEtcd.yaml  rag_env  standalone_embed.sh  test_graphrag  user.yaml  volumes
[root@iZ8vb5acpt0ebqsuf5mtiwZ home]# sudo docker compose up -d
WARN[0000] /home/docker-compose.yml: 'version' is obsolete
[+] Running 31/12
  ⚙ standalone [#####] 875.5MB / 876.2MB Pulling
  ✓ etcd Pulled
  ✓ minio Pulled
```

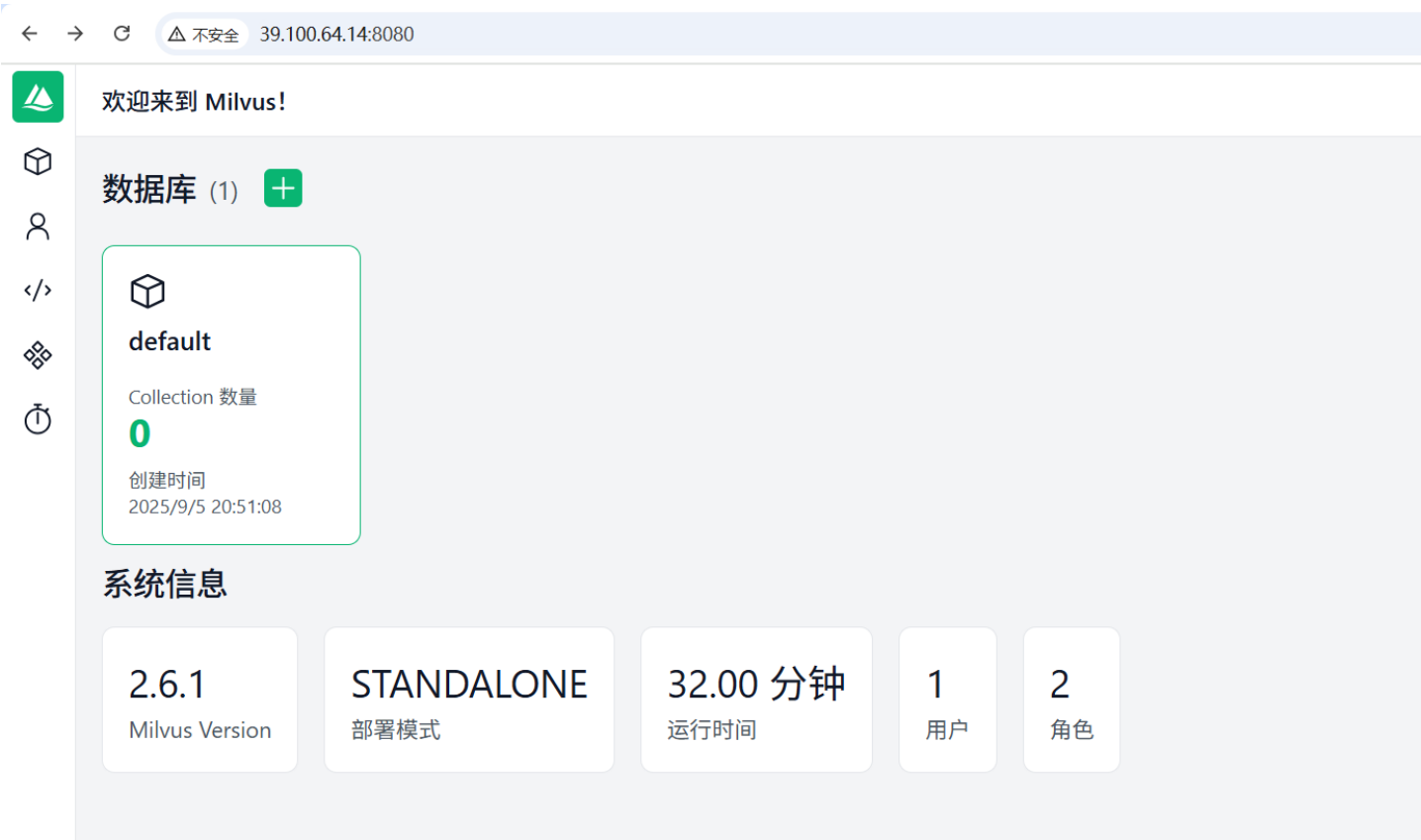
第一次启动会自动下载Milvus的docker镜像

```
[root@iZ8vb5acpt0ebqsuf5mtiwZ home]# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        NAMES
1feff57c72e7   milvusdb/milvus:v2.6.0             "/tini -- milvus run... 8 seconds ago  Up 7 seconds (he  milvus-st
dalone        0.0.0.0:9091->9091/tcp, :::9091->9091/tcp, 0.0.0.0:19530->19530/tcp, :::19530->19530/tcp
c3c62f4ed44f   minio/minio:RELEASE.2024-12-18T13-15-44Z  "/usr/bin/docker-ent... 2 minutes ago  Up 7 seconds (he  milvus-mi
o             0.0.0.0:9000-9001->9000-9001/tcp, :::9000-9001->9000-9001/tcp
ab9fc8080b74   quay.io/coreos/etcd:v3.5.18         "etcd -advertise-cli... 2 minutes ago  Up 7 seconds (he  milvus-et
th: starting) 2379-2380/tcp
[root@iZ8vb5acpt0ebqsuf5mtiwZ home]#
```

启动 Milvus 后，有三个名为milvus- standalone、milvus-minio 和milvus-etcd的容器启动。

- milvus-etcd容器不向主机暴露任何端口，并将其数据映射到当前文件夹中的volumes/etcd。
- milvus-minio容器使用默认身份验证凭据在本地为端口9090和9091提供服务，并将其数据映射到当前文件夹中的volumes/minio。
- Milvus-standalone容器使用默认设置为本地19530端口提供服务，并将其数据映射到当前文件夹中的volumes/milvus。

打开attu的客户端界面：



三、启动安全认证

a、下载数据库配置文件

```
代码块
1  wget https://raw.githubusercontent.com/milvus-io/milvus/v2.6.0/configs/milvus.yaml
2  # 如果下载不了，可以用Windows下载，然后在上传到服务器。
```

b、编辑配置文件：vi /etc/milvus.yaml

```

# The default value is 0, which means the caller will perform write operations directly
writer thread pool.
# In this case, the maximum concurrency of disk write operations is determined by the
diskWriteNumThreads: 0
security:
  authorizationEnabled: true
# The superusers will ignore some system check processes,
# like the old password verification when updating the credential
superUsers: root
# default password for root user. The maximum length is 72 characters.
# Large numeric passwords require double quotes to avoid yaml parsing precision issue
defaultRootPassword: Milvus
rootShouldBindRole: false # Whether the root user should bind a role when the author
enablePublicPrivilege: true # Whether to enable public privilege
rbac:
  overrideBuiltInPrivilegeGroups:
    enabled: false # Whether to override built in privilege groups
```

c、修改安装文件docker-compose.yml

在 `docker-compose.yml` 中，在每个 `milvus-standalone` 下添加 `volumes` 部分。

将 `milvus.yaml` 文件的本地路径映射到所有 `volumes` 部分下配置文件 `/milvus/configs/milvus.yaml` 的相应 docker 容器路径上。

```
standalone:
  container_name: milvus-standalone
  image: milvusdb/milvus:v2.6.0
  command: ["milvus", "run", "standalone"]
  security_opt:
    - seccomp:unconfined
  environment:
    ETCD_ENDPOINTS: etcd:2379
    MINIO_ADDRESS: minio:9000
    MQ_TYPE: woodpecker
  volumes:
    - /etc/milvus.yaml:/milvus/configs/milvus.yaml
    - ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/milvus:/var/lib/milvus
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:9091/healthz"]
    interval: 30s
    start_period: 90s
    timeout: 20s
    retries: 3
  ports:
```

增加一行配置

d、重启Milvus数据库

启动attu客户端容器。打开浏览器，输入用户名和密码（两种都行）。



Attu

版本: 2.6.0

☆ 给我一颗小星星

提交 Issue

Discord

联系专家

连接 Milvus 服务器 ⓘ

Milvus 地址 *

39.100.64.14:19530/default

Milvus 数据库 (可选)

default

认证

token (可选)

root:Milvus

root为用户名
Milvus为密码

用户名 (可选)

密码 (可选)

连接

☐ 启用 SSL ☒ 检查健康状态



Attu

版本: 2.6.0

☆ 给我一颗小星星

提交 Issue

Discord

联系专家

连接 Milvus 服务器 ⓘ

Milvus 地址 *

39.100.64.14:19530/default

Milvus 数据库 (可选)

default

认证

token (可选)

用户名 (可选)

root

密码 (可选)

.....

连接

☐ 启用 SSL ☒ 检查健康状态

3、操作Milvus数据库

当前版本的 Milvus 支持 Python、Node.js、GO 和 Java SDK。

要求

- 需要 Python 3.7 或更高版本。
- 已安装 Google protobuf。可以使用 `pip3 install protobuf` 命令安装。

- 已安装 grpcio-tools。可以使用 `pip3 install grpcio-tools` 命令安装。

代码块

```
1 python3 -m pip install pymilvus==2.6.0
```

一、数据库操作

在 Milvus 中，数据库是组织和管理数据的逻辑单元。为了提高数据安全性并实现多租户，你可以创建多个数据库，为不同的应用程序或租户从逻辑上隔离数据。例如，创建一个数据库用于存储用户 A 的数据，另一个数据库用于存储用户 B 的数据。

a、连接数据库

代码块

```
1 from pymilvus import MilvusClient
2
3 client = MilvusClient(
4     uri="http://39.100.64.14:19530",
5     token="root:Milvus",
6 )
7
8
9 client = MilvusClient(
10     uri="http://39.100.64.14:19530",
11     user="root",
12     password="Milvus",
13     # db_name="default",
14 )
15
16 print(client.list_databases())
17 print(client.list_users())
```

b、用户管理

代码块

```
1 # 创建新用户
2 client.create_user(
3     user_name="user_zs",
4     password="P@ssw0rd",
5 )
6
7 print(client.describe_user("user_zs"))
```



```
8
9  # 更新用户密码
10 client.update_password(
11     user_name="user_zs",
12     old_password="P@ssw0rd",
13     new_password="P@ssw0rd123"
14 )
15
16 # 删除用户
17 client.drop_user(user_name="user_zs")
```

c、数据库管理

每个数据库都有自己的属性，您可以在创建数据库时设置数据库属性。

属性名称	类型	属性描述
database.replica.number	整数	指定数据库的副本数量。
database.resource_groups	字符串	以逗号分隔的列表形式列出的与指定数据库相关的资源组。
database.diskQuota.mb	整数	指定数据库的最大磁盘空间大小（MB）。
database.max.collections	整数	指定数据库中允许的最大 Collections 数量。
database.force.deny.writing	布尔	是否强制指定的数据库拒绝写操作。
database.force.deny.reading	布尔	是否强制指定的数据库拒绝读取操作。

代码块

```
1  # 创建数据库
2  client.create_database(
3      db_name="my_database",
4      properties={
5          "database.max.collections": 10
6      }
7  )
8  print(client.list_databases())
9  # 查看数据库信息
10 print(client.describe_database(
11     db_name="my_database"
12 ))
13 # 使用数据库（切换数据库）
14 client.use_database(
15     db_name="my_database"
16 )
```

```

17 # 删除数据库
18 client.drop_database(
19     db_name="my_database"
20 )
21 print(client.list_databases())

```

4、collections和数据操作

在 Milvus 上，您可以创建多个 Collections 来管理数据，并将数据作为实体插入到 Collections 中。Collections 和实体类似于关系数据库中的表和记录。

Collection 是一个二维表，具有固定的列和变化的行。每列代表一个字段，每行代表一个实体。

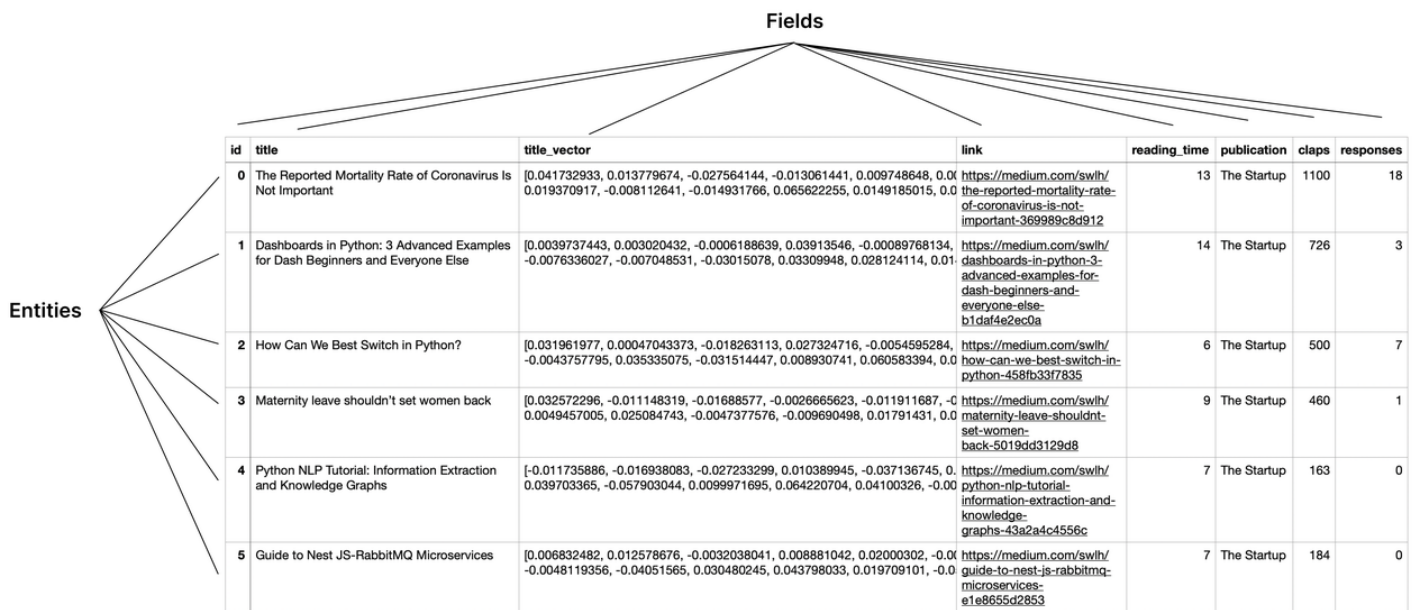
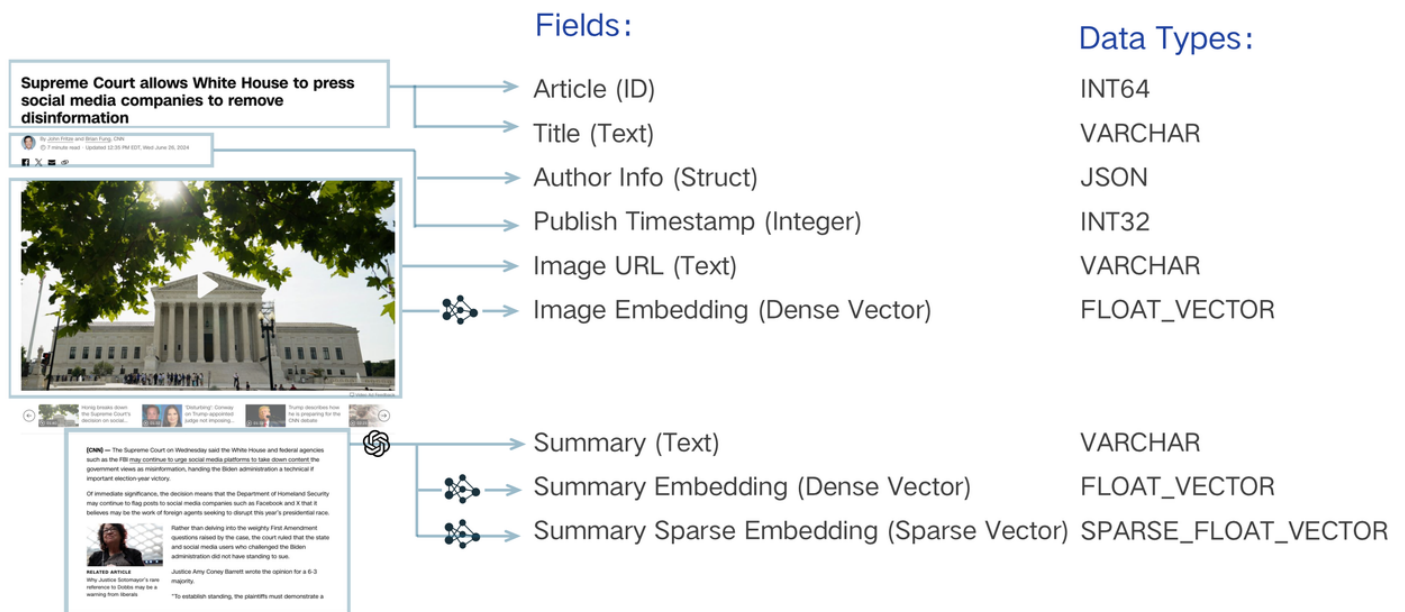


Figure 1: A Medium article collection in Zilliz Cloud database.

一、Schema 和字段

Schema 定义了 Collections 的数据结构。在创建一个 Collection 之前，你需要设计出它的 Schema。Collection Schema 定义了 数据库中如何组织 Collection 中的数据。一个 Collection Schema 有一个主键、最多四个向量字段和几个标量字段。下图说明了如何将文章映射到模式字段列表。



搜索系统的数据模型设计包括分析业务需求，并将信息抽象为模式表达的数据模型。例如，搜索一段文本必须通过 "嵌入" 将字面字符串转换为向量并启用向量搜索，从而实现 "索引"。除了这一基本要求外，可能还需要存储出版时间戳和作者等其他属性。有了这些元数据，就可以通过过滤来完善语义搜索，只返回特定日期之后或特定作者发表的文本。

代码块

```
1 from pymilvus import MilvusClient, DataType
2
3 schema = MilvusClient.create_schema()
4
```

a、主键和 Autoid

与关系数据库中的主字段类似，Collection 也有一个主字段，用于将实体与其他实体区分开来。主字段中的每个值都是全局唯一的，并与一个特定的实体相对应。

主字段只接受整数或字符串。插入实体时，默认情况下应包含主字段值。但是，如果在创建 Collections 时启用了 Autoid，Milvus 将在插入数据时生成这些值。在这种情况下，请从要插入的实体中排除主字段值。

代码块

```
1 schema.add_field(
2     field_name="my_id",
3     datatype=DataType.INT64,
4     is_primary=True,
5     auto_id=False,
6
7 )
8
```

```
9     schema.add_field(  
10         field_name="my_id",  
11         datatype=DataType.VARCHAR,  
12         is_primary=True,  
13         auto_id=True,  
14         max_length=512,  
15     )  
16
```

添加字段时，可以通过将 `is_primary` 属性设置为 `True` 来明确说明该字段是主字段。主字段默认接受 `Int64` 值。在这种情况下，主字段值应为整数，类似于 `12345`。如果选择在主字段中使用 `VarChar` 值，则其值应为字符串，类似于 `my_entity_1234`。

要使用 `VarChar` 主键，除了将 `data_type` 参数值更改为 `DataType.VARCHAR` 外，还需要为字段设置 `max_length` 参数。

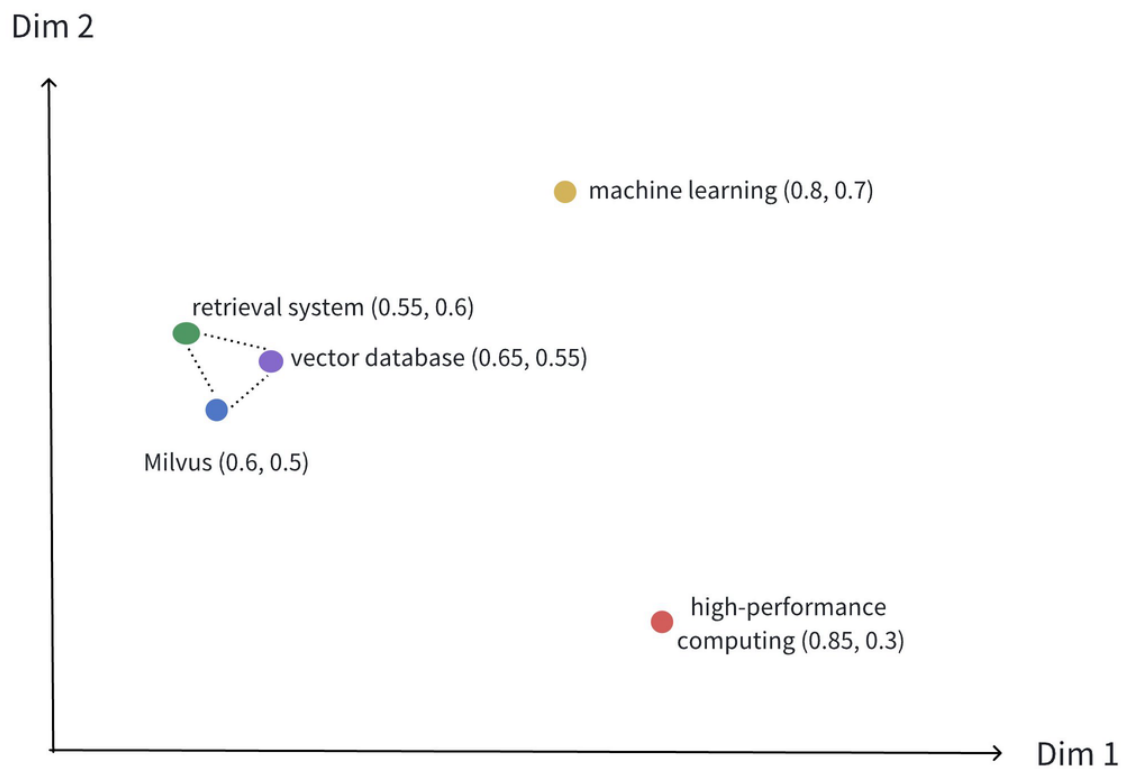
b、向量字段

向量字段接受各种稀疏和密集向量嵌入。，您可以向 `Collections` 添加最多四个向量字段。

密集向量主要用于需要理解数据语义的场景，如语义搜索和推荐系统。在语义搜索中，密集向量有助于捕捉查询和文档之间的潜在联系，提高搜索结果的相关性。在推荐系统中，密集矢量有助于识别用户和项目之间的相似性，从而提供更加个性化的建议。

稠密向量可使用各种[嵌入](#)模型生成，这些模型将原始数据转化为高维空间中的点，捕捉数据的语义特征。

密集向量通常表示为具有固定长度的浮点数数组，如 `[0.2, 0.7, 0.1, 0.8, 0.3, ..., 0.5]`。这些向量的维度通常从数百到数千不等，如 128、256、768 或 1024。每个维度都能捕捉对象的特定语义特征，通过相似性计算使其适用于各种场景。



Notes:

- The position of each point represents the relative relationship of words in the semantic space, with coordinate values indicating the strength of semantics across dimensions.
- Points that are closer together signify higher semantic similarity. The dashed lines indicate concepts that are closely related.

上图展示了密集向量在二维空间中的表现形式。虽然实际应用中的密集向量通常具有更高的维度，但这种二维插图有效地传达了几个关键概念：

- **多维表示：** 每个点代表一个概念对象（如Milvus、向量数据库、检索系统等），其位置由其维度值决定。
- **语义关系：** 点之间的距离反映了概念之间的语义相似性。距离较近的点表示语义关联度较高的概念。
- **聚类效应：** 相关概念（如Milvus、向量数据库和检索系统）在空间中的位置相互靠近，形成语义聚类。

代码块

```
1 schema.add_field(field_name="dense_vector", datatype=DataType.FLOAT_VECTOR,  
  dim=1024)
```

上述代码片段中的 `dim` 参数表示向量字段中要保存的向量嵌入的维数。 `FLOAT_VECTOR` 值表示向量字段持有 32 位浮点数列表，通常用于表示反比例。除此之外，还支持以下类型的向量嵌入：

- `FLOAT16_VECTOR`

这种类型的向量场保存一个 16 位半精度浮点数列表，通常适用于内存或带宽受限的深度学习或基于 GPU 的计算场景。

- `BFLOAT16_VECTOR`

这种类型的向量字段保存 16 位浮点数列表，精度有所降低，但指数范围与 Float32 相同。这种类型的数据常用于深度学习场景，因为它能在不明显影响精度的情况下减少内存使用量。

- `INT8_VECTOR`

这种类型的向量字段存储由 8 位有符号整数（int8）组成的向量，每个分量的范围为-128 到 127。它专为量化深度学习架构（如 ResNet 和 EfficientNet）量身定制，可大幅缩小模型大小，提高推理速度，同时只造成极小的精度损失。注：该向量类型仅支持 HNSW 索引。

- `BINARY_VECTOR`

这种类型的向量场保存着一个 0 和 1 的列表。在图像处理和信息检索场景中，它们是表示数据的紧凑特征。

为向量字段设置索引参数

为了加速语义搜索，必须为向量字段创建索引。索引可以大大提高大规模向量数据的检索效率。

代码块

```
1  index_params = client.prepare_index_params()
2
3  index_params.add_index(
4      field_name="dense_vector",
5      index_name="dense_vector_index",
6      index_type="AUTOINDEX",
7      metric_type="IP"
8  )
9
10 client.create_collection(
11     collection_name="my_collection",
12     schema=schema,
13     index_params=index_params
14 )
15
```

在上面的示例中，使用 `AUTOINDEX` 索引类型为 `dense_vector` 字段创建了名为 `dense_vector_index` 的索引。`metric_type` 设置为 `IP`，表示将使用内积作为距离度量。Milvus 提供多种索引类型，以获得更好的向量搜索体验。`AUTOINDEX` 是 Milvus 根据数据特征自动选择最优索引类型，旨在平滑向量搜索的学习曲线。

目前，Milvus 支持这些类型的相似性度量：欧氏距离（`L2`）、内积（`IP`）、余弦相似度（`COSINE`）、`JACCARD`（杰卡德距离）、`HAMMING`（汉明距离）和 `BM25`（专门为稀疏向量的全文检索而设计）。

度量类型	相似性距离值的特征	相似性距离值范围
L2	值越小表示相似度越高。	$[0, \infty)$
IP	值越大，表示相似度越高。	$[-1, 1]$
COSINE	数值越大，表示相似度越高。	$[-1, 1]$
JACCARD	值越小，表示相似度越高。	$[0, 1]$
MHJACCARD	根据 MinHash 签名位估算 Jaccard 相似度；距离越小 = 越相似	$[0, 1]$
HAMMING	值越小表示相似度越高。	$[0, \text{dim}(\text{向量})]$
BM25	根据术语频率、反转文档频率和文档规范化对相关性进行评分。	$[0, \infty)$

欧氏距离（L2）

从本质上讲，欧氏距离测量的是连接两点的线段的长度。这是最常用的距离度量，在数据连续时非常有用。

欧氏距离的计算公式如下：

$$d(a, b) = d(b, a) = \sqrt{\sum_{i=0}^{n-1} (b_i - a_i)^2}$$

内积（IP）

两个 Embeddings 之间的 IP 距离。如果使用 IP 计算嵌入式之间的相似性，必须对嵌入式进行归一化处理。归一化后，内积等于余弦相似度。定义如下：

$$p(A, B) = A \cdot B = \sum_{i=0}^{n-1} a_i \cdot b_i$$

余弦相似性

余弦相似度使用两组向量之间角度的余弦来衡量它们的相似程度。你可以把两组向量看成从同一点（如 [0,0,...]）出发，但指向不同方向的线段。

要计算两组向量 $A = ((a_0), (a_1), \dots, (a_{n-1}))$ 和 $B = ((b_0), (b_1), \dots, (b_{n-1}))$ 之间的余弦相似度，请使用下面的公式：

$$\cos\theta = \frac{\sum_0^{n-1} (a_i \cdot b_i)}{\sqrt{\sum_0^{n-1} a_i^2} \cdot \sqrt{\sum_0^{n-1} b_i^2}}$$

余弦相似度总是在区间[-1, 1] 内。例如，两个正比向量的余弦相似度为1，两个正交向量的余弦相似度为0，两个相反向量的余弦相似度为-1。余弦越大，两个向量之间的夹角越小，说明这两个向量之间的相似度越高。

用 1 减去它们的余弦相似度，就可以得到两个向量之间的余弦距离。

BM25 相似性

BM25 是一种广泛使用的文本相关性测量方法，专门用于[全文检索](#)。它结合了以下三个关键因素：

- 术语频率 (TF)：衡量术语在文档中出现的频率。虽然较高的频率通常表示较高的重要性，但 BM25 使用饱和参数 k_1 来防止过于频繁的术语主导相关性得分。
- 反向文档频率 (IDF)：反映术语在整个语料库中的重要性。在较少文档中出现的术语会获得较高的 IDF 值，这表明其对相关性的贡献更大。
- 文档长度归一化：较长的文档由于包含较多的术语，往往得分较高。BM25 通过对文档长度进行归一化处理来减轻这种偏差，参数 b 控制这种归一化处理的强度。

BM25 评分的计算方法如下：

$$\text{score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{\text{TF}(q_i, D) \cdot (k_1 + 1)}{\text{TF}(q_i, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{\text{avgdl}})}$$

c、稀疏向量

稀疏向量是信息检索和自然语言处理中捕捉**表层术语**匹配的重要方法。虽然稠密向量在语义理解方面表现出色，但稀疏向量往往能提供更可预测的匹配结果，尤其是在搜索特殊术语或文本标识符时。

稀疏向量是一种特殊的高维向量，其中大部分元素为零，只有少数维度具有非零值。

- 一个 `SPARSE_FLOAT_VECTOR` 字段，预留用于存储稀疏向量，可以从 `VARCHAR` 字段自动生成，也可以直接在输入数据中提供。

- 通常，稀疏向量所代表的原始文本也会存储在 Collections 中。您可以使用 `VARCHAR` 字段来存储原始文本。

代码块

```
1 schema.add_field(field_name="sparse_vector",
  datatype=DataType.SPARSE_FLOAT_VECTOR)
2 schema.add_field(field_name="text", datatype=DataType.VARCHAR,
  max_length=65535,
```

在此示例中，添加了两个字段：

- `sparse_vector` :该字段使用 `SPARSE_FLOAT_VECTOR` 数据类型存储稀疏向量。
- `text` :该字段使用 `VARCHAR` 数据类型存储文本字符串，最大长度为 65535 字节。

设置索引参数

为稀疏向量创建索引的过程与为稠密向量创建索引的过程类似，但在指定的索引类型 (`index_type`)、距离度量 (`metric_type`) 和索引参数 (`params`) 上有所不同。

代码块

```
1 index_params = client.prepare_index_params()
2
3 index_params.add_index(
4     field_name="sparse_vector",
5     index_name="sparse_inverted_index",
6     index_type="SPARSE_INVERTED_INDEX",
7     metric_type="BM25",
8     params={"inverted_index_algo": "DAAT_MAXSCORE"}, # or "DAAT_WAND" or
  "TAAT_NAIVE"
9 )
10
11
```

- `index_type` :要建立的索引类型。在本例中，将值设为 `SPARSE_INVERTED_INDEX` 。
- `metric_type` :用于计算稀疏向量间相似性的度量。有效值：
 - `IP` (内积) : 使用点积衡量相似性。
 - `BM25` :通常用于全文搜索，侧重于文本相似性。
- `params.inverted_index_algo` :用于建立和查询索引的算法。有效值：

- `"DAAT_MAXSCORE"` (默认)：使用 MaxScore 算法进行优化的 Document-at-a-Time (DAAT) 查询处理。MaxScore 通过跳过可能影响最小的术语和文档，为高 k 值或包含大量术语的查询提供更好的性能。为此，它根据最大影响分值将术语划分为基本组和非基本组，并将重点放在对前 k 结果有贡献的术语上。
- `"DAAT_WAND"` :使用 WAND 算法优化 DAAT 查询处理。WAND 算法利用最大影响分数跳过非竞争性文档，从而评估较少的命中文档，但每次命中的开销较高。这使得 WAND 对于 k 值较小的查询或较短的查询更有效，因为在这些情况下跳过更可行。
- `"TAAT_NAIVE"` :基本术语一次查询处理 (TAAT)。虽然与 `DAAT_MAXSCORE` 和 `DAAT_WAND` 相比速度较慢，但 `TAAT_NAIVE` 具有独特的优势。DAAT 算法使用的是缓存的最大影响分数，无论全局 Collections 参数 (avgdl) 如何变化，这些分数都保持静态，而 `TAAT_NAIVE` 不同，它能动态地适应这种变化。

特性	DAAT_MAXSCORE (文档遍历-最大分数)	DAAT_WAND (文档遍历-和与
核心思想	按文档ID顺序处理，利用每个词项的 最大可能得分 设置阈值，动态剪枝	按文档ID顺序处理，利用词项得分进行动态阈值剪枝
计算效率	高 （积极的剪枝策略能跳过大量不相关文档）	高 （剪枝策略比MAXSCORE更激进，查询时排序开销大）
内存效率	中等（需维护词项的最大得分信息）	中等（需维护词项的上界信息）
结果准确性	精确（保证返回Top-K结果）	精确（保证返回Top-K结果）
适用场景	通用且高效 ，尤其适合 查询词项数量较多或各词项权重差异大 的场景	适合 查询词项较少 的场景，长查询能成为瓶颈
优势	在查询长度和数据集规模上表现 均衡 ，剪枝效率高	能进行 非常激进的剪枝 ，尤其当查询限和与当前阈值关系明确时
劣势	需要预计算和存储每个词项的最大得分	处理长查询时， 对倒排链排序的开销大

d、全文索引和搜索

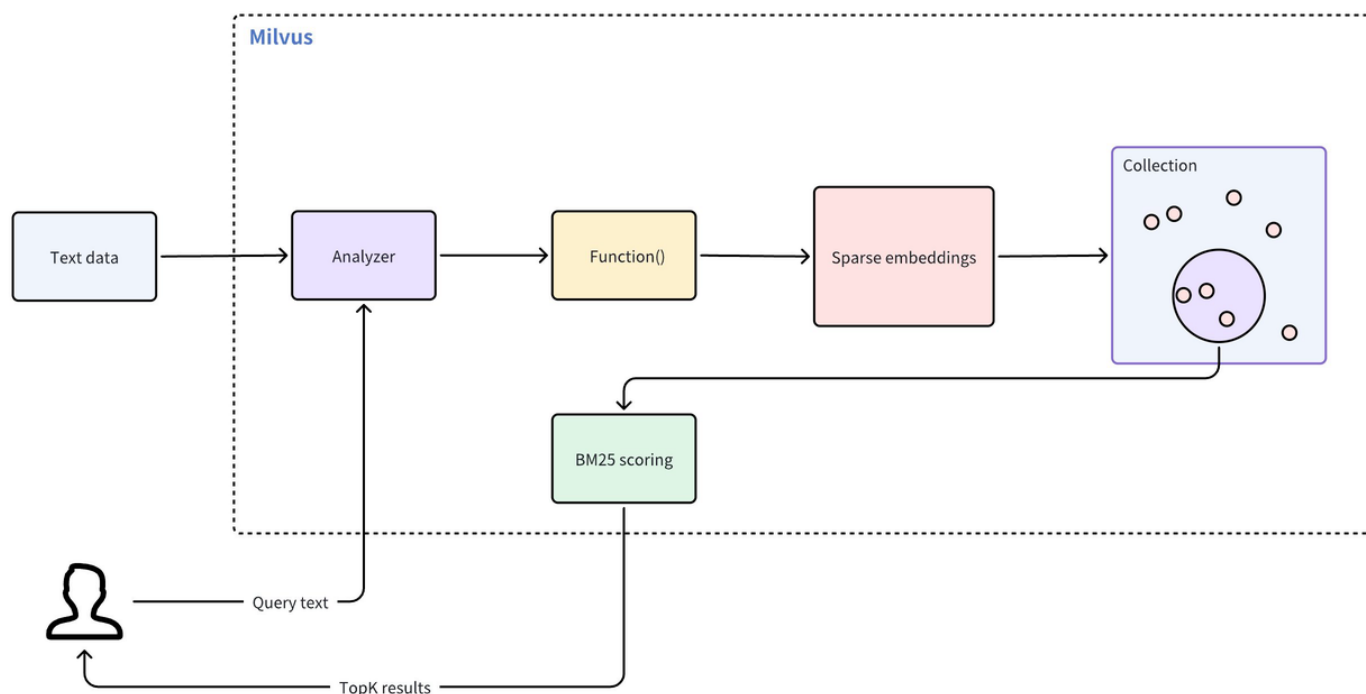
全文搜索是一种在文本数据集中检索包含特定术语或短语的文档，然后根据相关性对结果进行排序的功能。该功能克服了语义搜索的局限性（语义搜索可能会忽略精确的术语），确保您获得最准确且与上下文最相关的结果。此外，它还通过接受原始文本输入来简化向量搜索，自动将您的文本数据转换为稀疏嵌入，而无需手动生成向量嵌入。

该功能使用 BM25 算法进行相关性评分，在检索增强生成 (RAG) 场景中尤为重要，它能优先处理与特定搜索词密切匹配的文档。

总结：通过将全文检索与基于语义的密集向量搜索相结合，可以提高搜索结果的准确性和相关性。全文搜索无需手动嵌入，从而简化了基于文本的搜索过程。该功能通过以下工作流程进行操作符：

1. 文本输入：插入原始文本文档或提供查询文本，无需手动嵌入。

2. 文本分析：Milvus 使用分析器（分词）将输入文本标记为可搜索的单个术语。
3. 函数处理：内置函数接收标记化术语，并将其转换为稀疏向量表示。
4. Collections 存储：Milvus 将这些稀疏嵌入存储在 Collections 中，以便高效检索。
5. BM25 评分：在搜索过程中，Milvus 应用 BM25 算法为存储的文档计算分数，并根据与查询文本的相关性对匹配结果进行排序。



要启用全文搜索，请创建一个具有特定 Schema 的 Collections。此 Schema 必须包括三个必要字段：

- 唯一标识 Collections 中每个实体的主字段。
- 一个 `VARCHAR` 字段，用于存储原始文本文档，其 `enable_analyzer` 属性设置为 `True`（开启分析器）。这允许 Milvus 将文本标记为特定术语，以便进行函数处理。
- 一个 `SPARSE_FLOAT_VECTOR` 字段，预留用于存储稀疏嵌入，Milvus 将为 `VARCHAR` 字段自动生成稀疏嵌入。

分析器概述

在文本处理中，分析器是将原始文本转换为结构化可搜索格式的关键组件。每个分析器通常由两个核心部件组成：标记器和过滤器。它们共同将输入文本转换为标记，完善这些标记，并为高效索引和检索做好准备。

在 Milvus 中，创建 Collections 时，将 `VARCHAR` 字段添加到 Collections Schema 时，会对分析器进行配置。分析器生成的标记可用于建立关键字匹配索引，或转换为稀疏嵌入以进行全文检索。

`chinese` 分析器包括

- 标记化器：使用 `jieba` 标记化器，根据词汇和上下文将中文文本分割成标记。
- 过滤器：使用 `cnalphanumonly` 过滤器删除包含任何非汉字的标记。

```

1 analyzer_params = {
2     "tokenizer": "jieba",
3     "filter": ["cn_alphanumonly"]
4 }
5
6 analyzer_params = {
7     "type": "chinese",
8 }
9

```

`english` 分析器使用以下组件：

- 标记化器：使用 `standard` 标记化器将文本分割成离散的单词单位。
- 过滤器：包括多个过滤器，用于全面处理文本：
 - `lowercase` :将所有标记转换为小写，从而实现不区分大小写的搜索。
 - `stemmer` :将单词还原为词根形式，以支持更广泛的匹配（例如，"running"变为"run"）。
 - `stop_words` :删除常见的英文停止词，以便集中搜索文本中的关键词语。

代码块

```

1 analyzer_params = {
2     "tokenizer": "standard",
3     "filter": [
4         "lowercase",
5         {
6             "type": "stemmer",
7             "language": "english"
8         }, {
9             "type": "stop",
10            "stop_words": "_english_"
11        }
12    ]
13 }
14

```

现在，定义一个将文本转换为稀疏向量表示的函数，然后将其添加到 Schema 中：

代码块

```

1 bm25_function = Function(
2     name="text_bm25_emb", # Function name
3     input_field_names=["text"], # Name of the VARCHAR field containing raw
4     # text data
5     output_field_names=["sparse"],
6

```

```

5      # Name of the SPARSE_FLOAT_VECTOR field reserved to store generated
      embeddings
6      function_type=FunctionType.BM25, # Set to `BM25`
7  )
8  schema.add_function(bm25_function)

```

参数	说明
name	函数名称。该函数将text 字段中的原始文本转换为可搜索向量，这些向量存储在sparse 字段中。
input_field_names	需要将文本转换为稀疏向量的VARCHAR 字段的名称。对于FunctionType.BM25，该参数只接受一个字段名称。
output_field_names	存储内部生成的稀疏向量的字段名称。对于FunctionType.BM25，该参数只接受一个字段名称。
function_type	要使用的函数类型。将值设为FunctionType.BM25。

e、标量字段

在常见情况下，您可以使用标量字段来存储存储在 Milvus 中的向量嵌入的元数据，并通过元数据过滤进行 ANN 搜索，以提高搜索结果的正确性。支持多种标量字段类型，包括VarChar、Boolean、Int、Float、Double、Array 和JSON。

代码块

```

1  schema.add_field(
2      field_name="my_array",
3      datatype=DataType.ARRAY,
4      element_type=DataType.VARCHAR,
5      max_capacity=5,
6      max_length=512,
7  )
8
9  schema.add_field(
10     field_name="my_json",
11     datatype=DataType.JSON,
12 )
13
14 schema.add_field(
15     field_name="my_bool",
16     datatype=DataType.BOOL,
17 )
18
19 schema.add_field(
20     field_name="my_int64",

```

```

21     datatype=DataType.INT64, # Milvus 支持的数字类型有
    Int8,Int16,Int32,Int64,Float 和Double 。
22 )
23
24 schema.add_field(
25     field_name="my_varchar",
26     datatype=DataType.VARCHAR,
27     # highlight-next-line
28     max_length=512
29 )
30

```

f、可归零和默认值

Milvus 允许你为标量字段（主字段除外）设置 `nullable` 属性和默认值。对于标记为 `nullable=True` 的字段，您可以在插入数据时跳过该字段，或直接将其设置为空值，系统会将其视为空值而不会导致错误。当字段具有默认值时，如果在插入过程中没有为该字段指定数据，系统将自动应用该值。

限制规则

- 只有标量字段（主字段除外）支持默认值和 `nullable` 属性。
- JSON 和数组字段不支持默认值。
- 默认值或 `nullable` 属性只能在创建 Collections 时配置，之后不能修改。
- 标记为 `nullable` 的字段不能用作分区键。
- 在启用 `nullable` 属性的标量字段上创建索引时，索引将排除空值。
- JSON 和 ARRAY 字段：当使用 `IS NULL` 或 `IS NOT NULL` 操作符对 JSON 或 ARRAY 字段进行过滤时，这些操作符在列级别工作，这表明它们只评估整个 JSON 对象或数组是否为空。例如，如果 JSON 对象中的某个键为空，`IS NULL` 过滤器将无法识别该键。

代码块

```

1  schema.add_field(field_name="age", datatype=DataType.INT64, nullable=True)
2  schema.add_field(field_name="age", datatype=DataType.INT64, default_value=18)

```

可归零	默认值	默认值类型	用户输入	结果	示例
✓	✓	非空	无/空	使用默认值	字段： age 默认值： 18用户为18
✓	✗	-	无/空	存储为空	字段： middle_name 默认值果： 存储为空
✗	✓	非空	无/空	使用默认值	字段： status 默认值： "active"果： 存储为"active"
✗	✗	-	无/空	抛出错误	字段： email 默认值： -用户被拒绝，系统提示错误
✗	✓	空	无/空	抛出错误	字段： username 默认值： 空操作符被拒绝，系统提示错误

二、创建Collection

代码块

```
1  from pymilvus import MilvusClient, DataType, Function, FunctionType
2
3
4
5  client = MilvusClient(
6      uri="http://39.100.64.14:19530",
7      user="root",
8      password="Milvus",
9      # db_name="default",
10 )
11
12
13 schema = client.create_schema()
14 schema.add_field(field_name='id', datatype=DataType.INT64, is_primary=True,
15 auto_id=True)
16 schema.add_field(field_name='text', datatype=DataType.VARCHAR,
17 max_length=6000, enable_analyzer=True,
18 analyzer_params={"tokenizer": "jieba", "filter":
19 ["cn_alphanumonly"]})
20 schema.add_field(field_name='category', datatype=DataType.VARCHAR,
21 max_length=1000, nullable=True)
22 schema.add_field(field_name='filename', datatype=DataType.VARCHAR,
23 max_length=1000, nullable=True)
24 schema.add_field(field_name='filetype', datatype=DataType.VARCHAR,
25 max_length=1000, nullable=True)
26 schema.add_field(field_name='title', datatype=DataType.VARCHAR,
27 max_length=1000, nullable=True)
28 schema.add_field(field_name='sparse', datatype=DataType.SPARSE_FLOAT_VECTOR)
29 schema.add_field(field_name='dense', datatype=DataType.FLOAT_VECTOR, dim=1024)
```



```

23
24 bm25_function = Function(
25     name="text_bm25_emb", # Function name
26     input_field_names=["text"], # Name of the VARCHAR field containing raw
    text data
27     output_field_names=["sparse"],
28     function_type=FunctionType.BM25, # Set to `BM25`
29 )
30 schema.add_function(bm25_function)
31 index_params = client.prepare_index_params()
32
33 index_params.add_index(
34     field_name="sparse",
35     index_name="sparse_inverted_index",
36     index_type="SPARSE_INVERTED_INDEX", # Inverted index type for sparse
    vectors
37     metric_type="BM25",
38     params={
39         "inverted_index_algo": "DAAT_MAXSCORE",
40         "bm25_k1": 1.2, # 1.2 ~ 2.0 (1.2) 词频 (TF) 的饱和度: 高频词的贡献越大, 词
    频影响越线性, 饱和度增长越慢 (通俗: 控制一个词出现多少次才算“多”)
41         "bm25_b": 0.75 # 0.0 ~ 1.0 (0.75) 文档长度归一化的强度: 文档长度的影响越
    大, 对长文档的惩罚越强 (通俗: 控制“长篇大论”相对于“言简意赅”的劣势有多大, 旨在避免长文档仅
    仅因为包含更多词汇而在相似度计算中占据不公平的优势。)
42     },
43 )
44 index_params.add_index(
45     field_name="dense",
46     index_name="dense_inverted_index",
47     index_type="AUTOINDEX",
48     metric_type="IP"
49 )
50
51 client.create_collection(
52     collection_name='t_doc_collection',
53     schema=schema,
54     index_params=index_params
55 )
56
57 # 查看集合信息
58 res = client.describe_collection(
59     collection_name="t_doc_collection"
60 )
61
62 print(res)

```

代码块 加载 *Collections* 时, *Milvus* 会将索引文件和所有字段的原始数据加载到内存中, 以便快速响应搜索和查询。

```
2  # 在载入 Collections 后插入的实体会自动编入索引并载入。
3  client.load_collection(
4      collection_name="t_doc_collection"
5  )
6
7  res = client.get_load_state(
8      collection_name="t_doc_collection"
9  )
10
11 print(res)
12
13 # 释放当前不使用的 Collection。
14 client.release_collection(
15     collection_name="t_doc_collection"
16 )
17
18 res = client.get_load_state(
19     collection_name="t_doc_collection"
20 )
21
22 print(res)
23
24 # 删除集合
25 # client.drop_collection(
26 #     collection_name="t_doc_collection"
27 # )
```

三、插入数据

代码块

```
1  client = MilvusClient(
2      uri="http://39.100.64.14:19530",
3      user="root",
4      password="Milvus",
5      # db_name="default",
6  )
7  # 准备三条示例数据
8  sample_data = [
9      {
10         "text": "机器学习是人工智能的一个子领域, 它使计算机系统能够从数据中学习并改进, 而无需明确编程。常见的应用包括图像识别、自然语言处理和推荐系统。",
11         "category": "人工智能",
12         "filename": "ml_intro.pdf",
```

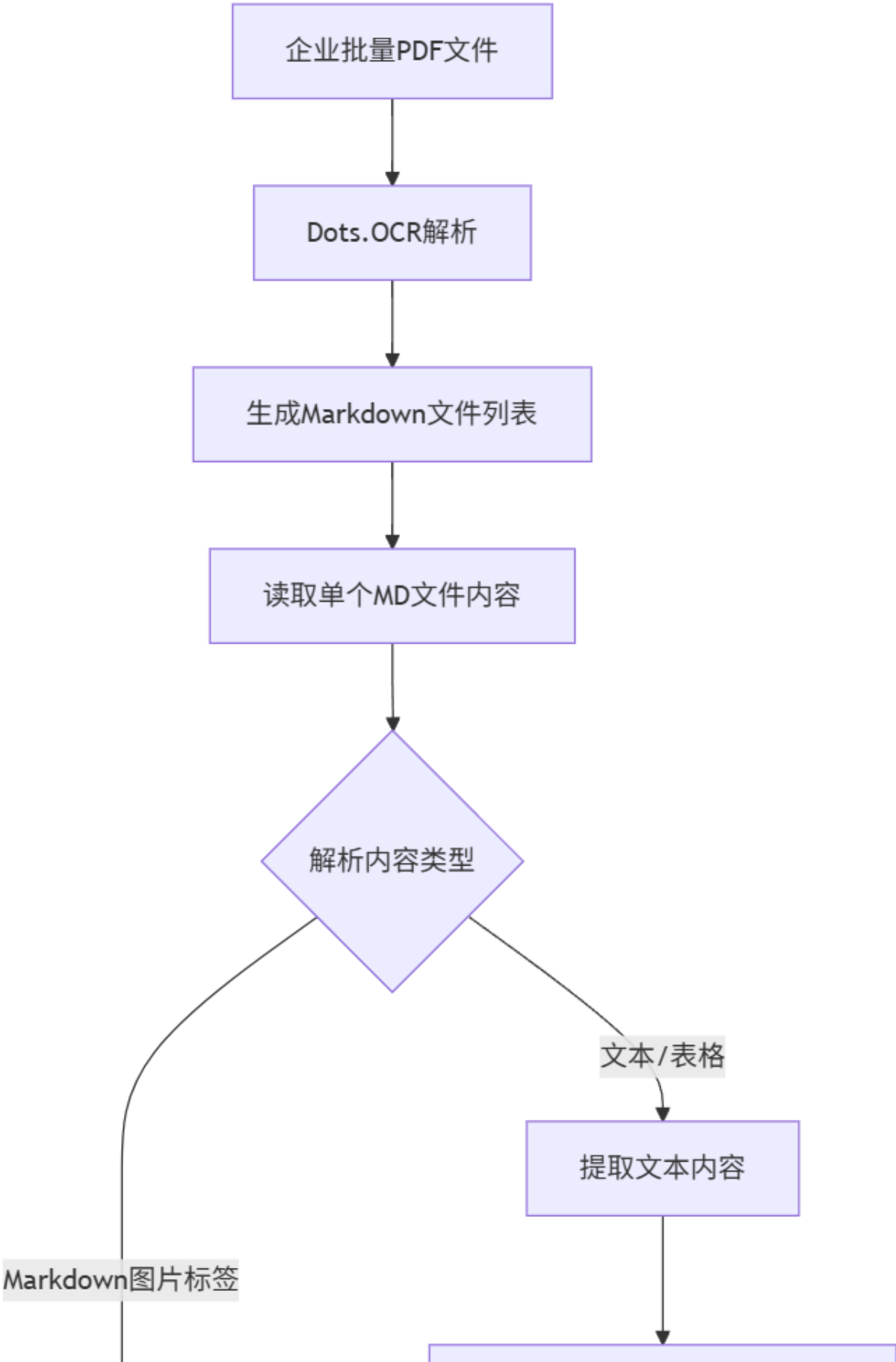
```

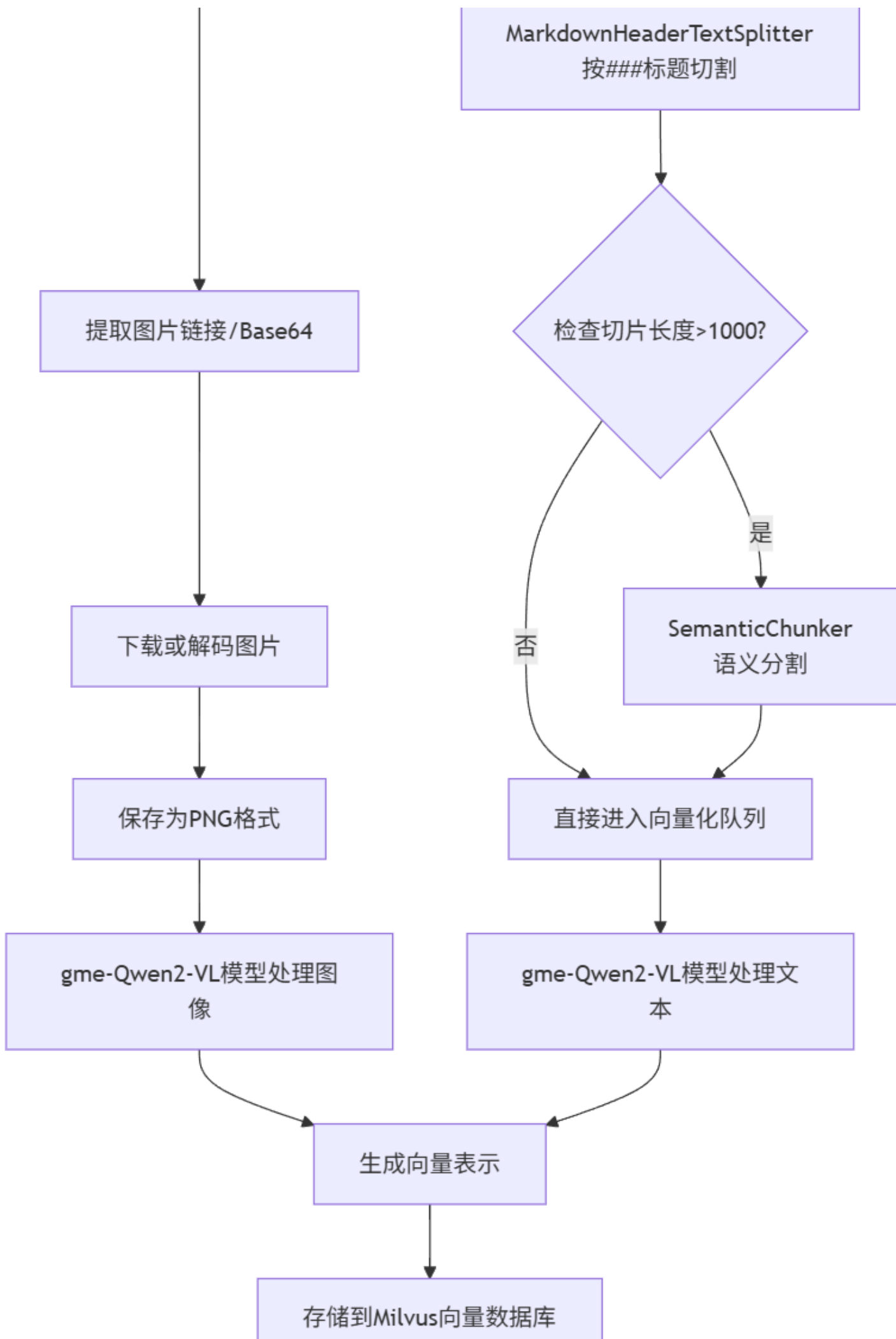
13         "filetype": "PDF",
14         "title": "机器学习基础"
15     },
16     {
17         "text": "Python是一种高级、解释型的通用编程语言。其设计哲学强调代码的可读性，语
            法允许程序员用更少的代码行表达概念。Python支持多种编程范式，包括结构化、面向对象和函数式编
            程。",
18         "category": "编程语言",
19         "filename": "python_guide.docx",
20         "filetype": "Word",
21         "title": "Python 语言特性"
22     },
23     {
24         "text": "区块链是一种分布式数据库，用于维护持续增长的记录列表，称为块。这些块使
            用加密技术链接和保护。区块链技术是比特币等加密货币的基础，但也在供应链管理、数字身份等领域
            有应用。",
25         "category": "信息技术",
26         "filename": "blockchain_overview.txt",
27         "filetype": "Text",
28         "title": "区块链技术简介"
29     }
30 ]
31
32
33 qwen3_embedding = SentenceTransformer("Qwen/Qwen3-Embedding-0.6B")
34 # resp = qwen3_embedding.encode('I like large language models.')
35 # print(resp)
36
37
38 # 为每条数据的 text 字段生成 dense 向量
39 for data in sample_data:
40     data["dense"] = qwen3_embedding.encode(data["text"])
41
42
43 print(sample_data)
44
45 # 插入数据
46 insert_result = client.insert(
47     collection_name="t_doc_collection",
48     data=sample_data
49 )
50
51 print(f"成功插入 {insert_result['insert_count']} 条数据。")
52 print(f"生成的主键 IDs: {insert_result['ids']}")

```

第七章、多模态RAG项目开发

1、多模态知识库构建流程





第八章、多模态嵌入模型：GME-Qwen2-VL-7B

GME-Qwen2-VL-7B 提供的三种嵌入方式各有其适用场景，效果好坏取决于你的具体数据和检索任务。下面是一个对比表格，帮你快速了解它们的区别和适用情况：

嵌入方式	原理	优势	适用场景
纯文本嵌入	仅对文本信息进行编码	计算高效，适合文本密集型内容检索	文档标题、推荐、纯文本检索
纯图片嵌入	仅对视觉信息进行编码	能捕捉图片的视觉特征和语义信息	以图搜图、图像检索（如“找红
文本+图片融合嵌入	同时编码关联的文本和图片信息，生成一个统一的、融合了多模态信息的向量表示	信息最丰富，能同时理解视觉和文本上下文，在复杂跨模态检索任务中通常表现最佳	图文紧密关联图片、图表、页面）

🧠 如何选择嵌入方式？

- 追求最佳检索效果：如果您的图片和文本高度相关（例如图片的标题、注释、 surrounding text），融合嵌入 (Fused-modal) 通常是更好的选择，因为它能同时捕获两种模态的信息，生成更全面的向量表示，在多模态检索基准（UMRB）上表现出色。
- 处理独立模态：如果您的数据中文本和图片关联性不强，或者您只需要进行单纯的文本搜索或图片搜索，那么相应的纯文本嵌入或纯图片嵌入更合适，且计算开销更小。获取融合嵌入中图片对应的文本

在使用 `get_fused_embeddings` 时，模型需要你同时提供要融合的文本列表 (`texts`) 和图片列表 (`images`)。关键点在于，你必须自行确保提供给 `get_fused_embeddings` 的每一张图片都有其对应的文本描述。

这意味着，你需要在代码逻辑中维护这种配对关系。模型本身不会自动从图片中提取文本（除非图片本身是包含文字的，但模型也不会直接返回给你文本内容），也不会去猜测哪段文本属于哪张图片。

1、本地推理和部署

1. 安装依赖

```
pip install -U huggingface_hub sentence_transformers
```

用于下载模型的： `huggingface-cli download --xxx --local-dir xxxx`

2. 设置环境变量

```
export HF_ENDPOINT=https://hf-mirror.com
```

3、下载模型，主要是下载配置和py文件

```
huggingface-cli download --resume-download Alibaba-NLP/gme-Qwen2-VL-2B-Instruct --local-dir /root/autodl-tmp/models/iic/gme-Qwen2-VL-2B-Instruct
```

4、配置Hugging Face 相关的顶级缓存路径

默认是：~/.cache/huggingface/hub/

```
export HF_HOME=/root/autodl-tmp/huggingface_cache
```

5、准备测试代码

代码块

```
1
2  from sentence_transformers import SentenceTransformer
3
4
5  # 指定本地模型路径
6  # model_path = "/root/autodl-tmp/models/iic/gme-Qwen2-VL-2B-Instruct" # 请替换
  为你的实际路径
7
8  # 方式1: 直接指定模型名并设置 cache_folder (如果模型已下载到本地, 且结构符合 sentence-
  transformers 的预期)
9  model = SentenceTransformer(
10      'Alibaba-NLP/gme-Qwen2-VL-2B-Instruct', # 或者使用本地路径 model_path
11      # cache_folder=model_path
12  )
13
14
15  corpus_texts = [
16      "Lambda架构中针对实时数据处理我们可以使用Spark计算框架进行分析,Spark针对实时数据进
  行分析本质是将实时流数据看成微批进行处理。",
17      "基于有状态计算的方式最大的优势是不需要将原始数据重新从外部存储中拿出来,从而进行全量计
  算,因为这种计算方式的代价可能是非常高的。",
18  ]
19
20  # 假设的图像数据路径列表 (本地路径)
21  corpus_images = [
22      "/root/autodl-tmp/code/PythonProject18/output/第一章 Apache Flink 概
  述/48deddd5ed7d0927ff5de3fa3ab7e635.png",
23      "/root/autodl-tmp/code/PythonProject18/output/第一章 Apache Flink 概
  述/52854c9de195f06b255e93ca363b15db.png",
24  ]
25
26
27  # 1. 编码文本
28  # 对于文本, 可以直接传入字符串列表
```



```
29 text_embeddings = model.encode(corpus_texts, convert_to_tensor=True) # 得到文本
    向量
30 print("文本向量形状:", text_embeddings.shape)
31
32
33 # 2. 编码图像
34 # 对于图像, GME模型通常需以特定格式传入, 例如字典表明类型
35 # 根据 GME 模型的预期输入格式, 可能需要将图像路径包装成字典
36 image_inputs = [{"image": img_path} for img_path in corpus_images]
37 image_embeddings = model.encode(image_inputs, convert_to_tensor=True) # 得到图像
    向量
38 print("图像向量形状:", image_embeddings.shape)
39
40
41
```

6、配置模型目录中的py模块路径

export PYTHONPATH=/root/autodl-tmp/models/iic/gme-Qwen2-VL-2B-Instruct:\$PYTHONPATH
python test_gme.py 最后可以执行测试代码了。

第九章、Ragas评估框架

1、Ragas框架概述：为什么需要评估RAG？

1.1 核心问题

检索增强生成（RAG）系统通常由多个组件构成（检索器、向量数据库、LLM等）。其复杂性导致评估变得困难。传统的评估方法（如人工评估）存在成本高、速度慢、主观性强且难以规模化的问题。

1.2 Ragas的解决方案

Ragas（RAG Assessment）是一个专为评估RAG和AI应用而设计的开源框架。它的核心思想是**自动化、标准化**的评估。它通过一系列精心设计的**指标**，在不依赖人工标注的情况下，科学地衡量RAG系统的性能，从而帮助开发者快速迭代和优化系统。

1.3 核心原理

Ragas基于“**以模型评估模型**”的理念。它利用LLM（如GPT-4, Deepseek, Qwen3）作为“裁判”，通过解析RAG系统运行过程中产生的各种数据（问题、上下文、答案、参考答案等），并根据预定义的规则 and 标准，对系统的各个方面进行评分。

2、核心概念解析

2.1 组件 (Components)

Ragas将评估过程抽象为几个核心组件，它们是构建评估流程的基石。

- **评估样本 (EvaluationRecord):**

评估样本是一个单一的结构化数据实例，用于评估和衡量您的LLM应用在特定场景下的性能。它代表AI应用需要处理的单个交互单元或特定用例。在 Ragas 中，评估样本使用

`SingleTurnSample` 和 `MultiTurnSample` 类来表示。

SingleTurnSample

`SingleTurnSample` 代表用户、LLM 和用于评估的预期结果之间的单轮交互。它适用于涉及单个问答对的评估和RAG评估。以下示例演示如何在基于 RAG 的应用中创建 `SingleTurnSample` 实例以评估单轮交互。在此场景中，用户提出问题，AI 提供答案。我们将创建一个 `SingleTurnSample` 实例来表示此交互，包括任何检索到的上下文、参考答案和评估标准。

- `user_input(question)`: 用户提出的问题。
- `response`: RAG系统生成的答案。
- `retrieved_contexts`: 检索器返回的用于生成答案的文档片段。
- `ground_truth (reference)`: (可选) 人工标注的标准答案，用于某些指标的评估。

代码块

```
1  from ragas import SingleTurnSample  # 导入Ragas框架中的单轮对话样本类
2
3  # 用户问题
4  user_input = "法国的首都市什么？"
5
6  # 检索到的上下文（例如从知识库或搜索引擎获取，）
7  retrieved_contexts = ["巴黎是法国的首都和人口最多的城市。"]
8
9  # AI生成的响应
10 response = "法国的首都是巴黎。"
11
12 # 参考答案（标准答案）
13 reference = "巴黎"
14
15
16 # 创建单轮对话样本实例
17 sample = SingleTurnSample(
18     user_input=user_input,  # 用户问题
19     retrieved_contexts=retrieved_contexts,  # 检索到的上下文
20     response=response,  # AI生成的响应
21     reference=reference,  # 参考答案
22 )
```

MultiTurnSample

它适用于在更复杂的交互中表示对话式agent以便进行评估（Agent和工作流的评估）。在 `MultiTurnSample` 中，`user_input` 属性表示消息序列，这些消息共同构成了人类用户和 AI 系统之间的多轮对话。这些消息是 `HumanMessage`、`AIMessage` 和 `ToolMessage` 类的实例。

- `user_input`：消息列表。
- `reference_tool_calls`：（可选）人工标注的调用工具列表，用于某些指标的评估。
- `ground_truth`（reference）：（可选）人工标注的标准答案，用于某些指标的评估。

代码块

```
1  from ragas.messages import HumanMessage, AIMessage, ToolMessage, ToolCall
   # 导入Ragas框架中的消息类型
2
3  # 用户询问纽约市天气
4  user_message = HumanMessage(content="今天纽约市的天气如何? ")
5
6  # AI决定使用天气API工具获取信息
7  ai_initial_response = AIMessage(
8      content="让我为您查询纽约市今天的天气情况。",
9      tool_calls=[ToolCall(name="WeatherAPI", args={"location": "纽约市"})] #
   工具调用参数已翻译
10 )
11
12 # 工具提供天气信息
13 tool_response = ToolMessage(content="纽约市今天晴朗，气温为75华氏度。")
14
15 # AI向用户交付最终回复
16 ai_final_response = AIMessage(content="今天纽约市天气晴朗，气温75华氏度。")
17
18 # 将所有消息组合成对话列表
19 conversation = [
20     user_message,
21     ai_initial_response,
22     tool_response,
23     ai_final_response
24 ]
25
26 from ragas import MultiTurnSample # 导入Ragas框架中的多轮对话样本类
27
28 # 用于评估的参考回复
29 reference_response = "向用户提供纽约市的当前天气信息。"
30
31 # 创建多轮对话样本实例
32 sample = MultiTurnSample(
```

```
33     user_input=conversation, # 用户输入的完整对话
34     reference=reference_response, # 参考答案
35 )
```

- **评估数据集 (EvaluationDataset):** 由多个 EvaluationRecord 组成的集合，代表整个测试集。

代码块

```
1 dataset = EvaluationDataset(samples=[sample1, sample2, sample3])
```

- **指标 (Metrics):** 衡量RAG系统特定方面性能的尺子。这是Ragas的核心。

2.2 指标 (Metrics)

Ragas提供了一套丰富的指标，可分为以下几类

指标类别	适用场景	主要指标
检索增强生成(RAG)	RAG系统评估	上下文精度、上下文召回率、忠实度等
Nvidia指标	优化LLM生成质量	答案准确性、上下文相关性等
Agent或工具使用场景	Agent工作流评估	主题一致性、工具调用准确性等
自然语言比较	生成内容质量评估	事实正确性、语义相似性等
传统非LLM指标	基础文本比较	BLEU、ROUGE、精确匹配等
SQL指标	SQL查询系统评估	基于执行的Datacompy分数、SQL查询等效
通用目的指标	多场景通用评估	方面批评、简单标准评分等
其他任务	特定任务评估	摘要评估

3、检索增强生成 (RAG)的评估

指标类别	指标名称	适用场景	评估重点
RAG指标	上下文精度	检索精准度	检索结果中相关片段的比例
RAG指标	上下文召回率	检索全面性	检索覆盖答案所需信息的比例
RAG指标	忠实度	生成内容质量	答案与检索内容的一致性
RAG指标	回答相关性	生成内容质量	答案与问题的相关度
所有指标	答案准确性	生成答案的准确率	答案与给定问题的参考标准答案之间的一致性

3.1、上下文精度 (Context Precision)：检查 检索结果的 精准度

上下文精度 (Context Precision) 是一个衡量 `retrieved_contexts` 中相关块比例的指标。它的计算方法是上下文中的每个块的 `precision@k` 的平均值。`Precision@k` 是在排名 `k` 处的相关块数量与排名 `k` 处的块总数的比率。

$$\text{Context Precision@K} = \frac{\sum_{k=1}^K (\text{Precision@k} \times v_k)}{\text{前 K 个结果中的相关项总数}}$$

其中 `K` 是 `retrieved_contexts` 中的总块数。

无参考的上下文精度

当您对某个 `user_input` 同时拥有检索到的上下文和参考上下文时，可以使用 `LLMContextPrecisionWithoutReference` 指标。为了判断检索到的上下文是否相关，此方法使用 LLM 将 `retrieved_contexts` 中存在的每个检索到的上下文或块与 `response` 进行比较。

有参考的上下文精度

当您对某个 `user_input` 同时拥有检索到的上下文和参考答案时，可以使用 `LLMContextPrecisionWithReference` 指标。为了判断检索到的上下文是否相关，此方法使用 LLM 将 `retrieved_contexts` 中存在的每个检索到的上下文或块与 `reference` 进行比较。

代码块

```
1  # 1. 创建评估样本 (SingleTurnSample)
2  # SingleTurnSample用于表示单轮对话的评估样本
3  sample = SingleTurnSample(
4      user_input=question, # 用户输入的问题
5      retrieved_contexts=[context['text'] for context in contexts], # 检索到的上下文
6      response=response, # 生成的答案
7      reference=reference # 参考答案 (用于需要参考答案的指标)
8  )
9
10 # 2. 初始化评估指标
11 # 该指标评估生成答案中与检索上下文相关部分的比例
12 if reference:
13     # 如果有参考答案，则初始化指标为LLMContextPrecisionWithReference
14     context_precision =
15     LLMContextPrecisionWithReference(llm=self.evaluator_llm)
16 else:
17     # 如果没有参考答案，则初始化指标为LLMContextPrecisionWithoutReference
18     context_precision =
19     LLMContextPrecisionWithoutReference(llm=self.evaluator_llm)
20
21 # 3. 计算评估分数 (异步方法，需要使用await)
```

```
20 score = await context_precision.single_turn_ascore(sample)
21 print(f"上下文精度评分: {score:.3f}")
```

3.2、上下文召回率（ContextRecall）

上下文召回率衡量成功检索到的相关文档（或信息片段）的数量。它侧重于不遗漏重要结果。更高的召回率意味着遗漏的相关文档更少。简而言之，召回率就是不遗漏任何重要的东西。由于召回率是关于不遗漏任何东西的，因此计算上下文召回率总是需要一个参考来比较。

`LLMContextRecall` 使用 `user_input`、`reference` 和 `retrieved_contexts` 计算，值介于 0 和 1 之间，值越高表示性能越好。

上下文召回率的计算公式如下

$$\text{上下文召回率} = \frac{\text{检索到的上下文支持的参考中的主张数量}}{\text{参考中的主张总数}}$$

代码块

```
1 # 上下文召回率，侧重于不遗漏重要结果。
2 context_recall = LLMContextRecall(llm=self.evaluator_llm)
```

3.3、响应(回答)相关性（ResponseRelevancy）

`ResponseRelevancy` 指标衡量回答与用户输入的关联程度。分数越高表示与用户输入越匹配，分数越低则表示回答不完整或包含冗余信息。

该指标使用 `user_input` 和 `response` 按如下方式计算，无需：`reference`：

$$\text{Answer Relevancy} = \frac{1}{N} \sum_{i=1}^N \text{cosine similarity}(E_{g_i}, E_o)$$

$$\text{Answer Relevancy} = \frac{1}{N} \sum_{i=1}^N \frac{E_{g_i} \cdot E_o}{\|E_{g_i}\| \|E_o\|}$$

其中

- E_{g_i} : 第 i 个生成问题的嵌入。
- E_o : 用户输入的嵌入。
- N : 生成问题的数量（默认为 3）。

如果回答直接且恰当地回应了原始问题，则被认为是相关的。此指标侧重于回答与问题意图的匹配程度，但不评估事实准确性。它会惩罚不完整或包含不必要细节的回答。

代码块

```
1 # 答案相关性, `ResponseRelevancy` 指标衡量回答与用户输入的关联程度。
2 response_relevancy = ResponseRelevancy(llm=self.evaluator_llm,
    embeddings=self.evaluator_embeddings)
3 response_relevancy_score = await response_relevancy.single_turn_ascore(sample)
4 print(f"答案相关性: {response_relevancy_score}")
```

3.4、忠诚度或者忠实度 (Faithfulness)

忠实度指标衡量了 **响应** 与 **检索到的上下文** 的事实一致性。其取值范围为 0 到 1，分数越高表示一致性越好。

如果一个响应的所有主张都能得到检索到的上下文的支持，则该响应被认为是**忠实**的。无需：
reference

计算方法如下

1. 识别响应中的所有主张。
2. 检查每个主张，看它是否可以从检索到的上下文中推断出来。
3. 使用以下公式计算忠实度分数

$$\text{忠实度分数} = \frac{\text{响应中得到检索到的上下文支持的主张数量}}{\text{响应中的主张总数}}$$

代码块

```
1 faithfulness = Faithfulness(llm=self.evaluator_llm)
```

3.5、答案准确性（AnswerAccuracy）

答案正确性的评估涉及衡量生成答案与参考答案（ground truth）相比的准确性。此评估依赖于 `ground truth`（**reference**）和 `answer`，分数范围从0到1。分数越高表示生成答案与参考答案越一致，表明正确性越好。

$$\text{F1 Score} = \frac{|\text{TP}|}{(|\text{TP}| + 0.5 \times (|\text{FP}| + |\text{FN}|))}$$

代码块

```
1  # 新版本
2  sample2 = SingleTurnSample(
3      user_input=question,
4      response=response,
5      reference=reference
6  )
7
8  answer_accuracy = AnswerAccuracy(llm=self.evaluator_llm)
9  answer_accuracy_score = await answer_accuracy.single_turn_ascore(sample2)
10 print(f"答案准确性（度）：{answer_accuracy_score}")
11
12
13 # 老版本（非常慢）
14 data_samples = {
15     'question': [question],
16     'answer': [response],
17     'ground_truth': [reference]
18 }
19 dataset = Dataset.from_dict(data_samples)
20 answer_correctness_score = evaluate(dataset, metrics=
    [answer_correctness], llm=self.evaluator_llm,
    embeddings=self.evaluator_embeddings)
21 print(f"答案正确性：{answer_correctness_score}")
```

第十章、项目核心，难点和重点

- 1、降低幻觉
- 2、提高响应速度

上下文工程+评估

聊天历史记录： BaseMessage--->

短期记忆存储: 一个会话内的所有的聊天历史记录存储的方案。

长期记忆存储： 从系统上线开始所有的历史聊天记录的存储方案。---关系型数据库

三个问题？

